

Grammars & Top-down Parsing

CptS 355 — Fall 2026

Thomas Gilray

$$L = \{ \mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N} \}$$

$$L = \{\mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N}\}$$

$$S \rightarrow \mathbf{a} S \mathbf{b} \mid \epsilon$$

$$L = \{\mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N}\}$$

$$S \rightarrow \mathbf{a} S \mathbf{b} \mid \epsilon$$



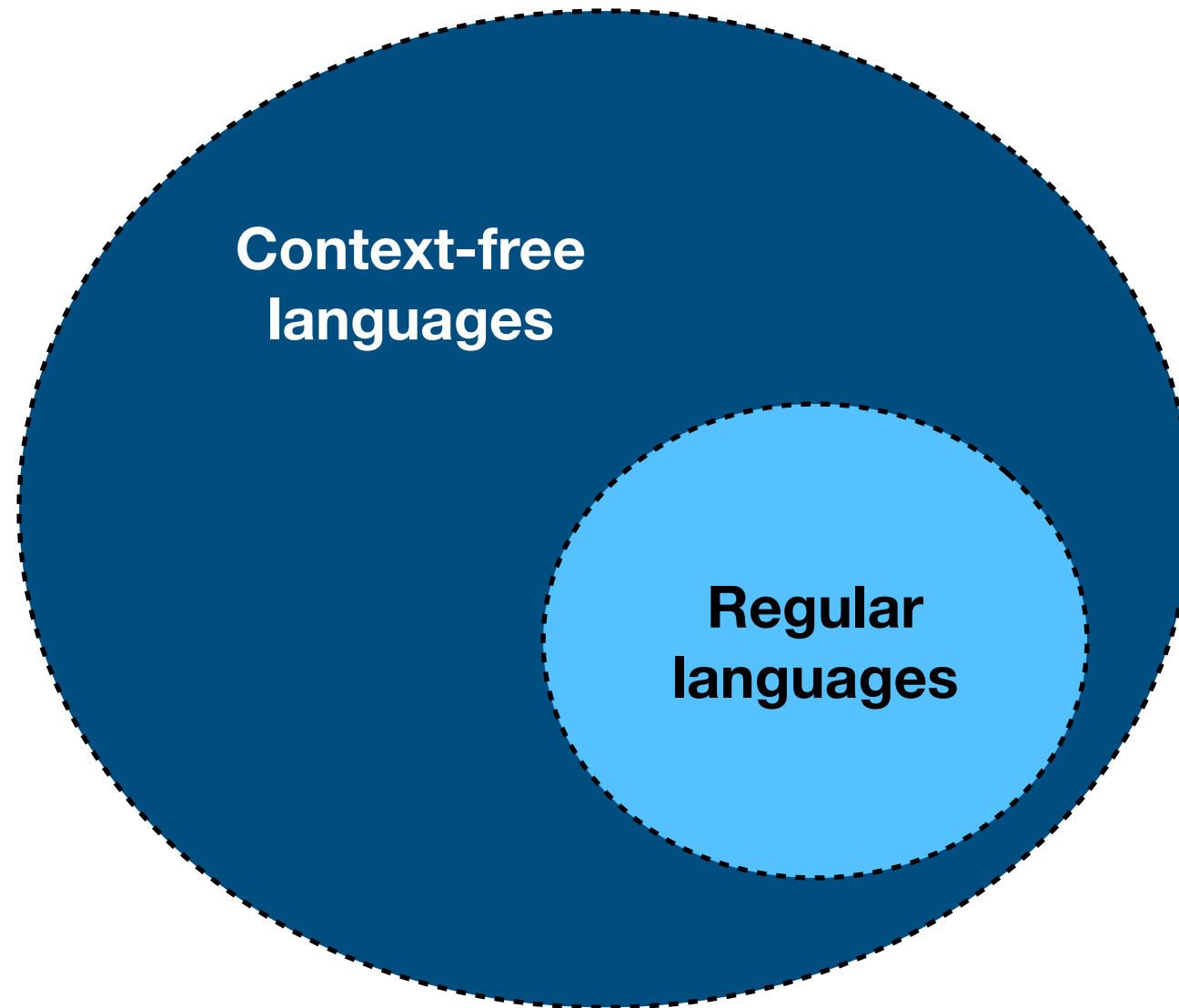
The empty string is in S.

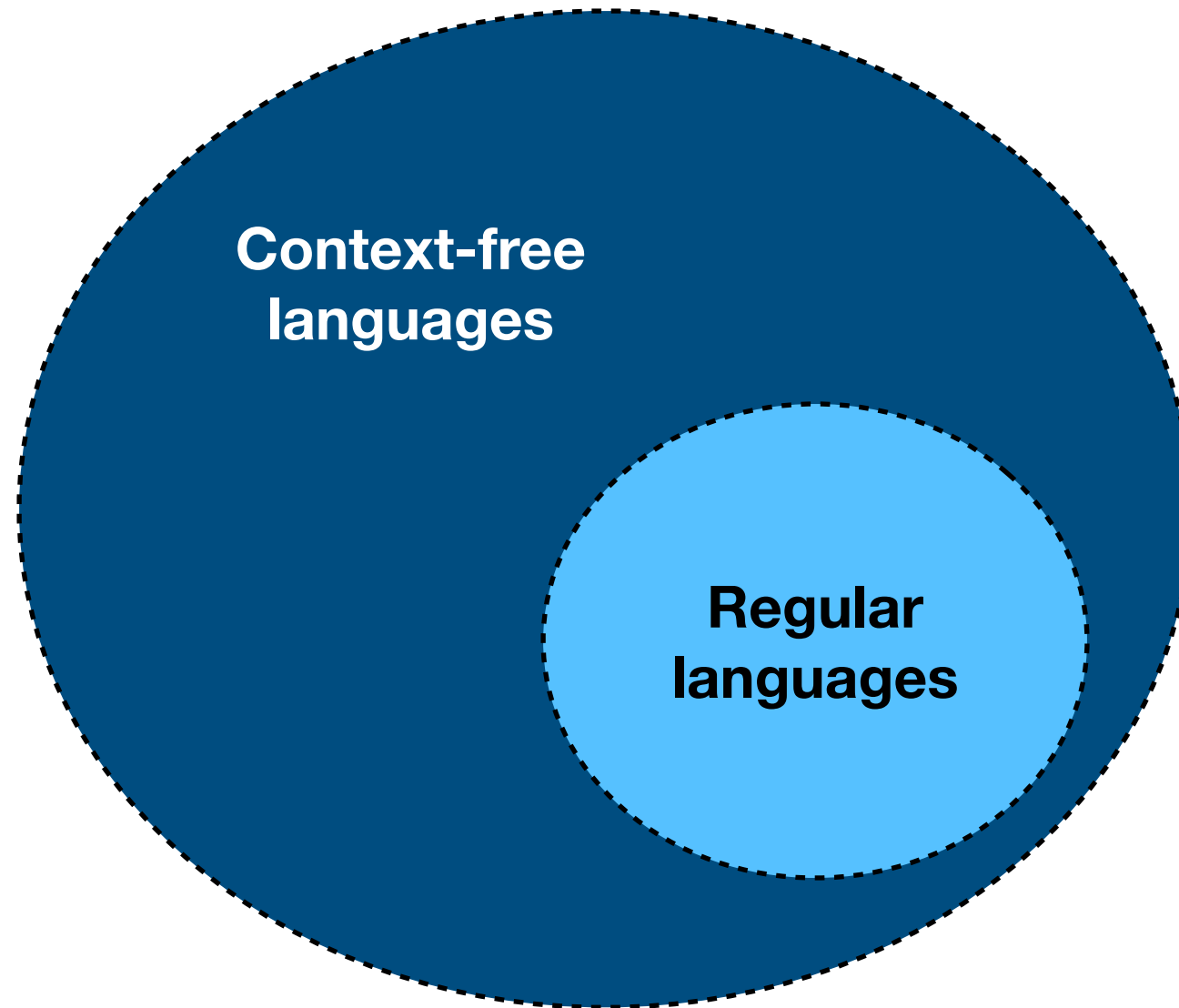
$$L = \{\mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N}\}$$

$$S \rightarrow \mathbf{a} S \mathbf{b} \mid \epsilon$$

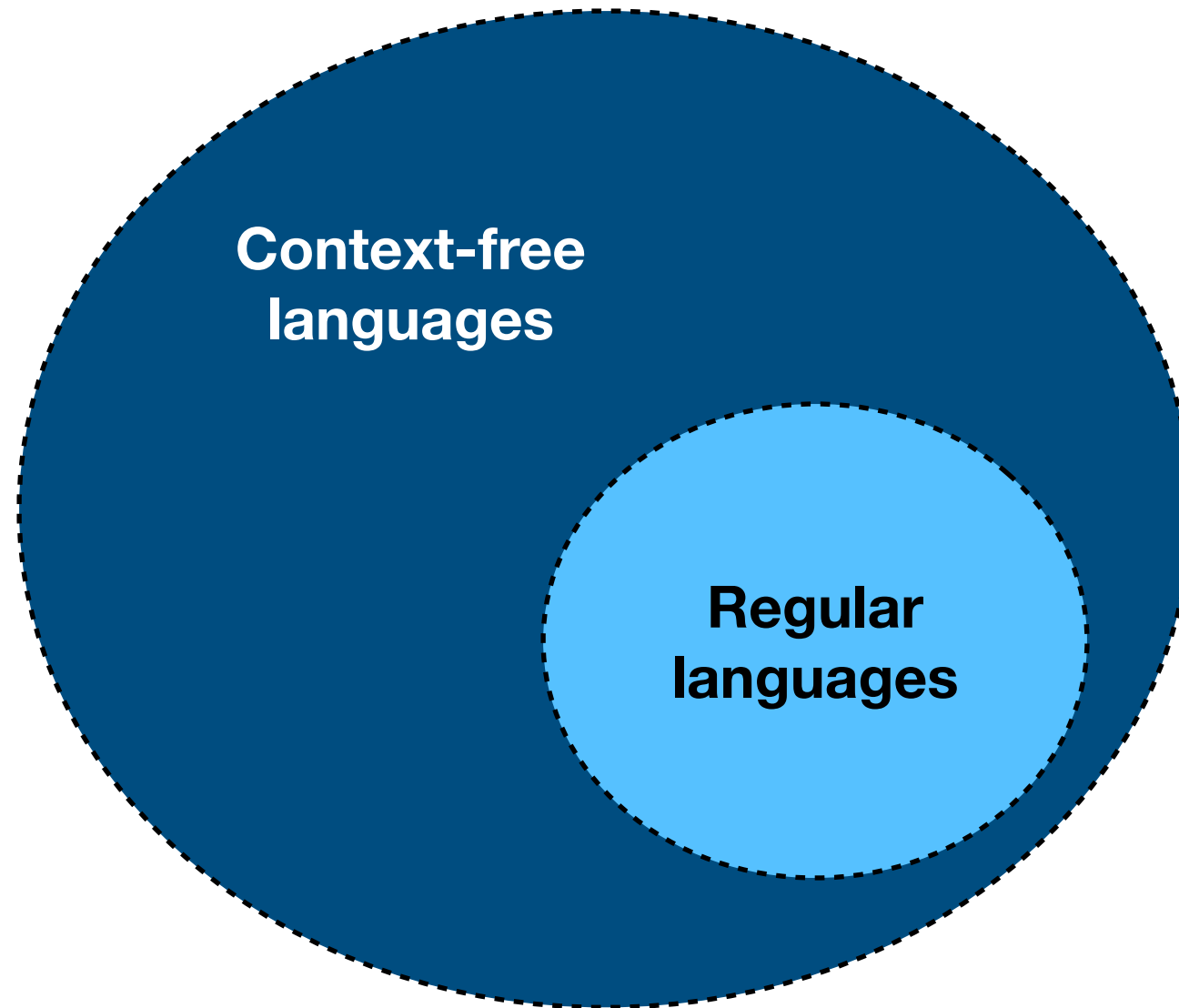

Any string s in the language (denoted by) S may be prepended with **a** and appended with **b** and the resulting string is also in the language S .

The empty string is in S .





RE (Turing-equivalent computation)



RE (Turing-equivalent computation)

Formally, a CFG is a 4-tuple:

$$\langle \Sigma, N, P, S \rangle, \text{ where}$$

Σ is a set of strings/symbols (alphabet) the grammar operates over. These are called the **terminals** of the grammar. E.g., "+", "void", "foo". In toy examples these will often be chars, but in practice: keywords, IDs.

N is a finite set of symbols called **nonterminals**. Often written as upper-case letters (esp. in class/book). Must be disjoint with the set of terminals.

P is a finite set of **productions** that take the form: $N(\Sigma \mid N)^*$ for which we use the notation: $N \rightarrow (\Sigma \mid N)^*$

S is a designated nonterminal called the **start symbol**.

Formalizing the CFG: $S \rightarrow \mathbf{aSb} \mid \epsilon$

$\langle \Sigma, N, P, S \rangle$, where

$$\Sigma = \{\mathbf{a, b}\}$$

$$N = \{S\}$$

$$P = \{S\mathbf{aSb}, S\}$$

Notational Conventions:

$$S \rightarrow \mathbf{aSb} \\ \quad \quad | \epsilon$$

Notational Conventions:

$$S \rightarrow \mathbf{aSb} \\ | \epsilon$$

The same as writing
a separate production:

$$S \rightarrow \\ \text{or} \\ S \rightarrow \epsilon$$

- Productions with the same LHS will often be joined using “|”.

Notational Conventions:

$$S \rightarrow \mathbf{aSb} \\ | \epsilon$$

The same as writing
a separate production:

$$S \rightarrow \\ \text{or} \\ S \rightarrow \epsilon$$

- Productions with the same LHS will often be joined using “|”.
- If no start symbol is specified, it is assumed to be the LHS of the first production listed. (Will often be S in toy examples.)

Notational Conventions:

$$S \rightarrow \mathbf{aSb} \\ | \epsilon$$

The same as writing
a separate production:

$$S \rightarrow \\ \text{or} \\ S \rightarrow \epsilon$$

- Productions with the same LHS will often be joined using “|”.
- If no start symbol is specified, it is assumed to be the LHS of the first production listed. (Will often be S in toy examples.)
- If a RHS is empty, this means epsilon/empty-string.

Backus-Naur Form

Backus-Naur Form

- Context-free grammars often use Backus-Naur Form (BNF):

Backus-Naur Form

- Context-free grammars often use Backus-Naur Form (BNF):
 - Invented by John Backus and Peter Naur in order to describe the syntax of Algol in 1962.

Backus-Naur Form

- Context-free grammars often use Backus-Naur Form (BNF):
 - Invented by John Backus and Peter Naur in order to describe the syntax of Algol in 1962.
- Nonterminals are often written with angle brackets around them and productions use “ $::=$ ” in place of “ $\rightarrow$ ”.

Backus-Naur Form

- Context-free grammars often use Backus-Naur Form (BNF):
 - Invented by John Backus and Peter Naur in order to describe the syntax of Algol in 1962.
- Nonterminals are often written with angle brackets around them and productions use “ $::=$ ” in place of “ $\rightarrow$ ”.
- The productions $S \rightarrow \mathbf{aSb} \mid \epsilon$ might instead be written:

$\langle S \rangle ::= a\langle S \rangle b \mid ;$

Backus-Naur Form

- Context-free grammars often use Backus-Naur Form (BNF):
 - Invented by John Backus and Peter Naur in order to describe the syntax of Algol in 1962.
- Nonterminals are often written with angle brackets around them and productions use “ $::=$ ” in place of “ $\rightarrow$ ”.
- The productions $S \rightarrow \mathbf{aSb} \mid \epsilon$ might instead be written:
$$\langle S \rangle ::= a\langle S \rangle b \mid ;$$
- There are many slightly different flavors of BNF (EBNF, etc).

Grammars are generative (via induction)

Grammars are generative (via induction)

- We can think of a grammar G as ***generating*** a set of strings. CFGs are often called ***generative grammars***.

Grammars are generative (via induction)

- We can think of a grammar G as ***generating*** a set of strings. CFGs are often called ***generative grammars***.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

$$S \Rightarrow \mathbf{aS} \quad // \text{ Using first production}$$

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

$$S \Rightarrow \mathbf{aS}$$

// Using first production

$$\Rightarrow \mathbf{aaS}$$

// Using first production again

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

$S \Rightarrow \mathbf{aS}$	// Using first production
$\Rightarrow \mathbf{aaS}$	// Using first production again
$\Rightarrow \mathbf{aacS}$	// Using third production

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

$S \Rightarrow \mathbf{aS}$	// Using first production
$\Rightarrow \mathbf{aaS}$	// Using first production again
$\Rightarrow \mathbf{aacS}$	// Using third production
$\Rightarrow \mathbf{aacbS}$	// Using second production

Grammars are generative (via induction)

- We can think of a grammar G as *generating* a set of strings. CFGs are often called *generative grammars*.
- Consider the grammar: $S \rightarrow \mathbf{aS} \mid \mathbf{bS} \mid \mathbf{cS} \mid \epsilon$
- We can generate the string **aacb** by applying our productions as follows:

$S \Rightarrow \mathbf{aS}$	// Using first production
$\Rightarrow \mathbf{aaS}$	// Using first production again
$\Rightarrow \mathbf{aacS}$	// Using third production
$\Rightarrow \mathbf{aacbS}$	// Using second production
$\Rightarrow \mathbf{aacb}$	// Using last production

Checking for *acceptance*

Checking for *acceptance*

- **Informally:** we can check if a string s is in the language $L(G)$ by finding a ***rewriting*** (*using only G 's productions*) from the start symbol of G to a string of terminals that matches s .

Checking for *acceptance*

- **Informally:** we can check if a string s is in the language $L(G)$ by finding a ***rewriting*** (*using only G 's productions*) from the start symbol of G to a string of terminals that matches s .
 - Only rewrites that start with G 's start symbol and end with a string *containing no nonterminal symbols* is valid.

Checking for *acceptance*

- **Informally:** we can check if a string s is in the language $L(G)$ by finding a ***rewriting*** (*using only G 's productions*) from the start symbol of G to a string of terminals that matches s .
 - Only rewrites that start with G 's start symbol and end with a string *containing no nonterminal symbols* is valid.
 - Such a valid sequence of rewrites is called a ***derivation*** or ***parse*** of the string.

Checking for *acceptance*

- **Informally:** we can check if a string s is in the language $L(G)$ by finding a ***rewriting*** (*using only G 's productions*) from the start symbol of G to a string of terminals that matches s .
 - Only rewrites that start with G 's start symbol and end with a string *containing no nonterminal symbols* is valid.
 - Such a valid sequence of rewrites is called a ***derivation*** or ***parse*** of the string.
 - If no such derivation exists for a string, it is not in $L(G)$.

Checking for *acceptance*

- **Informally:** we can check if a string s is in the language $L(G)$ by finding a ***rewriting*** (*using only G 's productions*) from the start symbol of G to a string of terminals that matches s .
 - Only rewrites that start with G 's start symbol and end with a string *containing no nonterminal symbols* is valid.
 - Such a valid sequence of rewrites is called a ***derivation*** or ***parse*** of the string.
 - If no such derivation exists for a string, it is not in $L(G)$.
- The process of discovering a derivation for a string is ***parsing***!

Notation for derivations

$\Rightarrow$ indicates one step of a derivation (one rewrite using one production)

$\Rightarrow^*$ indicates zero-or-more steps of a derivation

$\Rightarrow^+$ indicates one-or-more steps of a derivation

$$S \rightarrow (S) \mid \epsilon$$

For example, for the string $(((()))$:

$$S \Rightarrow (S) \Rightarrow ((S)) \Rightarrow (((S))) \Rightarrow ((((S)))) \Rightarrow ((((()))$$

$$S \Rightarrow (S) \Rightarrow^* (S) \Rightarrow^+ ((()))$$

The language for a CFG

$$L(G) = \{s \in \Sigma^* \mid S \Rightarrow^+ s\}$$

Where Σ is the set of terminals for G ,
S is the start state for G ,
and $\Rightarrow$ is the step (derivation) relation for G .

Try an example. Which strings match the following CFG:

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

ababab

aacccc

ccc

abdbbb

bbcc

acb

Try an example. Which strings match the following CFG:

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

ababab

aacccc

ccc

abdbbb

bbcc

acb

Try an example. Which of the following is a derivation of **aabc**?

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aab}S \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$$

$$S \Rightarrow \mathbf{a}T \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aa}V \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}$$

Try an example. Which of the following is a derivation of **aabc**?

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aab}S \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V$$

$$S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$$

✓ $S \Rightarrow \mathbf{a}S \Rightarrow \mathbf{aa}S \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aab}T \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}V \Rightarrow \mathbf{aabc}$

$$S \Rightarrow \mathbf{a}T \Rightarrow \mathbf{aa}T \Rightarrow \mathbf{aa}V \Rightarrow \mathbf{aab}V \Rightarrow \mathbf{aabc}$$

Try an example. Can this grammar be written as a regular expression instead?

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

Try an example. Can this grammar be written as a regular expression instead?

$$\begin{aligned} S &\rightarrow \mathbf{a}S \mid T \\ T &\rightarrow \mathbf{b}T \mid V \\ V &\rightarrow \mathbf{c}V \mid \epsilon \end{aligned}$$

$$\mathbf{a^*b^*c^*}$$

YES!

Try an example. Write a CFG for the regular expression **aa^*** :

Try an example. Write a CFG for the regular expression **aa^*** :

$$S \rightarrow aS \mid a$$

Principles of designing CFGs

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**
 - $\mathbf{a}^*$ can be written: $A \rightarrow \mathbf{aA} \mid \epsilon$

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**
 - $\mathbf{a^*}$ can be written: $A \rightarrow \mathbf{aA} \mid \epsilon$
 - $\mathbf{b^+}$ can be written: $B \rightarrow \mathbf{bB} \mid \mathbf{b}$

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**
 - $\mathbf{a^*}$ can be written: $A \rightarrow \mathbf{aA} \mid \epsilon$
 - $\mathbf{b^+}$ can be written: $B \rightarrow \mathbf{bB} \mid \mathbf{b}$
- Then use different nonterminals to produce disjoint parts of the language you want and combine them in a production.

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**
 - $\mathbf{a^*}$ can be written: $A \rightarrow \mathbf{aA} \mid \epsilon$
 - $\mathbf{b^+}$ can be written: $B \rightarrow \mathbf{bB} \mid \mathbf{b}$
- Then use different nonterminals to produce disjoint parts of the language you want and combine them in a production.
 - $\mathbf{a^* \mid b^+}$ can be written: $S \rightarrow A \mid B$

Principles of designing CFGs

- To generate an arbitrary-lengthed string, use **recursion!**
 - $\mathbf{a^*}$ can be written: $A \rightarrow \mathbf{aA} \mid \epsilon$
 - $\mathbf{b^+}$ can be written: $B \rightarrow \mathbf{bB} \mid \mathbf{b}$
- Then use different nonterminals to produce disjoint parts of the language you want and combine them in a production.
 - $\mathbf{a^* \mid b^+}$ can be written: $S \rightarrow A \mid B$
 - $\mathbf{a^*b^+}$ can be written: $S \rightarrow AB$

Principles of designing CFGs

Principles of designing CFGs

- To generate languages with matching, balanced, or related numbers of symbols, use a nonterminal between terminals:

Principles of designing CFGs

- To generate languages with matching, balanced, or related numbers of symbols, use a nonterminal between terminals:
 - $\{\mathbf{a^n b^n} \mid n > 0\}$ can be written:

$$S \rightarrow \mathbf{a S b} \mid \mathbf{ab}$$

Principles of designing CFGs

- To generate languages with matching, balanced, or related numbers of symbols, use a nonterminal between terminals:
 - $\{\mathbf{a^n b^n} \mid n > 0\}$ can be written:

$$S \rightarrow \mathbf{aSb} \mid \mathbf{ab}$$

- $\{\mathbf{a^n b^{3n}} \mid n \geq 0\}$ can be written:

$$S \rightarrow \mathbf{aSbbb} \mid \epsilon$$

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mid n \geq m \}$:

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mid n \geq m \}$:

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mid n \geq m \}$:

$$S \rightarrow \mathbf{x}S\mathbf{y} \mid \mathbf{x}S \mid \epsilon$$

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mathbf{z}^k \mid n \geq 2m \text{ and } k > 1 \}$:

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mathbf{z}^k \mid n \geq 2m \text{ and } k > 1 \}$:

Try an example. Write a CFG for $\{ \mathbf{x}^n \mathbf{y}^m \mathbf{z}^k \mid n \geq 2m \text{ and } k > 1 \}$:

$$S \rightarrow XZ$$

$$X \rightarrow \mathbf{xxXy} \mid \mathbf{xX} \mid \epsilon$$

$$Z \rightarrow \mathbf{zZ} \mid \mathbf{zz}$$

Why do we want context-free grammars?

Why do we want context-free grammars?

- ***Context-free grammars*** (CFGs) denote formal languages (sets of strings). We will continue to use $\mathcal{L}(G)$ to denote the language itself (in this case for a grammar G).

Why do we want context-free grammars?

- **Context-free grammars** (CFGs) denote formal languages (sets of strings). We will continue to use $\mathcal{L}(G)$ to denote the language itself (in this case for a grammar G).
- The languages we can denote with (context-free) grammars are called **context-free languages** (CFLs).

Why do we want context-free grammars?

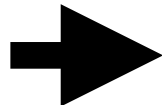
- **Context-free grammars** (CFGs) denote formal languages (sets of strings). We will continue to use $\mathcal{L}(G)$ to denote the language itself (in this case for a grammar G).
- The languages we can denote with (context-free) grammars are called **context-free languages** (CFLs).
- Context-free grammars are *strictly* more expressive than regular expressions (CFGs subsume REs, NFAs, DFAs). *Every RE can be written as a CFG, but many CFGs cannot be written as REs.*

Why do we want context-free grammars?

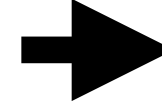
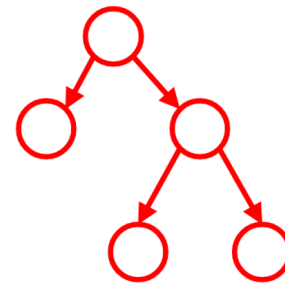
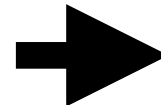
- **Context-free grammars** (CFGs) denote formal languages (sets of strings). We will continue to use $\mathcal{L}(G)$ to denote the language itself (in this case for a grammar G).
- The languages we can denote with (context-free) grammars are called **context-free languages** (CFLs).
- Context-free grammars are *strictly* more expressive than regular expressions (CFGs subsume REs, NFAs, DFAs). *Every RE can be written as a CFG, but many CFGs cannot be written as REs.*
- **Crucially:** CFGs both *match* strings and *endow them with a tree structure*. This makes CFGs very useful for defining a parser, e.g., for interpreting/compiling a programming language!

Typical (big-picture) organization of a compiler:

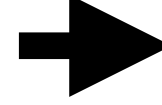
Compiler organization



Front-end:
Lexing
Parsing



**Middle &
Back-end:**
Desugaring
Simplification
Analysis
Code emission

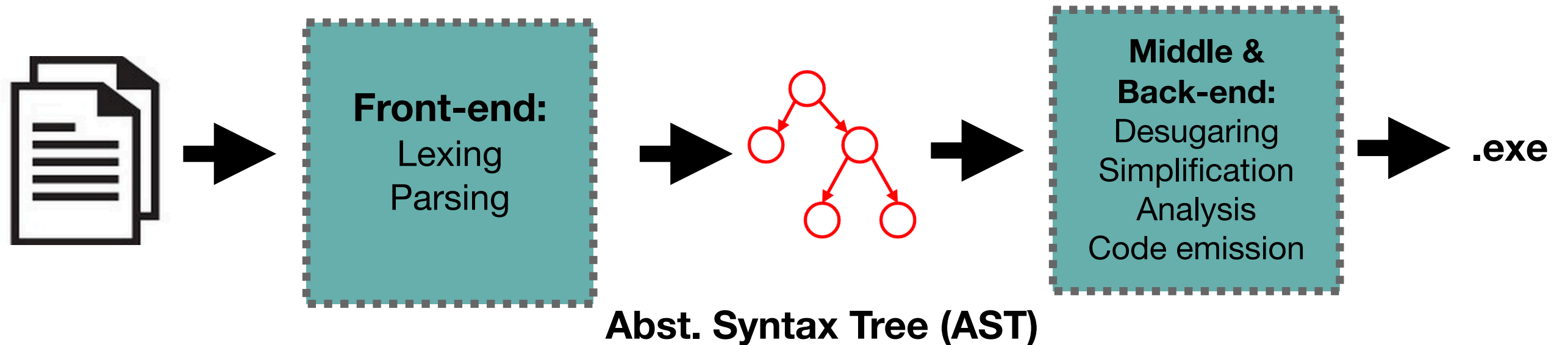


.exe

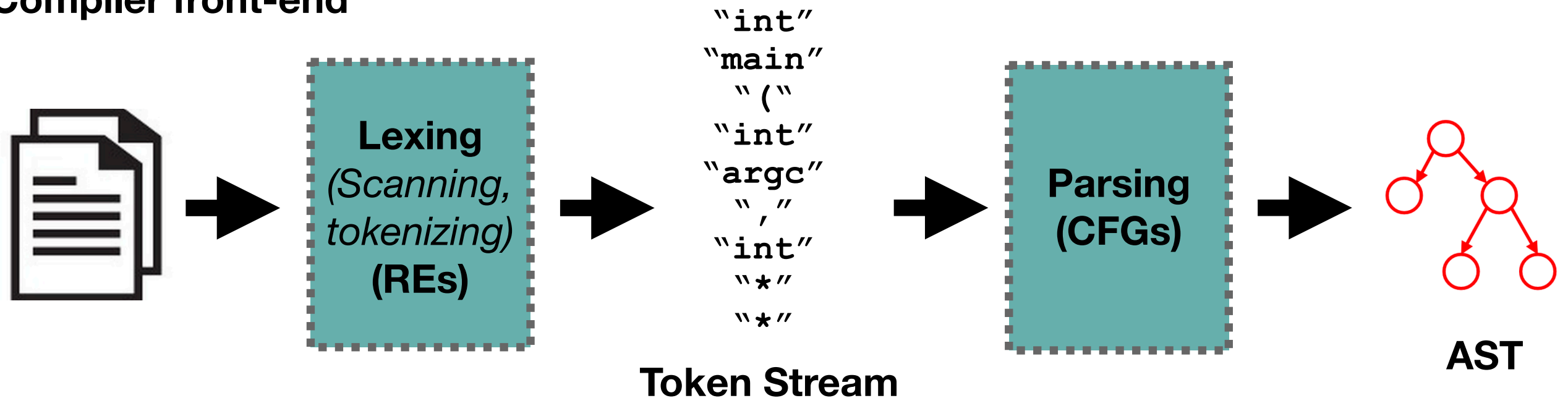
Abst. Syntax Tree (AST)

Typical (big-picture) organization of a compiler:

Compiler organization



Compiler front-end



Lexical Analysis (i.e., Tokenizing, Scanning)

Lexical Analysis (i.e., Tokenizing, Scanning)

- ***Lexical analysis*** (also called ***tokenizing***, ***scanning***) is a process that breaks the program text into a sequence/stream of ***lexemes*** or ***tokens***: the smallest sensical units of text in the language.

Lexical Analysis (i.e., Tokenizing, Scanning)

- ***Lexical analysis*** (also called ***tokenizing***, ***scanning***) is a process that breaks the program text into a sequence/stream of ***lexemes*** or ***tokens***: the smallest sensical units of text in the language.
- Typically, tokenizing ignores whitespace except as delineation for non-whitespace tokens (and in some languages such as Python).

Lexical Analysis (i.e., Tokenizing, Scanning)

- ***Lexical analysis*** (also called ***tokenizing***, ***scanning***) is a process that breaks the program text into a sequence/stream of ***lexemes*** or ***tokens***: the smallest sensical units of text in the language.
- Typically, tokenizing ignores whitespace except as delineation for non-whitespace tokens (and in some languages such as Python).
- Tokens match a regular expression and commonly include symbols (“+”, “*”, “=”, “:”), identifiers (“baz44”, “Foo_Bar”), numbers (5, 4.4, 0x1F3A), and keywords (“if”, “else”, “return”).

Lexical Analysis (i.e., Tokenizing, Scanning)

- **Lexical analysis** (also called **tokenizing**, **scanning**) is a process that breaks the program text into a sequence/stream of **lexemes** or **tokens**: the smallest sensical units of text in the language.
- Typically, tokenizing ignores whitespace except as delineation for non-whitespace tokens (and in some languages such as Python).
- Tokens match a regular expression and commonly include symbols (“+”, “*”, “=”, “:”), identifiers (“baz44”, “Foo_Bar”), numbers (5, 4.4, 0x1F3A), and keywords (“if”, “else”, “return”).
- The set of possible tokens corresponds to the **terminal** set or alphabet of the context-free grammar used for parsing.

Parsing with CFGs

Parsing with CFGs

- CFGs give us a way to specify CFLs, but they do not provide a ready *algorithm* for accepting/structuring strings.

Parsing with CFGs

- CFGs give us a way to specify CFLs, but they do not provide a ready *algorithm* for accepting/structuring strings.
- This process is called ***parsing***, and there are many algorithms which have been developed. Each operates on a *restricted* form of context-free grammar. E.g.,
 - **LL(k) parsing** (next week), LR(k), LALR(k), SLR(k), etc...

Parsing with CFGs

- CFGs give us a way to specify CFLs, but they do not provide a ready *algorithm* for accepting/structuring strings.
- This process is called ***parsing***, and there are many algorithms which have been developed. Each operates on a *restricted* form of context-free grammar. E.g.,
 - **LL(k) parsing** (next week), LR(k), LALR(k), SLR(k), etc...
- Many good tools exist for doing this for us. Most languages have multiple such libraries. Some notable examples are:
 - YACC, JavaCC, ANTLR, PLY, etc...

Parsing

Parsing

- ***Parsing*** is the task of using a grammar to find a particular ***derivation*** for a given string (a sequence of production rules which rewrite the grammar's start symbol to that input string).

Parsing

- ***Parsing*** is the task of using a grammar to find a particular ***derivation*** for a given string (a sequence of production rules which rewrite the grammar's start symbol to that input string).
- This *both* decides whether the string is in the language *and* endows the string (i.e., a sequence of tokens) with a tree structure.

Parsing

- **Parsing** is the task of using a grammar to find a particular **derivation** for a given string (a sequence of production rules which rewrite the grammar's start symbol to that input string).
- This both decides whether the string is in the language *and* endows the string (i.e., a sequence of tokens) with a tree structure.
 - Adds a logical grouping (*think: invisible dotted parentheses*) at each use of a production rule to rewrite a nonterminal.

Parsing

- **Parsing** is the task of using a grammar to find a particular **derivation** for a given string (a sequence of production rules which rewrite the grammar's start symbol to that input string).
- This both decides whether the string is in the language *and* endows the string (i.e., a sequence of tokens) with a tree structure.
 - Adds a logical grouping (*think: invisible dotted parentheses*) at each use of a production rule to rewrite a nonterminal.
- This **parse tree** visualizes the structure of a derivation: each nonterminal has child nodes listing the symbols that replaced it.

Parsing

- **Parsing** is the task of using a grammar to find a particular **derivation** for a given string (a sequence of production rules which rewrite the grammar's start symbol to that input string).
- This both decides whether the string is in the language *and* endows the string (i.e., a sequence of tokens) with a tree structure.
 - Adds a logical grouping (*think: invisible dotted parentheses*) at each use of a production rule to rewrite a nonterminal.
- This **parse tree** visualizes the structure of a derivation: each nonterminal has child nodes listing the symbols that replaced it.
- An **abstract syntax tree (AST)** is a more compact data-structure defined by a parse (e.g., ASTs may exclude nodes with one child).

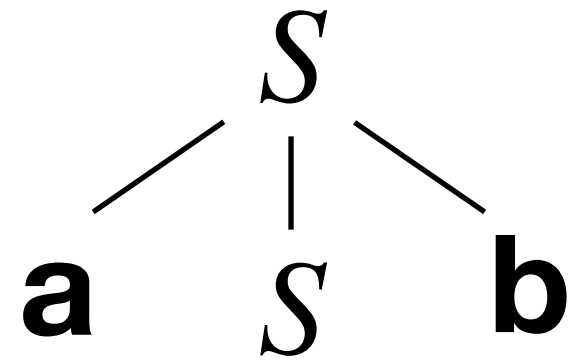
Derivations and Parse Trees

$$S \rightarrow aSb \mid \epsilon$$

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaasbbb \\ &\Rightarrow aaabbb \end{aligned}$$

Derivations and Parse Trees

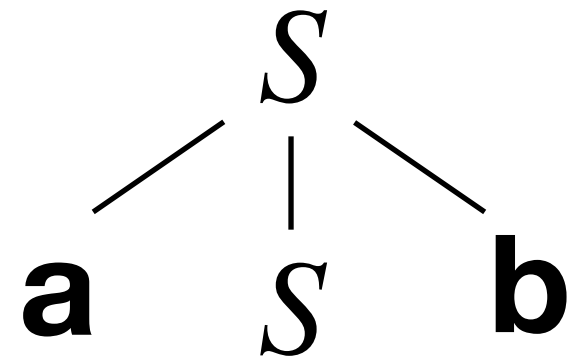
$$S \rightarrow aSb \mid \epsilon$$



Derivations and Parse Trees

$$S \rightarrow aSb \mid \epsilon$$

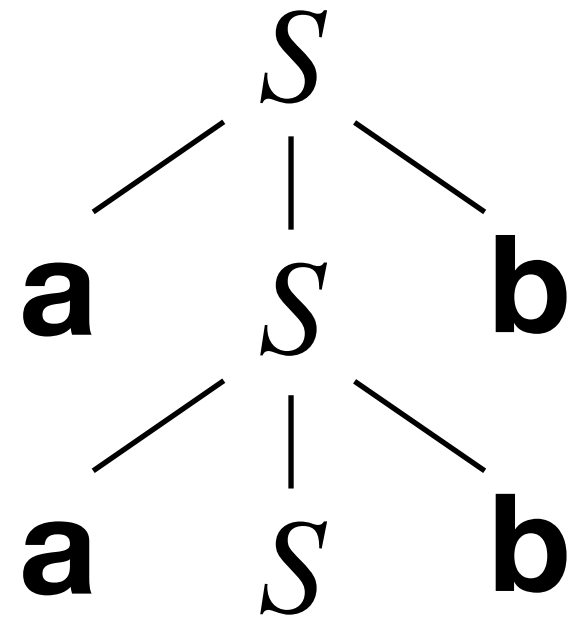
$$S \Rightarrow aSb$$



Derivations and Parse Trees

$$S \rightarrow aSb \mid \epsilon$$

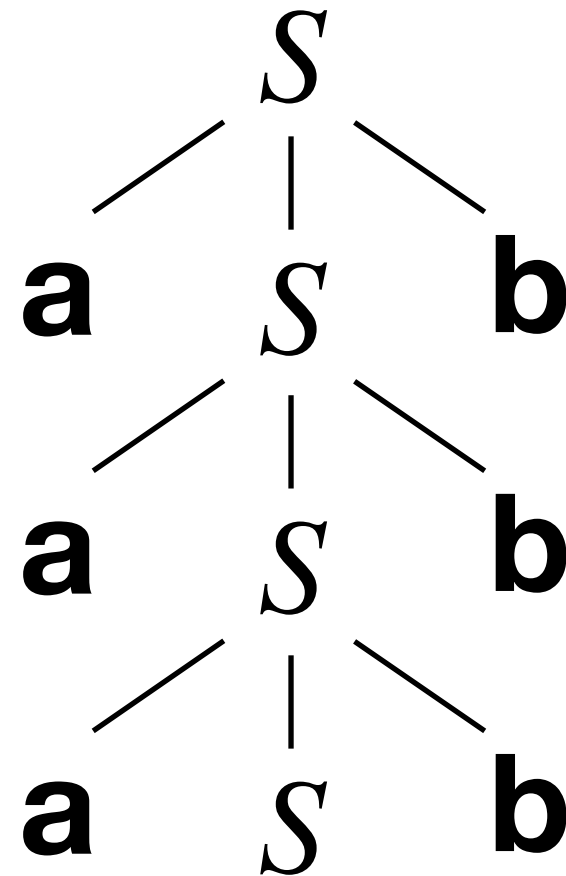
$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \end{aligned}$$



Derivations and Parse Trees

$$S \rightarrow aSb \mid \epsilon$$

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaaSbbb \end{aligned}$$



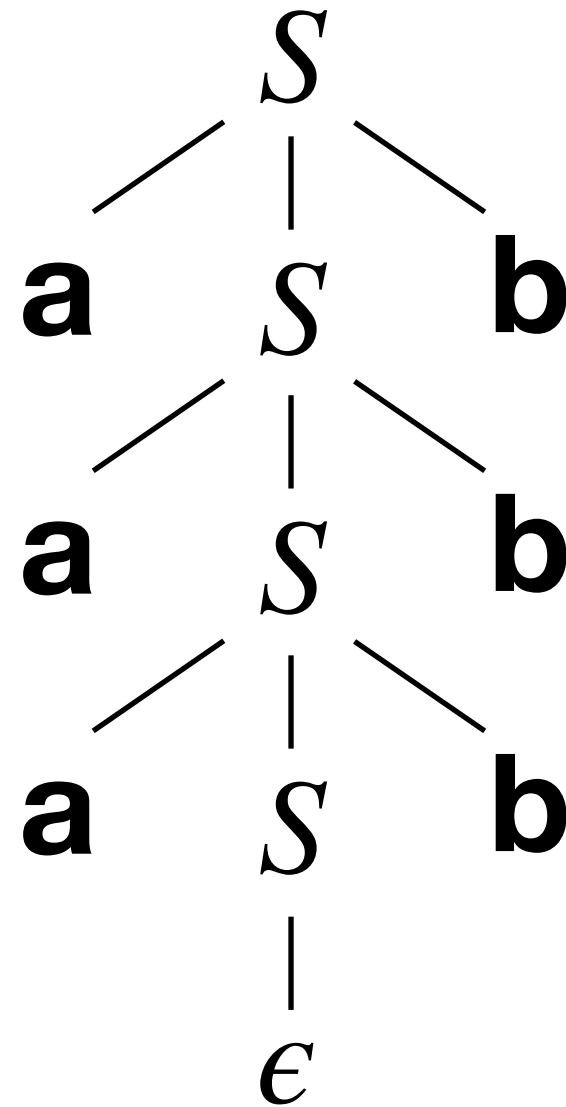
Derivations and Parse Trees

$$S \rightarrow aSb \mid \epsilon$$

$S \Rightarrow aSb$
 $\Rightarrow aaSbb$
 $\Rightarrow aaasbbb$
 $\Rightarrow aaabbbb$

(((())))

Dotted “invisible” parentheses shows
the tree structure of a parse.



Parse Trees

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.
 - Internal nodes are nonterminal symbols.

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.
 - Internal nodes are nonterminal symbols.
 - The children of each internal node are the RHS symbols of the applied production rule for that nonterminal.

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.
 - Internal nodes are nonterminal symbols.
 - The children of each internal node are the RHS symbols of the applied production rule for that nonterminal.
 - Leaf nodes are terminal symbols (or ϵ).

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.
 - Internal nodes are nonterminal symbols.
 - The children of each internal node are the RHS symbols of the applied production rule for that nonterminal.
 - Leaf nodes are terminal symbols (or ϵ).
- In-order traversal of leaf nodes (left-to-right) reveals the string.

Parse Trees

- ***Parse trees*** show how a string is *generated* by a grammar.
 - The root node is the start symbol.
 - Internal nodes are nonterminal symbols.
 - The children of each internal node are the RHS symbols of the applied production rule for that nonterminal.
 - Leaf nodes are terminal symbols (or ϵ).
- In-order traversal of leaf nodes (left-to-right) reveals the string.
- Each derivation has a unique parse tree; each string does not!

Parse Tree Example

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Parse Tree Example

$$E \rightarrow n$$

$$E \Rightarrow E * E$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Parse Tree Example

$$E \rightarrow n$$

$$E \Rightarrow E * E$$

$$\Rightarrow (E) * E$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Parse Tree Example

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

$$E \Rightarrow E * E$$

$$\Rightarrow (E) * E$$

$$\Rightarrow (E - E) * E$$

Parse Tree Example

$$\begin{array}{l} E \rightarrow n \\ \quad | \\ \quad E + E \\ \quad | \\ \quad E - E \\ \quad | \\ \quad E * E \\ \quad | \\ \quad (E) \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l} E \Rightarrow E * E \\ \Rightarrow (E) * E \\ \Rightarrow (E - E) * E \\ \Rightarrow (E + E - E) * E \end{array}$$

Parse Tree Example

$$\begin{array}{l}
 E \rightarrow n \\
 \quad | \\
 \quad E + E \\
 \quad | \\
 \quad E - E \\
 \quad | \\
 \quad E * E \\
 \quad | \\
 \quad (E) \\
 \text{(where } n \in \mathbb{N}\text{)}
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E * E \\
 \Rightarrow (E) * E \\
 \Rightarrow (E - E) * E \\
 \Rightarrow (E + E - E) * E \\
 \Rightarrow (8 + E - E) * E
 \end{array}$$

Parse Tree Example

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E)
 \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l}
 E \Rightarrow E * E \\
 \Rightarrow (E) * E \\
 \Rightarrow (E - E) * E \\
 \Rightarrow (E + E - E) * E \\
 \Rightarrow (8 + E - E) * E \\
 \Rightarrow (8 + 7 - E) * E
 \end{array}$$

Parse Tree Example

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E)
 \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l}
 E \Rightarrow E * E \\
 \Rightarrow (E) * E \\
 \Rightarrow (E - E) * E \\
 \Rightarrow (E + E - E) * E \\
 \Rightarrow (8 + E - E) * E \\
 \Rightarrow (8 + 7 - E) * E \\
 \Rightarrow (8 + 7 - 5) * E
 \end{array}$$

Parse Tree Example

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E)
 \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l}
 E \Rightarrow E * E \\
 \Rightarrow (E) * E \\
 \Rightarrow (E - E) * E \\
 \Rightarrow (E + E - E) * E \\
 \Rightarrow (8 + E - E) * E \\
 \Rightarrow (8 + 7 - E) * E \\
 \Rightarrow (8 + 7 - 5) * E \\
 \Rightarrow (8 + 7 - 5) * 2
 \end{array}$$

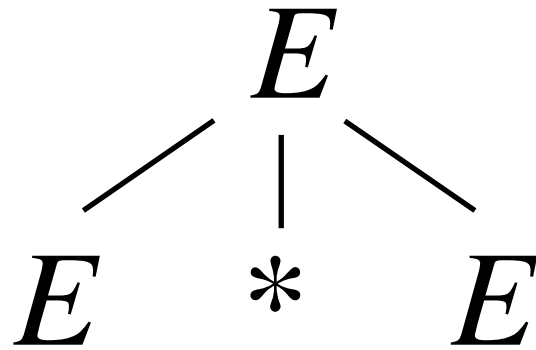
Try an example. Draw the corresponding parse tree:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E)
 \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l}
 E \Rightarrow E * E \\
 \Rightarrow (E) * E \\
 \Rightarrow (E - E) * E \\
 \Rightarrow (E + E - E) * E \\
 \Rightarrow (8 + E - E) * E \\
 \Rightarrow (8 + 7 - E) * E \\
 \Rightarrow (8 + 7 - 5) * E \\
 \Rightarrow (8 + 7 - 5) * 2
 \end{array}$$

Try an example. Draw the corresponding parse tree:



$$E \Rightarrow E * E$$

$$\Rightarrow (E) * E$$

$$\Rightarrow (E - E) * E$$

$$\Rightarrow (E + E - E) * E$$

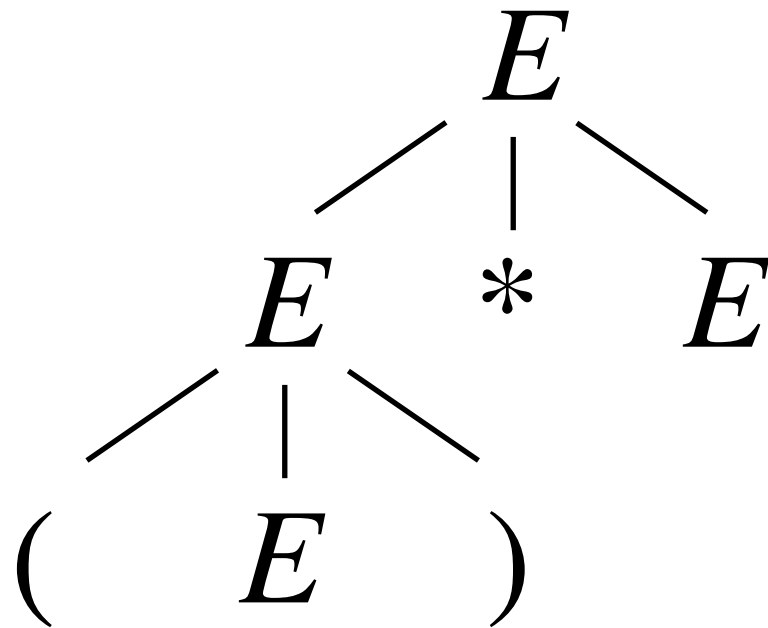
$$\Rightarrow (8 + E - E) * E$$

$$\Rightarrow (8 + 7 - E) * E$$

$$\Rightarrow (8 + 7 - 5) * E$$

$$\Rightarrow (8 + 7 - 5) * 2$$

Try an example. Draw the corresponding parse tree:



$$E \Rightarrow E * E$$

$$\Rightarrow (E) * E$$

$$\Rightarrow (E - E) * E$$

$$\Rightarrow (E + E - E) * E$$

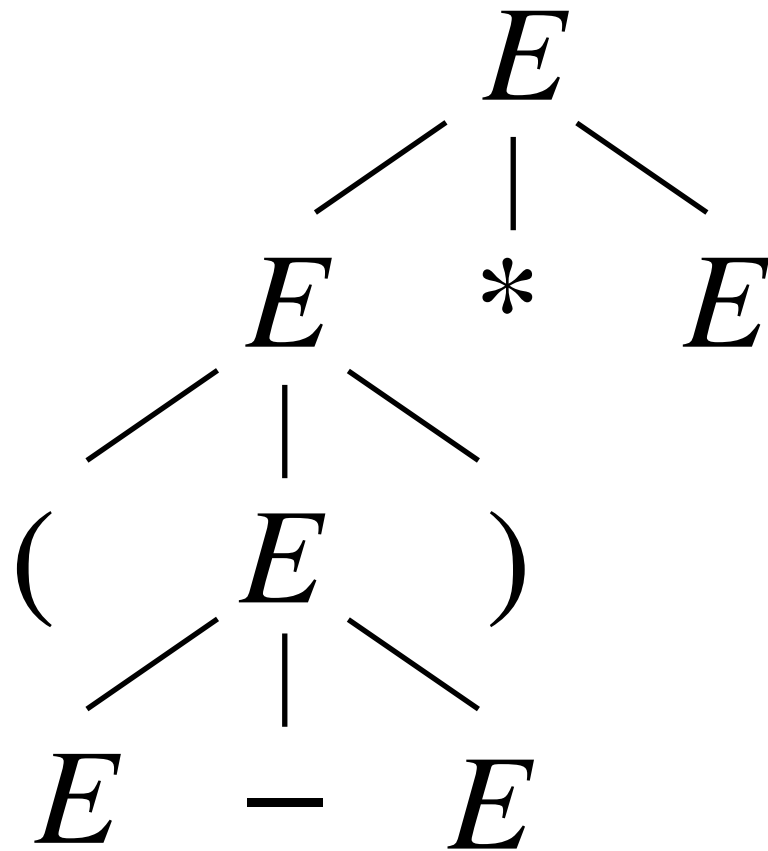
$$\Rightarrow (8 + E - E) * E$$

$$\Rightarrow (8 + 7 - E) * E$$

$$\Rightarrow (8 + 7 - 5) * E$$

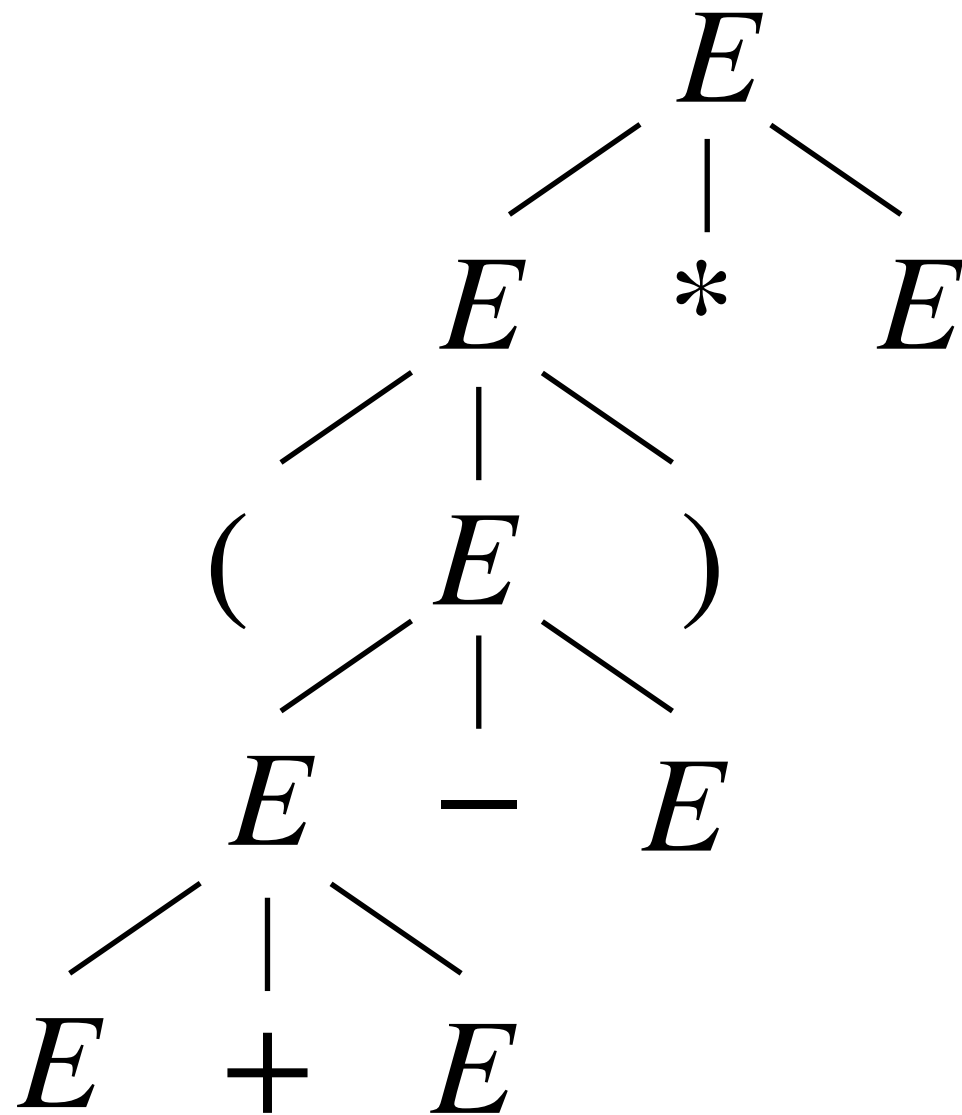
$$\Rightarrow (8 + 7 - 5) * 2$$

Try an example. Draw the corresponding parse tree:

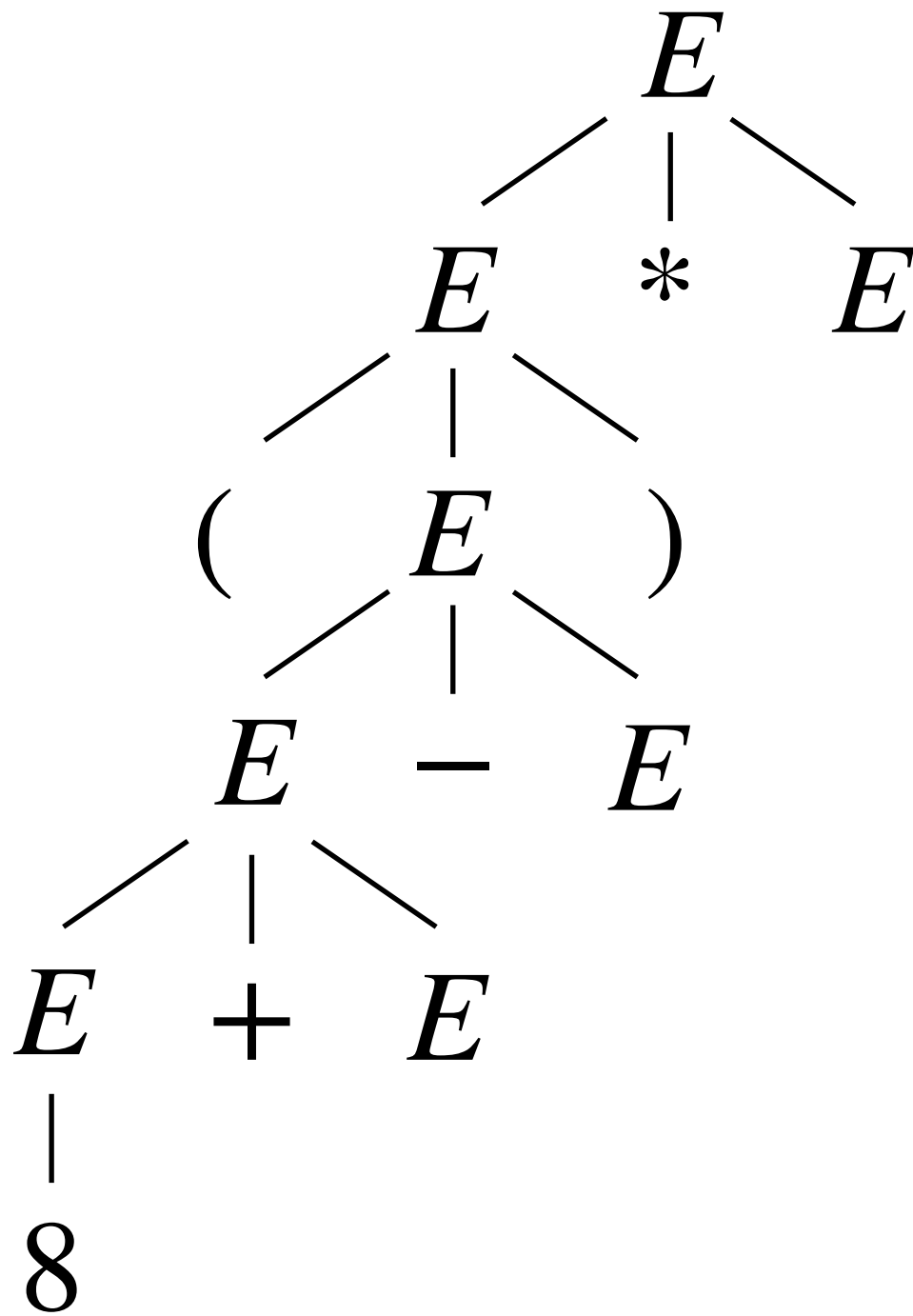


$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow (E) * E \\
 &\Rightarrow (E - E) * E \\
 &\Rightarrow (E + E - E) * E \\
 &\Rightarrow (8 + E - E) * E \\
 &\Rightarrow (8 + 7 - E) * E \\
 &\Rightarrow (8 + 7 - 5) * E \\
 &\Rightarrow (8 + 7 - 5) * 2
 \end{aligned}$$

Try an example. Draw the corresponding parse tree:

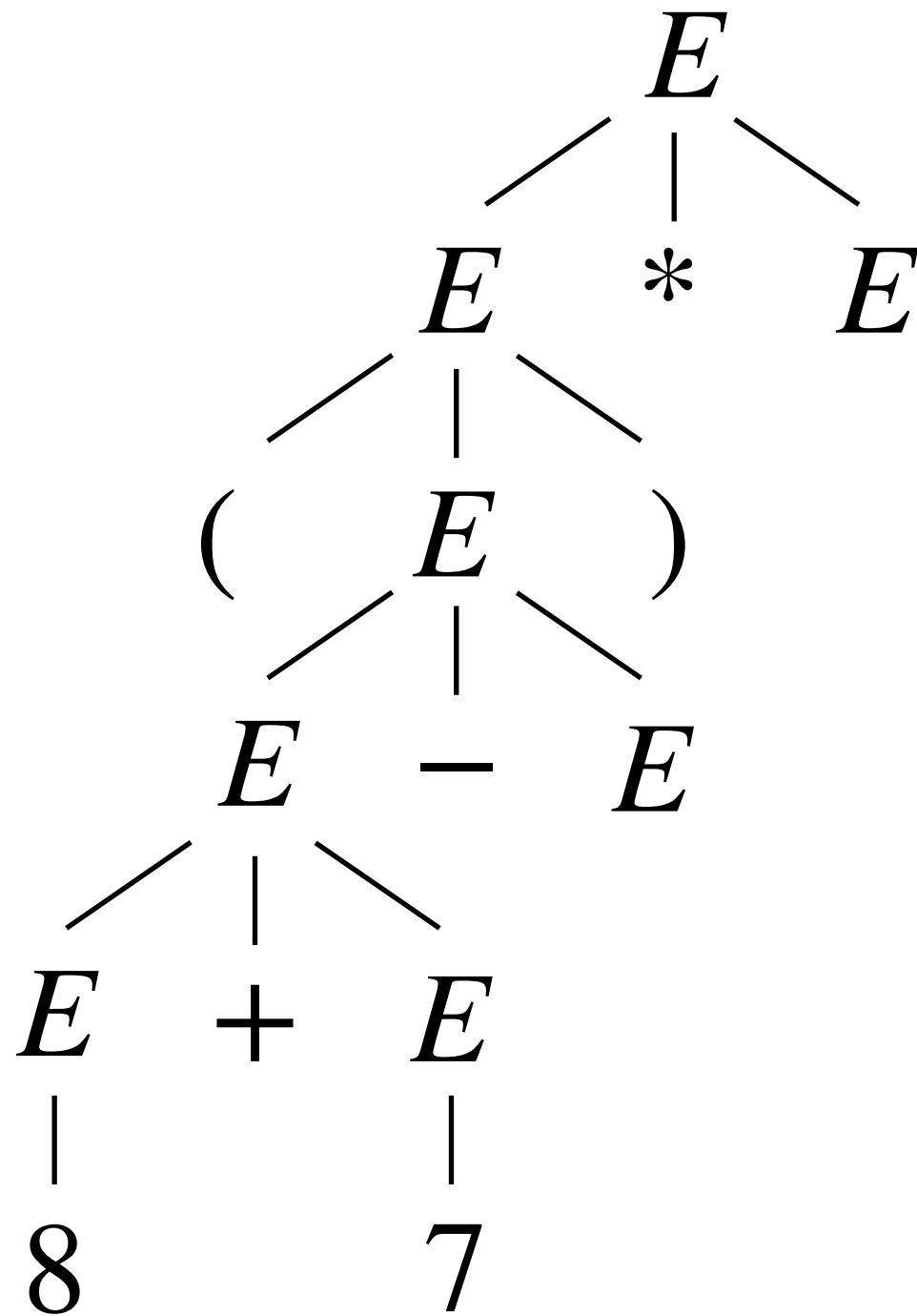


Try an example. Draw the corresponding parse tree:



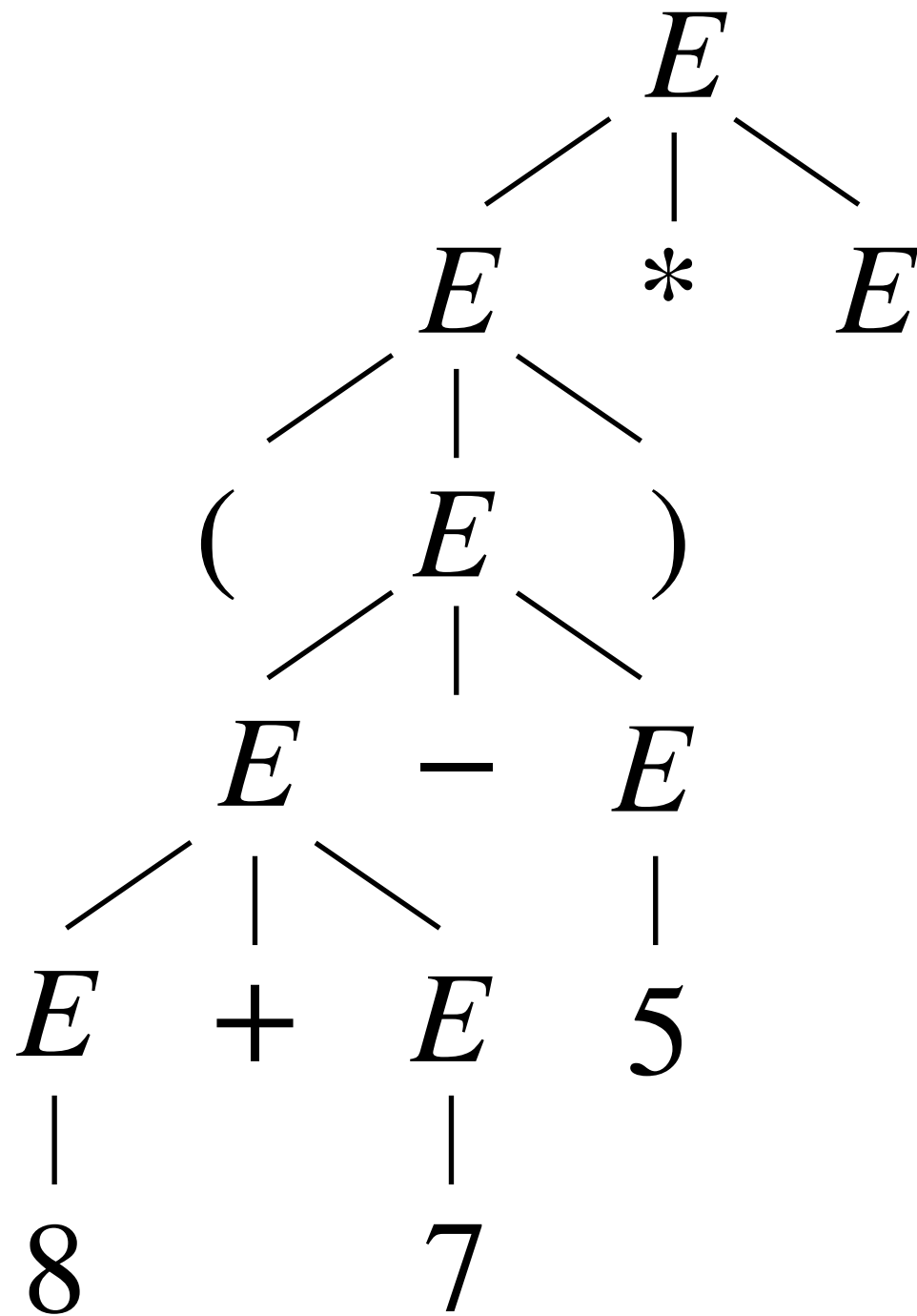
$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow (E) * E \\
 &\Rightarrow (E - E) * E \\
 &\Rightarrow (E + E - E) * E \\
 &\Rightarrow (8 + E - E) * E \\
 &\Rightarrow (8 + 7 - E) * E \\
 &\Rightarrow (8 + 7 - 5) * E \\
 &\Rightarrow (8 + 7 - 5) * 2
 \end{aligned}$$

Try an example. Draw the corresponding parse tree:



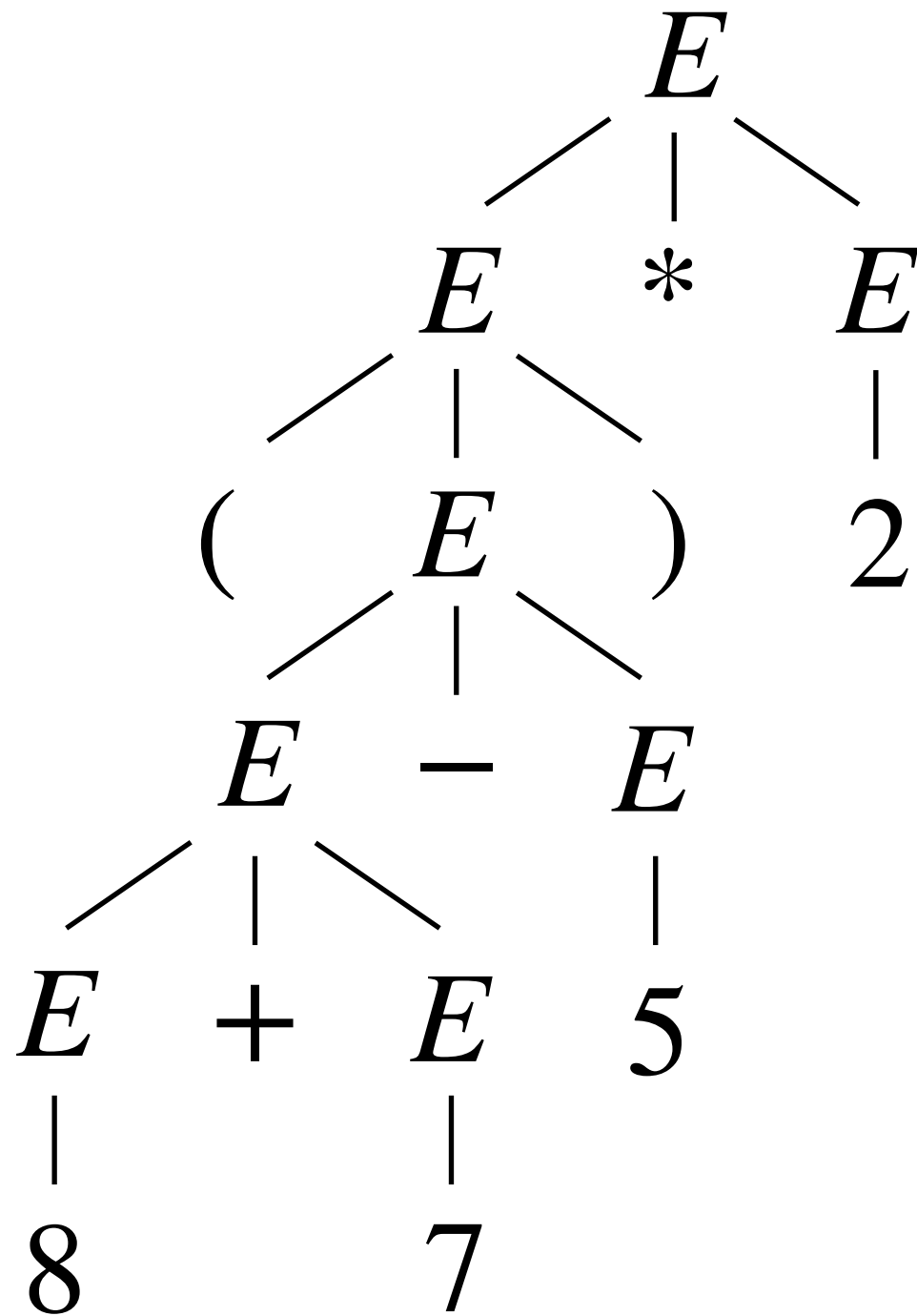
$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow (E) * E \\
 &\Rightarrow (E - E) * E \\
 &\Rightarrow (E + E - E) * E \\
 &\Rightarrow (8 + E - E) * E \\
 &\Rightarrow (8 + 7 - E) * E \\
 &\Rightarrow (8 + 7 - 5) * E \\
 &\Rightarrow (8 + 7 - 5) * 2
 \end{aligned}$$

Try an example. Draw the corresponding parse tree:



$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow (E) * E \\
 &\Rightarrow (E - E) * E \\
 &\Rightarrow (E + E - E) * E \\
 &\Rightarrow (8 + E - E) * E \\
 &\Rightarrow (8 + 7 - E) * E \\
 &\Rightarrow (8 + 7 - 5) * E \\
 &\Rightarrow (8 + 7 - 5) * 2
 \end{aligned}$$

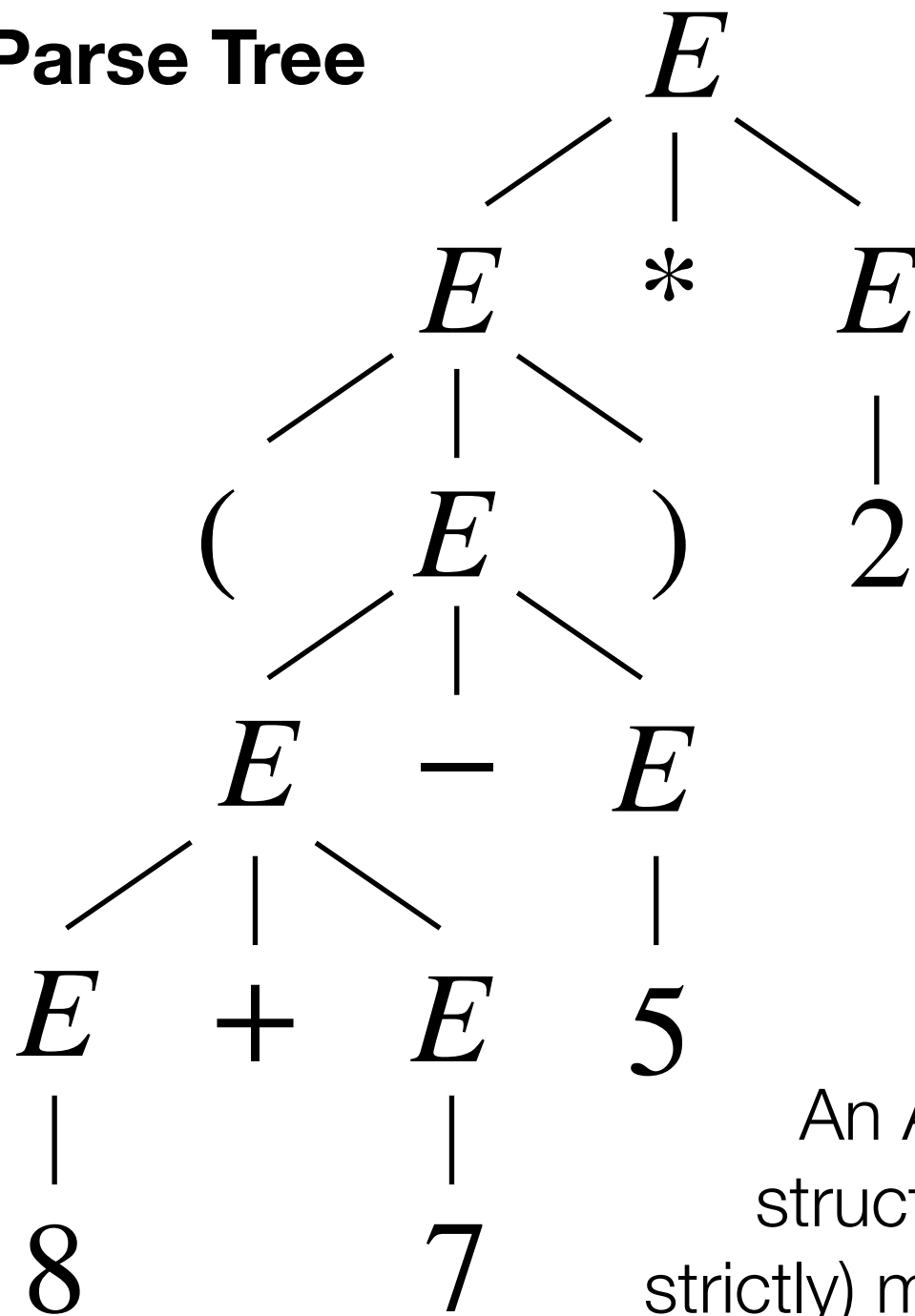
Try an example. Draw the corresponding parse tree:



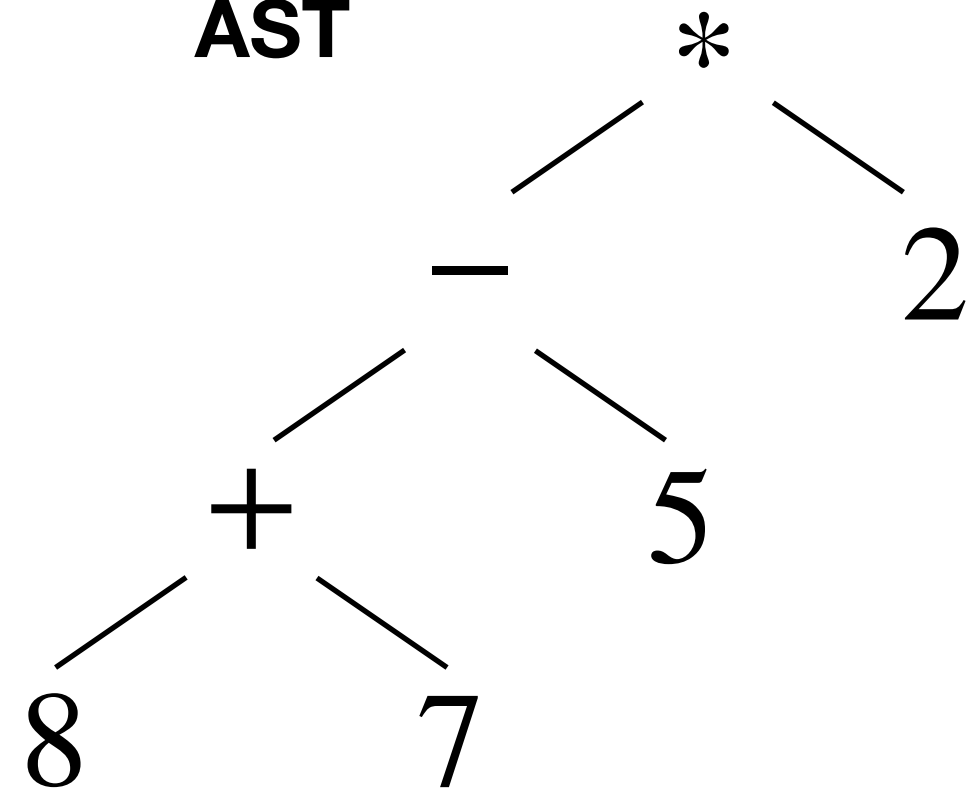
$$\begin{aligned}
 E &\Rightarrow E * E \\
 &\Rightarrow (E) * E \\
 &\Rightarrow (E - E) * E \\
 &\Rightarrow (E + E - E) * E \\
 &\Rightarrow (8 + E - E) * E \\
 &\Rightarrow (8 + 7 - E) * E \\
 &\Rightarrow (8 + 7 - 5) * E \\
 &\Rightarrow (8 + 7 - 5) * 2
 \end{aligned}$$

Abstract Syntax Trees (ASTs)

Parse Tree



AST



An AST is not just a parse tree. ASTs are data-structures, generated by parsing, that are (usually strictly) more compact than parse trees. Note how depth in the AST or parse tree defines operator precedence!

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$E \rightarrow n \quad (\text{where } n \in \mathbb{N})$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$E \Rightarrow E - E$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

$$E \Rightarrow E - E$$

$$\Rightarrow E + E - E$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l} E \rightarrow n \\ | \\ E + E \\ | \\ E - E \\ | \\ E * E \\ | \\ (E) \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l} E \Rightarrow E - E \\ \Rightarrow E + E - E \\ \Rightarrow E + E - E + E \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E)
 \end{array}$$

(where $n \in \mathbb{N}$)

$$\begin{array}{l}
 E \Rightarrow E - E \\
 \Rightarrow E + E - E \\
 \Rightarrow E + E - E + E \\
 \Rightarrow (E) + E - E + E
 \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E - E \\
 \Rightarrow E + E - E \\
 \Rightarrow E + E - E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E
 \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E - E \\
 \Rightarrow E + E - E \\
 \Rightarrow E + E - E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E
 \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

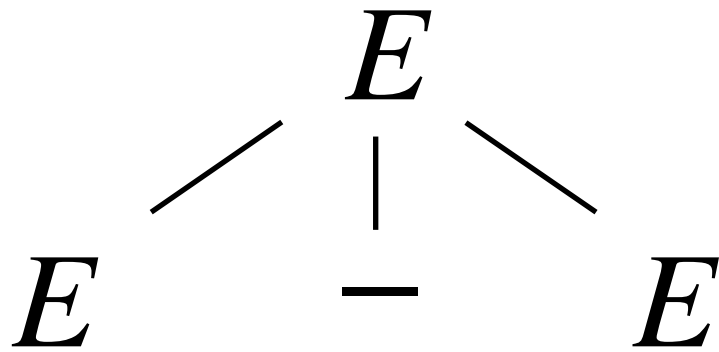
$$\begin{array}{l}
 E \Rightarrow E - E \\
 \Rightarrow E + E - E \\
 \Rightarrow E + E - E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E \\
 \Rightarrow (2) + 5 - 3 + E
 \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

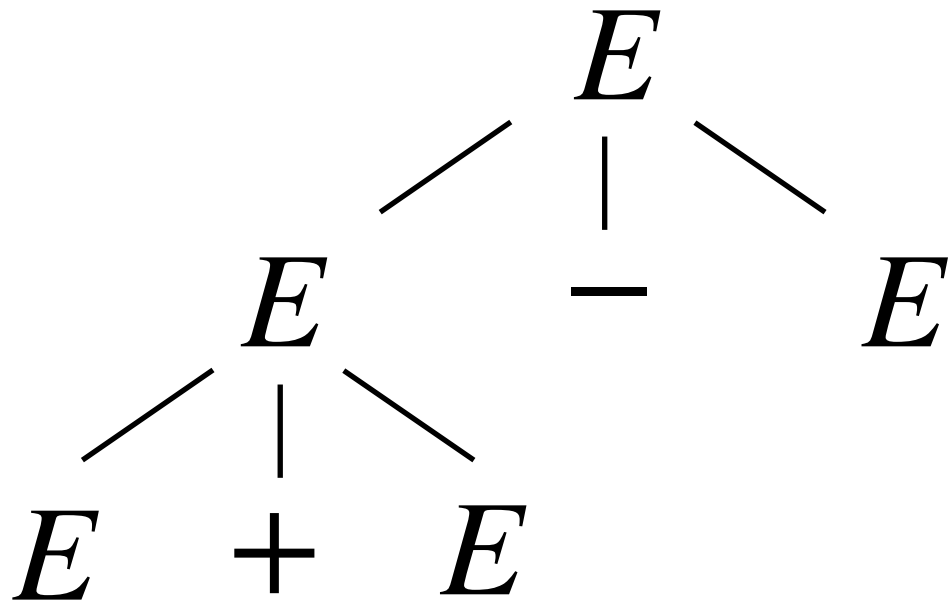
$$\begin{array}{l}
 E \Rightarrow E - E \\
 \Rightarrow E + E - E \\
 \Rightarrow E + E - E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E \\
 \Rightarrow (2) + 5 - 3 + E \\
 \Rightarrow (2) + 5 - 3 + 1
 \end{array}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



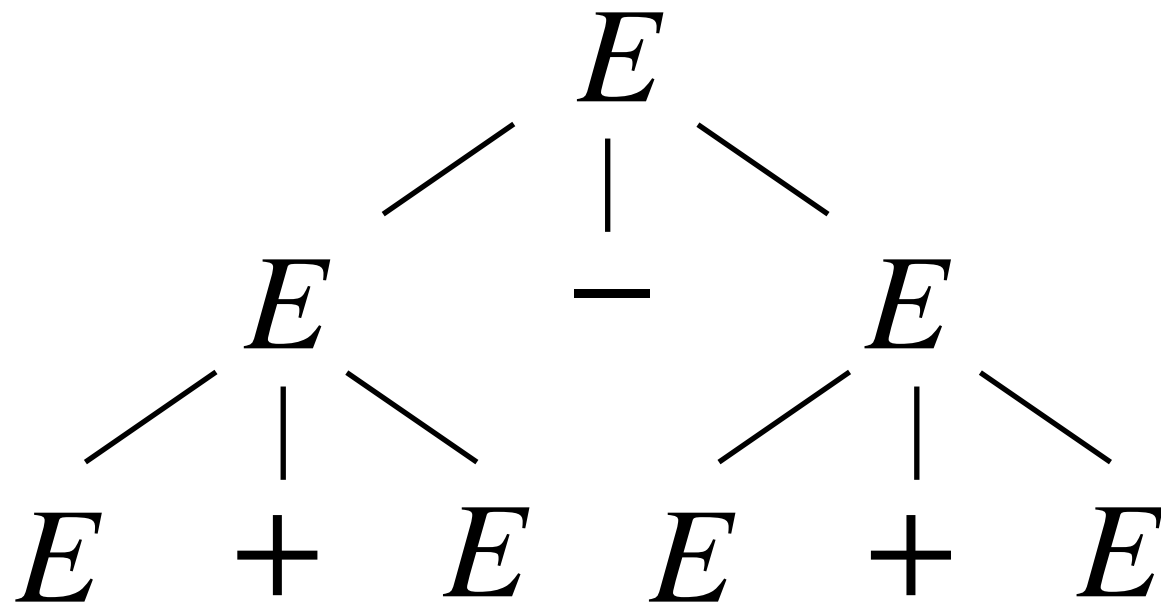
$$\begin{aligned}
 E &\Rightarrow E - E \\
 &\Rightarrow E + E - E \\
 &\Rightarrow E + E - E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E - E \\
 &\Rightarrow E + E - E \\
 &\Rightarrow E + E - E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



$$E \Rightarrow E - E$$

$$\Rightarrow E + E - E$$

$$\Rightarrow E + E - E + E$$

$$\Rightarrow (E) + E - E + E$$

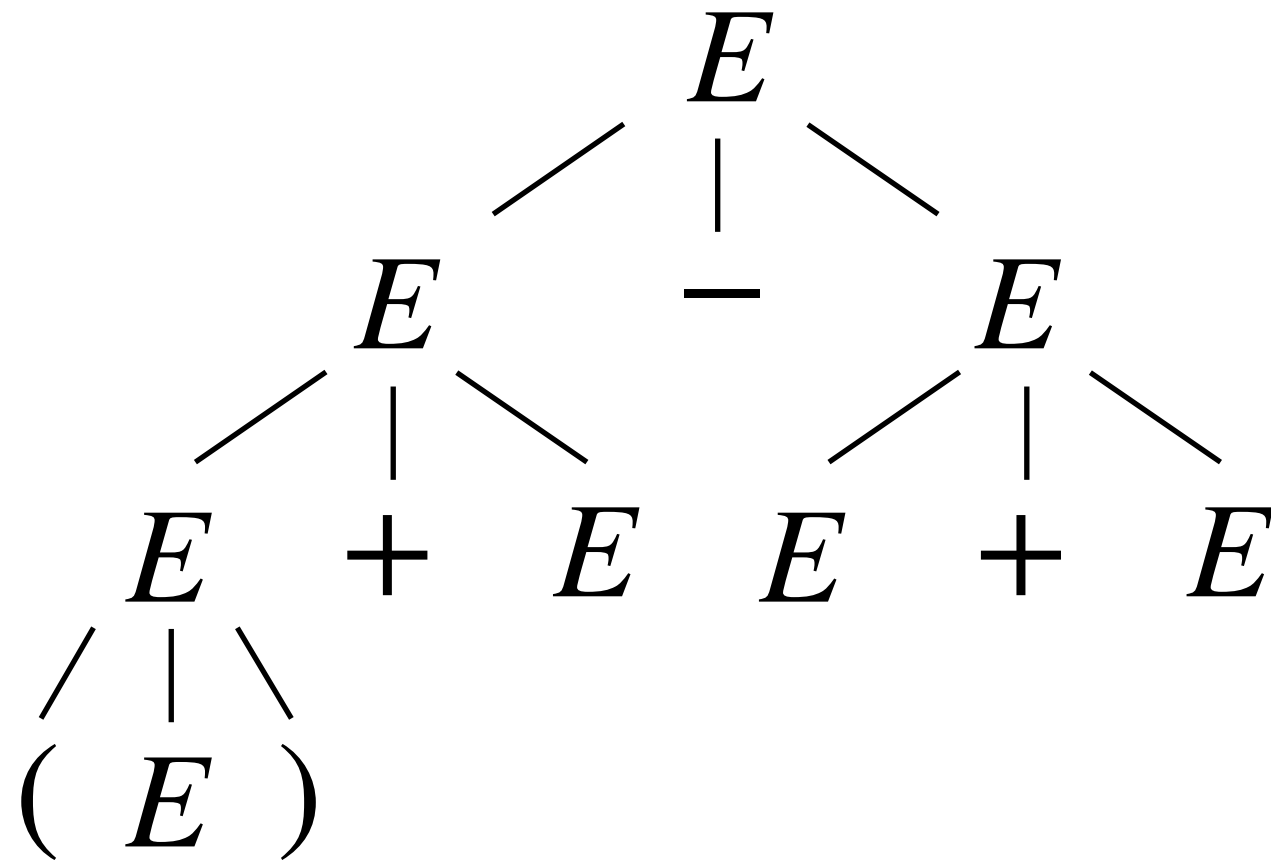
$$\Rightarrow (2) + E - E + E$$

$$\Rightarrow (2) + 5 - E + E$$

$$\Rightarrow (2) + 5 - 3 + E$$

$$\Rightarrow (2) + 5 - 3 + 1$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



$$E \Rightarrow E - E$$

$$\Rightarrow E + E - E$$

$$\Rightarrow E + E - E + E$$

$$\Rightarrow (E) + E - E + E$$

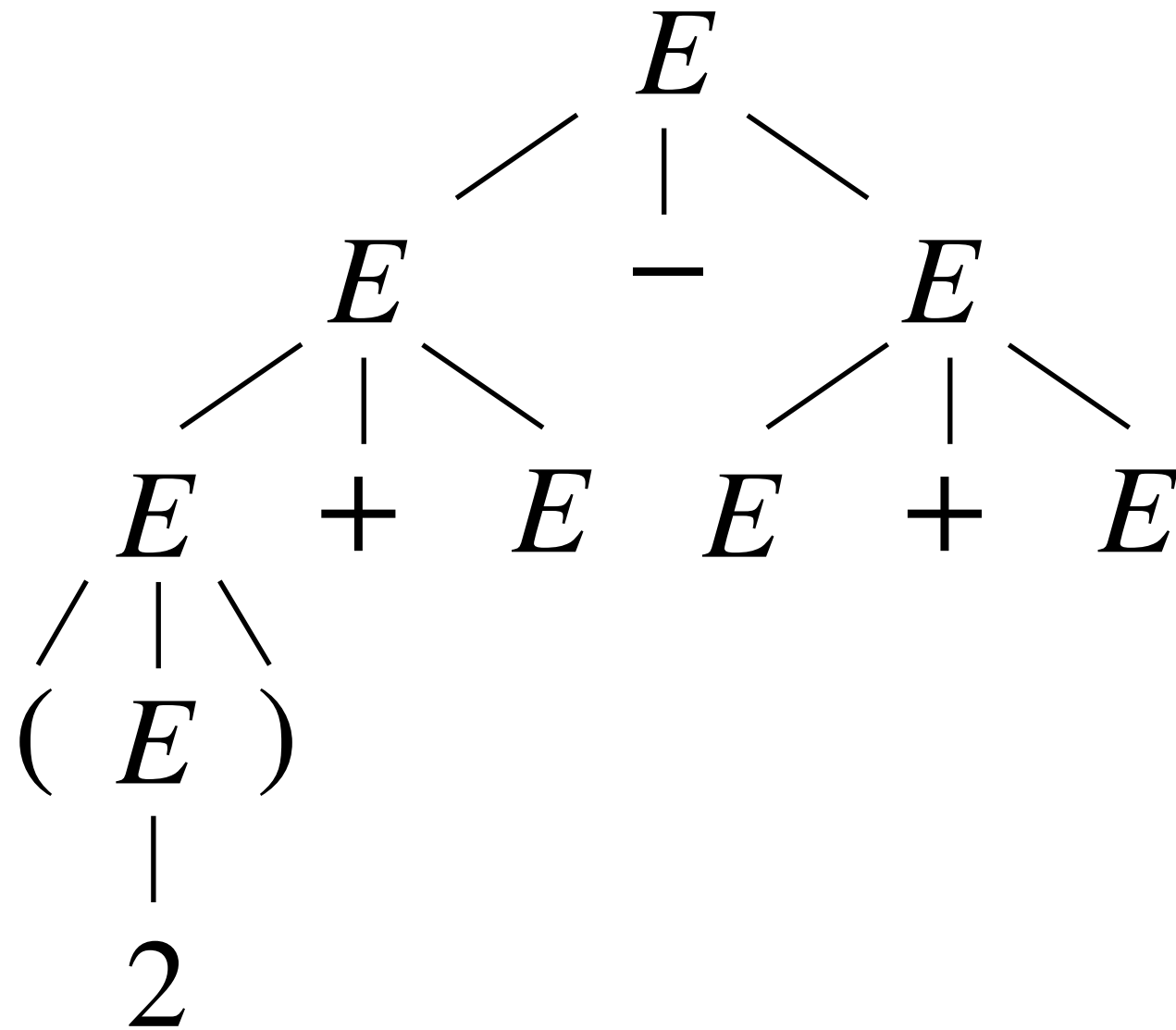
$$\Rightarrow (2) + E - E + E$$

$$\Rightarrow (2) + 5 - E + E$$

$$\Rightarrow (2) + 5 - 3 + E$$

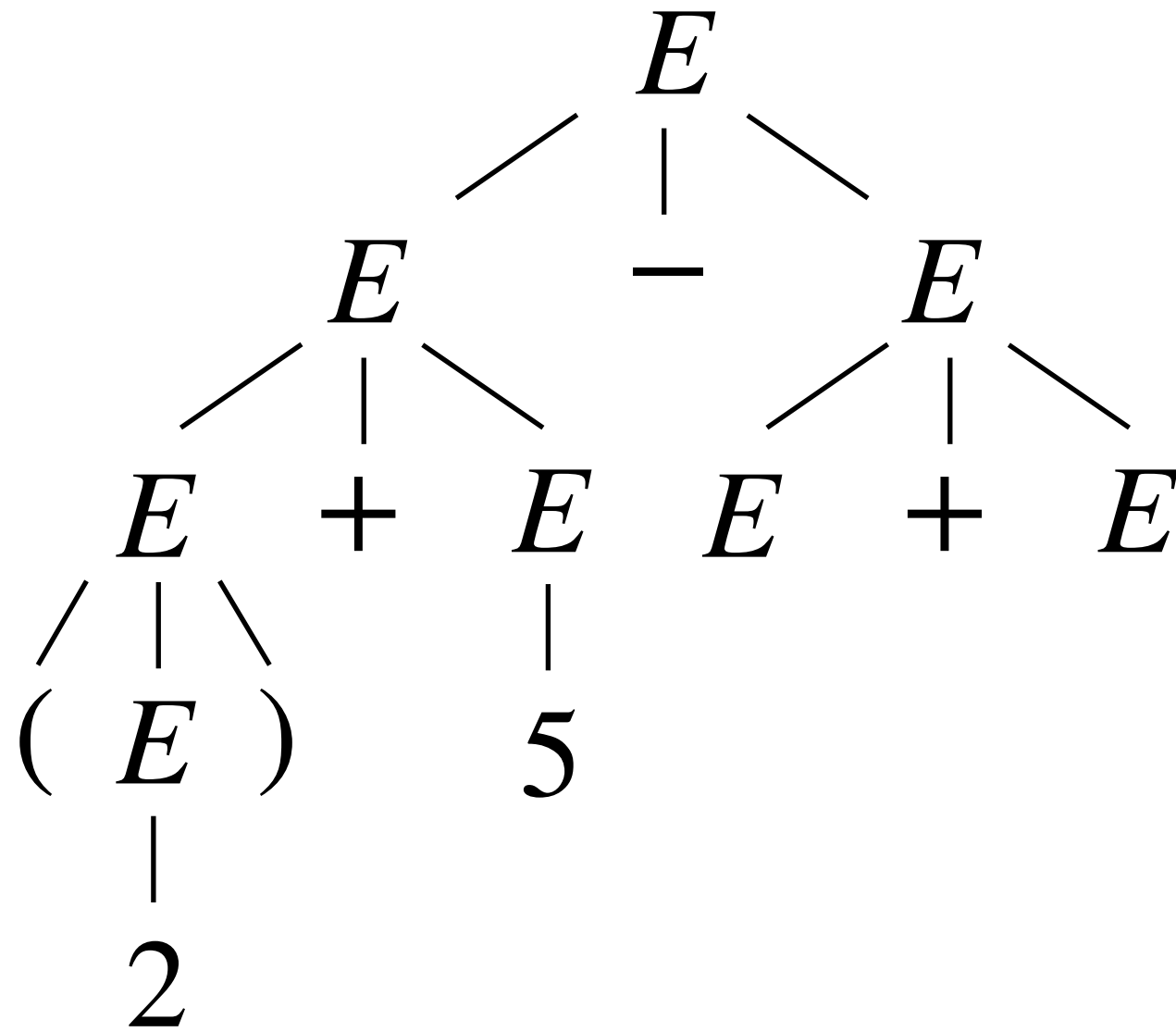
$$\Rightarrow (2) + 5 - 3 + 1$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



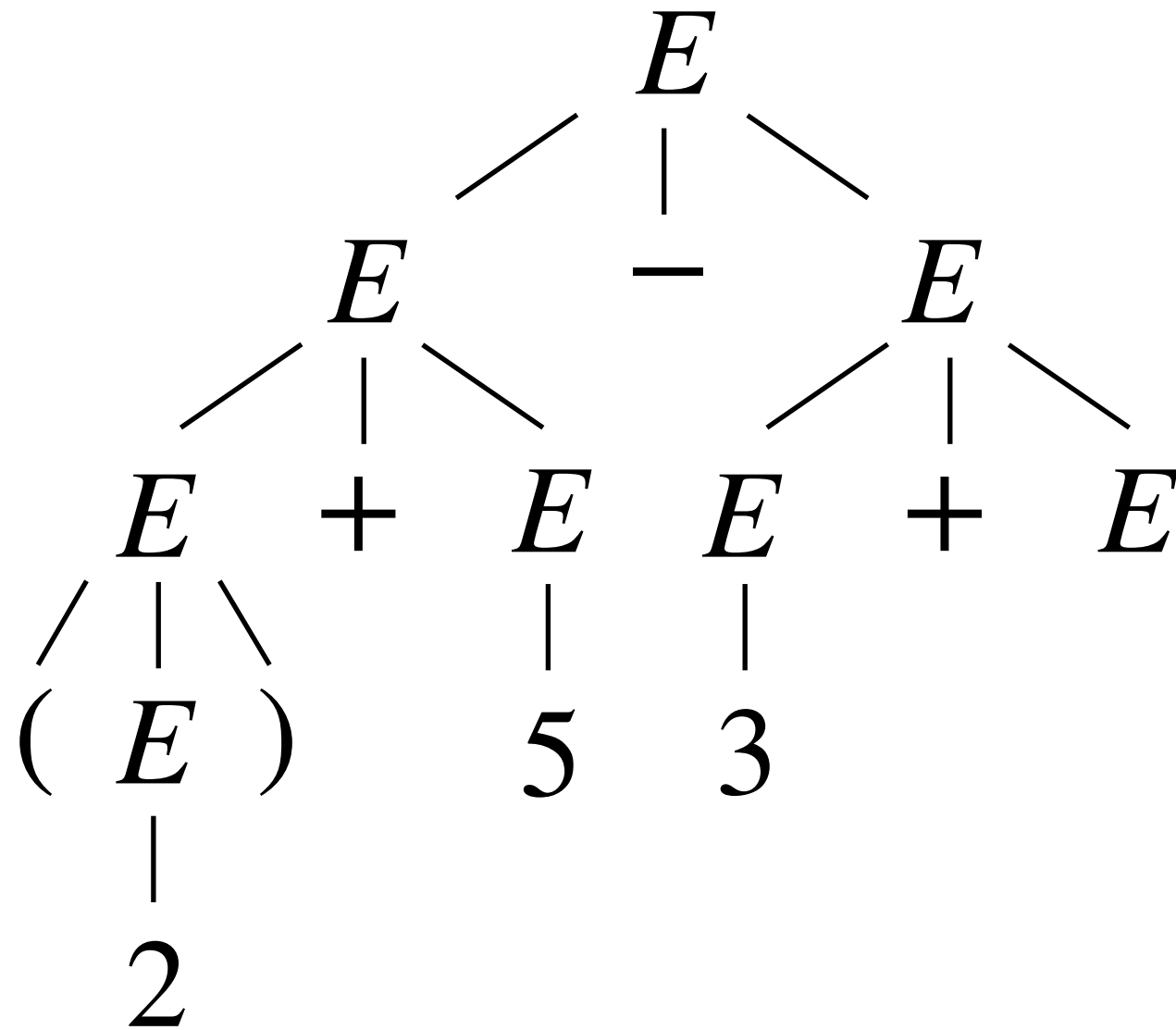
$$\begin{aligned}
 E &\Rightarrow E - E \\
 &\Rightarrow E + E - E \\
 &\Rightarrow E + E - E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



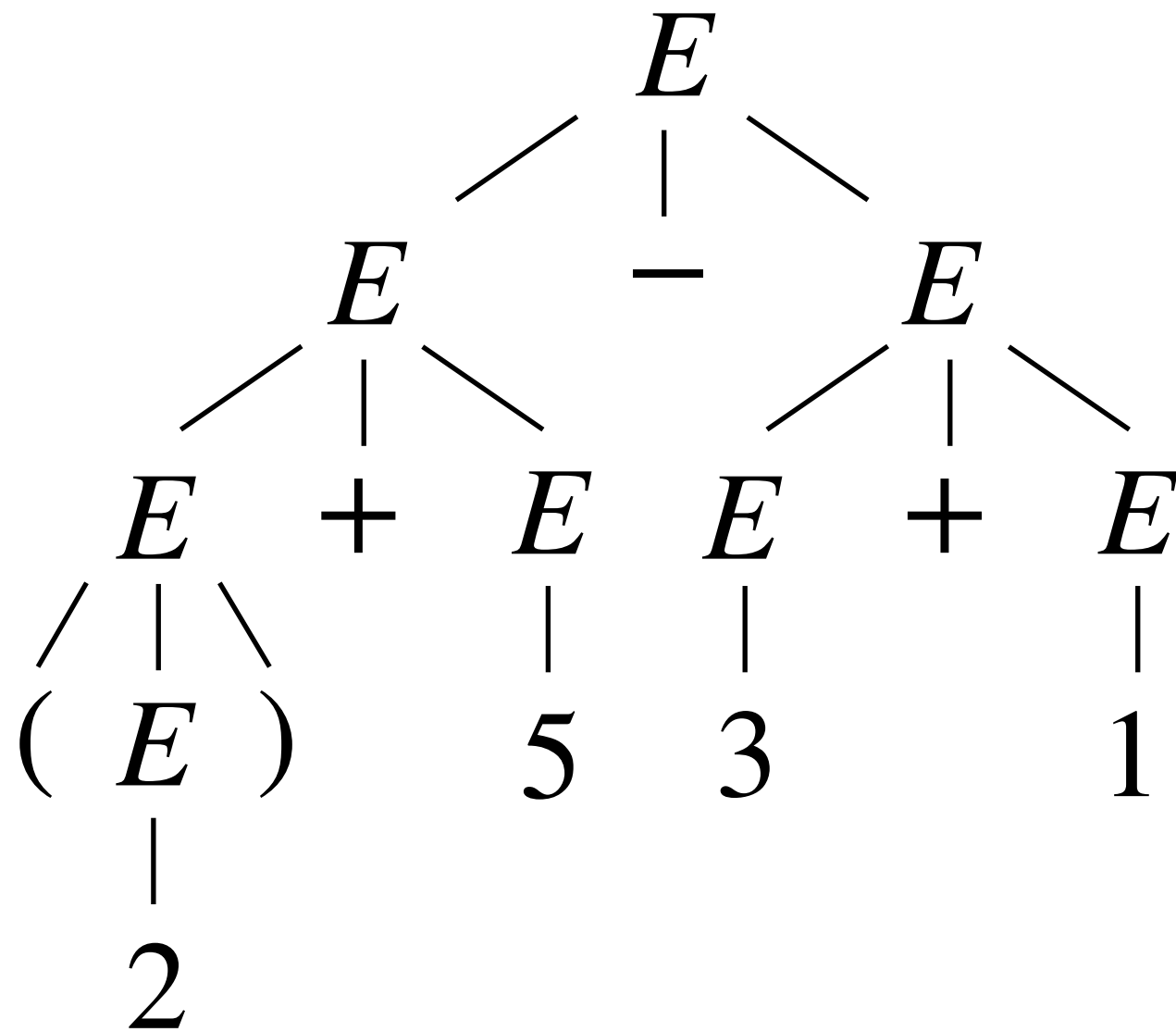
$$\begin{aligned}
 E &\Rightarrow E - E \\
 &\Rightarrow E + E - E \\
 &\Rightarrow E + E - E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E - E \\
 &\Rightarrow E + E - E \\
 &\Rightarrow E + E - E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a parse tree for $(2)+5-3+1$:



$$\begin{aligned} E &\Rightarrow E - E \\ &\Rightarrow E + E - E \\ &\Rightarrow E + E - E + E \\ &\Rightarrow (E) + E - E + E \\ &\Rightarrow (2) + E - E + E \\ &\Rightarrow (2) + 5 - E + E \\ &\Rightarrow (2) + 5 - 3 + E \\ &\Rightarrow (2) + 5 - 3 + 1 \end{aligned}$$

Leftmost and rightmost derivations

Leftmost and rightmost derivations

- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.

Leftmost and rightmost derivations

- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.
- A ***rightmost derivation*** is one where the rightmost nonterminal is substituted at each step.

Leftmost and rightmost derivations

- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.
- A ***rightmost derivation*** is one where the rightmost nonterminal is substituted at each step.
- Example: $S \rightarrow XY$ $X \rightarrow \mathbf{xx}$ $Y \rightarrow \mathbf{y}$

Leftmost and rightmost derivations

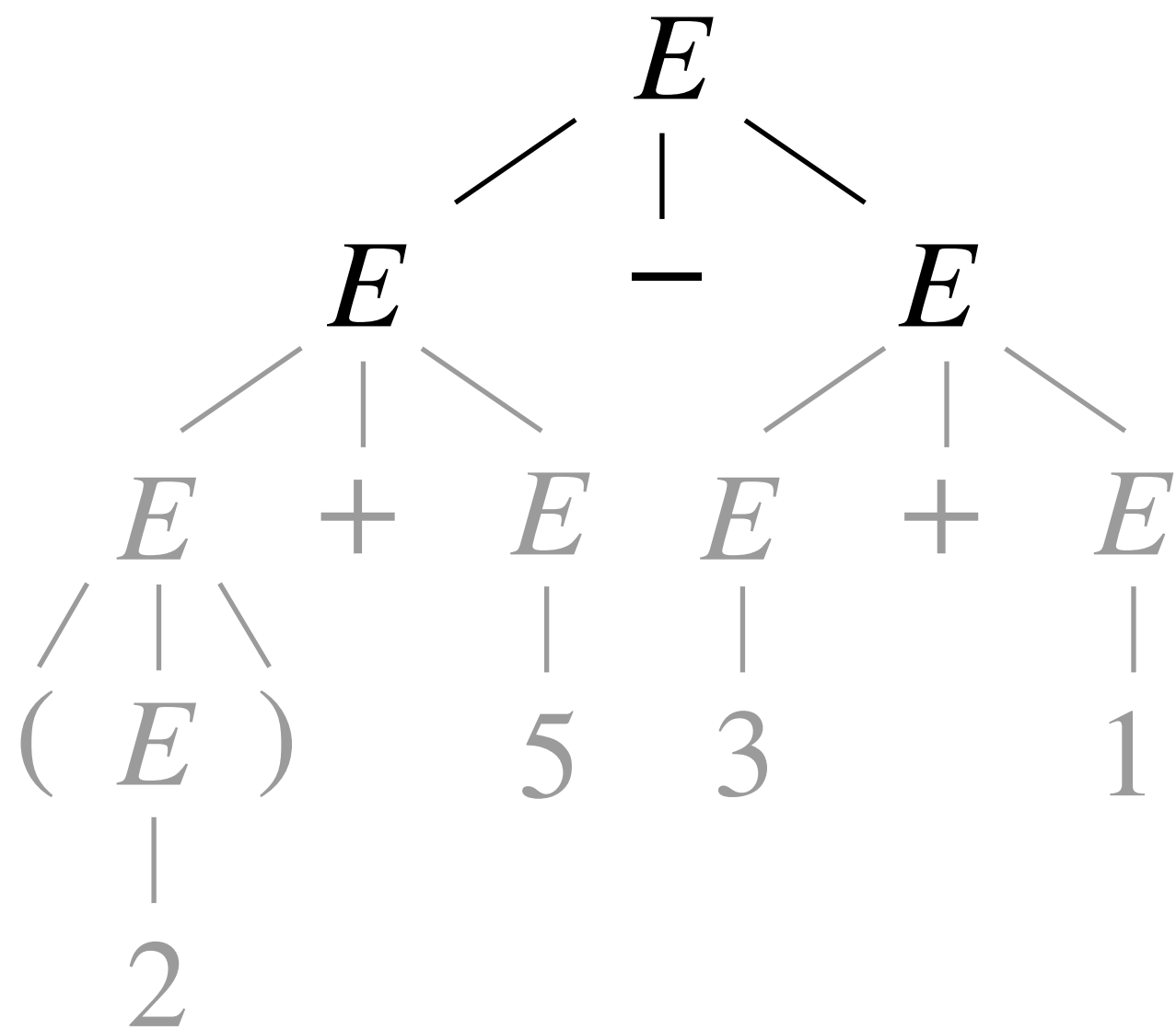
- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.
- A ***rightmost derivation*** is one where the rightmost nonterminal is substituted at each step.
- Example: $S \rightarrow XY \quad X \rightarrow \mathbf{xx} \quad Y \rightarrow \mathbf{y}$
 - The leftmost derivation of ***xxxy*** is: $S \Rightarrow XY \Rightarrow \mathbf{xx}Y \Rightarrow \mathbf{xxxy}$

Leftmost and rightmost derivations

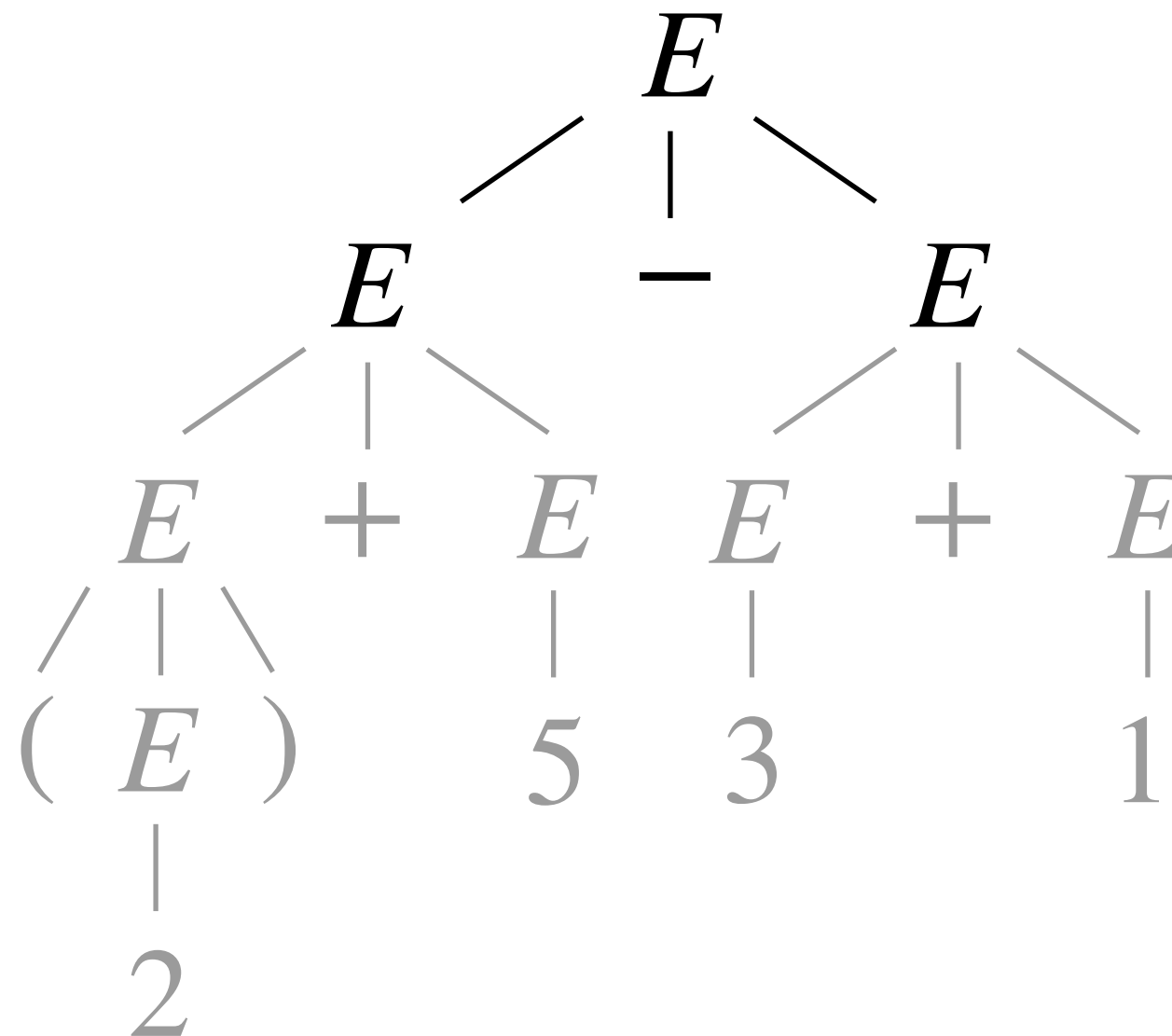
- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.
- A ***rightmost derivation*** is one where the rightmost nonterminal is substituted at each step.
- Example: $S \rightarrow XY \quad X \rightarrow \mathbf{xx} \quad Y \rightarrow \mathbf{y}$
 - The leftmost derivation of ***xy*** is: $S \Rightarrow XY \Rightarrow \mathbf{xx}Y \Rightarrow \mathbf{xy}$
 - The rightmost derivation of ***xy*** is: $S \Rightarrow XY \Rightarrow X\mathbf{y} \Rightarrow \mathbf{xy}$

Leftmost and rightmost derivations

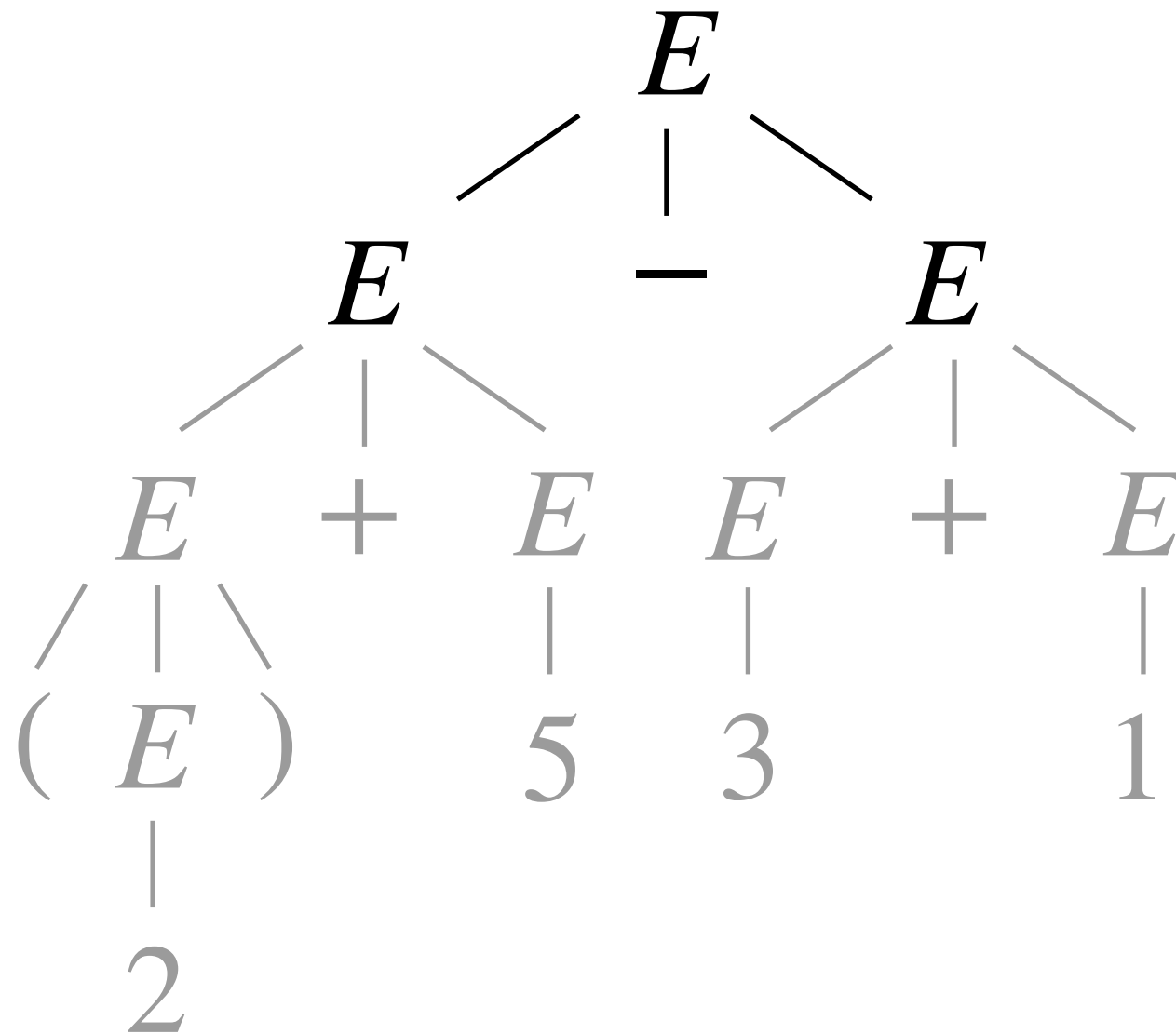
- A ***leftmost derivation*** is one where the leftmost nonterminal is substituted at each step.
- A ***rightmost derivation*** is one where the rightmost nonterminal is substituted at each step.
- Example: $S \rightarrow XY \quad X \rightarrow \mathbf{xx} \quad Y \rightarrow \mathbf{y}$
 - The leftmost derivation of ***xxxy*** is: $S \Rightarrow XY \Rightarrow \mathbf{xx}Y \Rightarrow \mathbf{xxxy}$
 - The rightmost derivation of ***xxxy*** is: $S \Rightarrow XY \Rightarrow X\mathbf{y} \Rightarrow \mathbf{xxxy}$
- Parse trees show which nonterminal is expanded using which production rule, but do not show the order of expansion!



Each non-terminal must expand
using the same production rule
(e.g., $E ::= E - E$) to obtain this parse tree.



Each non-terminal must expand
using the same production rule
(e.g., $E ::= E - E$) to obtain this parse tree.



However, whether the right subtree is expanded before the left, or the left is fully expanded before the right, does not change the parse tree (and thus the AST) we obtain.

Parse trees and ambiguity

Parse trees and ambiguity

- A parse tree may be the same for both leftmost and rightmost derivations.

Parse trees and ambiguity

- A parse tree may be the same for both leftmost and rightmost derivations.
- Each parse tree has a unique leftmost derivation and a unique rightmost derivation.

Parse trees and ambiguity

- A parse tree may be the same for both leftmost and rightmost derivations.
- Each parse tree has a unique leftmost derivation and a unique rightmost derivation.
- Not every string has a unique parse tree! Example: **xyxyx**

$$S \rightarrow \mathbf{x} \mid S\mathbf{y}S$$

Parse trees and ambiguity

- A parse tree may be the same for both leftmost and rightmost derivations.
- Each parse tree has a unique leftmost derivation and a unique rightmost derivation.
- Not every string has a unique parse tree! Example: **xyxyx**

$$S \rightarrow \mathbf{x} \mid S\mathbf{y}S$$

- One leftmost derivation is:

$$S \Rightarrow S\mathbf{y}S \Rightarrow S\mathbf{y}S\mathbf{y}S \Rightarrow^+ \mathbf{xyxyx}$$

Parse trees and ambiguity

- A parse tree may be the same for both leftmost and rightmost derivations.
- Each parse tree has a unique leftmost derivation and a unique rightmost derivation.
- Not every string has a unique parse tree! Example: **xyxyx**

$$S \rightarrow \mathbf{x} \mid S\mathbf{y}S$$

- One leftmost derivation is:

$$S \Rightarrow S\mathbf{y}S \Rightarrow S\mathbf{y}S\mathbf{y}S \Rightarrow^+ \mathbf{xyxyx}$$

- A different leftmost derivation is:

$$S \Rightarrow S\mathbf{y}S \Rightarrow \mathbf{xy}S \Rightarrow \mathbf{xy}S\mathbf{y}S \Rightarrow^+ \mathbf{xyxyx}$$

Parse trees and ambiguity

Parse trees and ambiguity

- There may be many ways to parse a particular string using a particular grammar. This is the case, even if we specify left-to-right parsing (i.e., leftmost derivation) or right-to-left parsing.

Parse trees and ambiguity

- There may be many ways to parse a particular string using a particular grammar. This is the case, even if we specify left-to-right parsing (i.e., leftmost derivation) or right-to-left parsing.
- A CFG **G** is ***ambiguous*** if there exists any string for which parsing is ambiguous: for which two distinct parse trees exist.

Parse trees and ambiguity

- There may be many ways to parse a particular string using a particular grammar. This is the case, even if we specify left-to-right parsing (i.e., leftmost derivation) or right-to-left parsing.
- A CFG **G** is ***ambiguous*** if there exists any string for which parsing is ambiguous: for which two distinct parse trees exist.
 - For example: $S \rightarrow (S) \mid () \mid \epsilon$

Parse trees and ambiguity

- There may be many ways to parse a particular string using a particular grammar. This is the case, even if we specify left-to-right parsing (i.e., leftmost derivation) or right-to-left parsing.
- A CFG **G** is ***ambiguous*** if there exists any string for which parsing is ambiguous: for which two distinct parse trees exist.
 - For example: $S \rightarrow (S) \mid () \mid \epsilon$
 - Or the grammar of expressions, E , we just saw!

Parse trees and ambiguity

- There may be many ways to parse a particular string using a particular grammar. This is the case, even if we specify left-to-right parsing (i.e., leftmost derivation) or right-to-left parsing.
- A CFG **G** is ***ambiguous*** if there exists any string for which parsing is ambiguous: for which two distinct parse trees exist.
 - For example: $S \rightarrow (S) \mid () \mid \epsilon$
 - Or the grammar of expressions, E , we just saw!
- A CFL **L** is called ***inherently ambiguous*** if there does not exist an unambiguous grammar for generating the language.

Try an example. Which grammar(s) are ambiguous?

$$S \rightarrow A\mathbf{0}S\mathbf{1}A \mid \epsilon \quad A \rightarrow \epsilon \mid \mathbf{2}A$$

$$S \rightarrow \mathbf{0}SS\mathbf{1} \mid \mathbf{0}S\mathbf{1} \mid \epsilon$$

$$S \rightarrow (S, S) \mid \mathbf{0} \mid \mathbf{1}$$

$$S \rightarrow S + S \mid n \in \mathbb{N}$$

Try an example. Which grammar(s) are ambiguous?

$$S \rightarrow A\mathbf{0}S\mathbf{1}A \mid \epsilon \quad A \rightarrow \epsilon \mid \mathbf{2}A$$

$$S \rightarrow \mathbf{0}SS\mathbf{1} \mid \mathbf{0}S\mathbf{1} \mid \epsilon$$

$$S \rightarrow (S, S) \mid \mathbf{0} \mid \mathbf{1}$$

$$S \rightarrow S + S \mid n \in \mathbb{N}$$

Try an example. Which grammar(s) are ambiguous?

$$S \rightarrow A\mathbf{0}S\mathbf{1}A \mid \epsilon \quad A \rightarrow \epsilon \mid \mathbf{2}A$$

$$S \rightarrow \mathbf{0}SS\mathbf{1} \mid \mathbf{0}S\mathbf{1} \mid \epsilon$$

$$S \rightarrow (S, S) \mid \mathbf{0} \mid \mathbf{1}$$

$$S \rightarrow S + S \mid n \in \mathbb{N}$$

Try an example. Write an ambiguous grammar for ***aa****.

Try an example. Write an ambiguous grammar for ***aa****.

Try an example. Write an ambiguous grammar for ***aa****.

These grammars
are all **ambiguous**.



$$S \rightarrow \mathbf{aS} \mid \mathbf{Sa} \mid \mathbf{a}$$

$$S \rightarrow \mathbf{aS} \mid \mathbf{aa} \mid \mathbf{a}$$

$$S \rightarrow \mathbf{SS} \mid \mathbf{a}$$

$$S \rightarrow \mathbf{S} \mid \mathbf{aS} \mid \mathbf{a}$$

Try an example. Write an ambiguous grammar for ***aa****.

These grammars
are all **ambiguous**.



$$S \rightarrow aS \mid Sa \mid a$$

$$S \rightarrow aS \mid aa \mid a$$

$$S \rightarrow SS \mid a$$

$$S \rightarrow S \mid aS \mid a$$

These grammars are
all **unambiguous**.



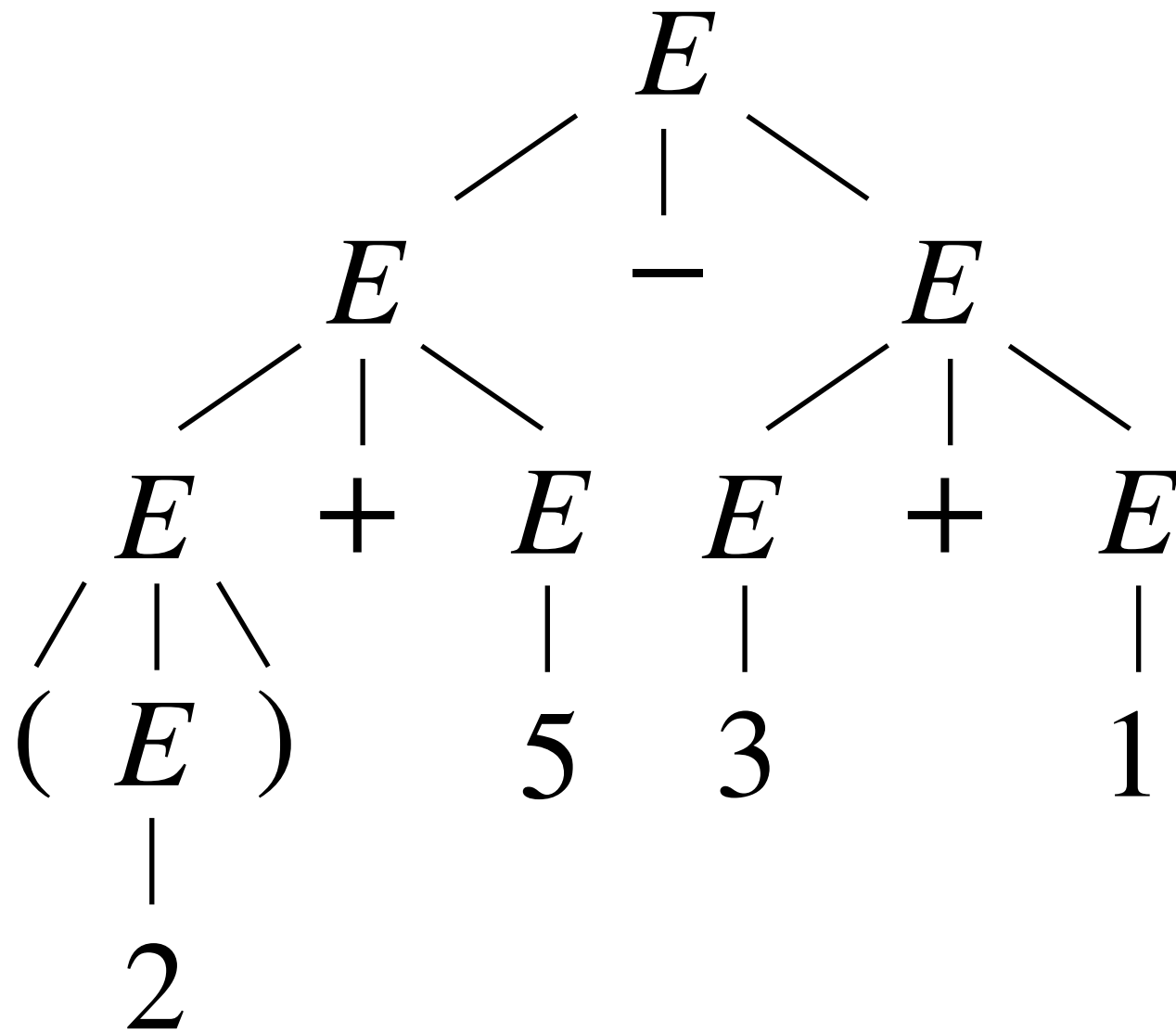
$$S \rightarrow aS \mid a$$

$$S \rightarrow Sa \mid a$$

$$S \rightarrow Saa \mid aa \mid a$$

$$S \rightarrow aaS \mid aa \mid a$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$E \Rightarrow E + E$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

$$E \Rightarrow E + E$$

$$\Rightarrow E + E + E$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$E \rightarrow n$$

$$| E + E$$

$$| E - E$$

$$| E * E$$

$$| (E)$$

(where $n \in \mathbb{N}$)

$$E \Rightarrow E + E$$

$$\Rightarrow E + E + E$$

$$\Rightarrow (E) + E + E$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E + E \\
 \Rightarrow E + E + E \\
 \Rightarrow (E) + E + E \\
 \Rightarrow (E) + E - E + E
 \end{array}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E + E \\
 \Rightarrow E + E + E \\
 \Rightarrow (E) + E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E
 \end{array}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E + E \\
 \Rightarrow E + E + E \\
 \Rightarrow (E) + E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E
 \end{array}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

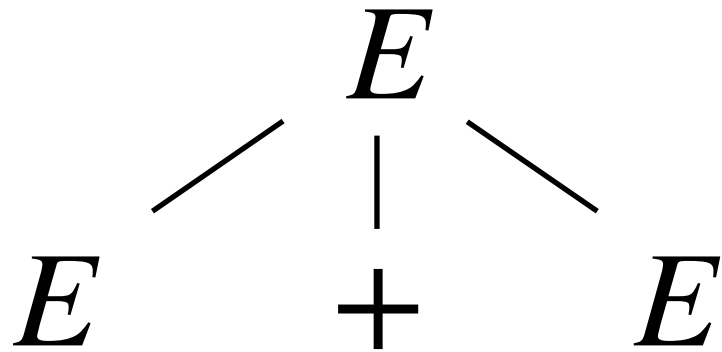
$$\begin{array}{l}
 E \Rightarrow E + E \\
 \Rightarrow E + E + E \\
 \Rightarrow (E) + E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E \\
 \Rightarrow (2) + 5 - 3 + E
 \end{array}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow n \\
 | \\
 E + E \\
 | \\
 E - E \\
 | \\
 E * E \\
 | \\
 (E) \\
 \text{(where } n \in \mathbb{N})
 \end{array}$$

$$\begin{array}{l}
 E \Rightarrow E + E \\
 \Rightarrow E + E + E \\
 \Rightarrow (E) + E + E \\
 \Rightarrow (E) + E - E + E \\
 \Rightarrow (2) + E - E + E \\
 \Rightarrow (2) + 5 - E + E \\
 \Rightarrow (2) + 5 - 3 + E \\
 \Rightarrow (2) + 5 - 3 + 1
 \end{array}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$E \Rightarrow E + E$$

$$\Rightarrow E + E + E$$

$$\Rightarrow (E) + E + E$$

$$\Rightarrow (E) + E - E + E$$

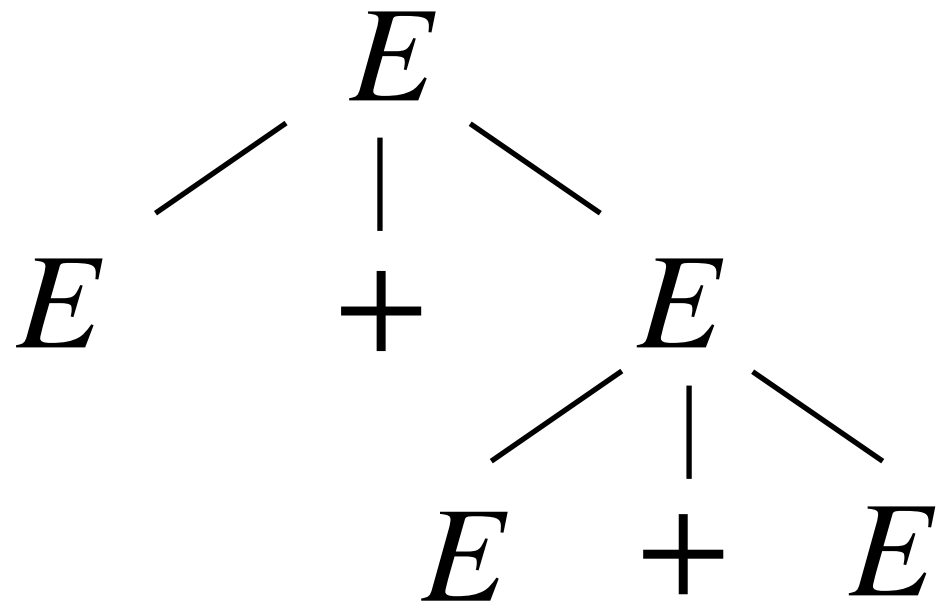
$$\Rightarrow (2) + E - E + E$$

$$\Rightarrow (2) + 5 - E + E$$

$$\Rightarrow (2) + 5 - 3 + E$$

$$\Rightarrow (2) + 5 - 3 + 1$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$E \Rightarrow E + E$$

$$\Rightarrow E + E + E$$

$$\Rightarrow (E) + E + E$$

$$\Rightarrow (E) + E - E + E$$

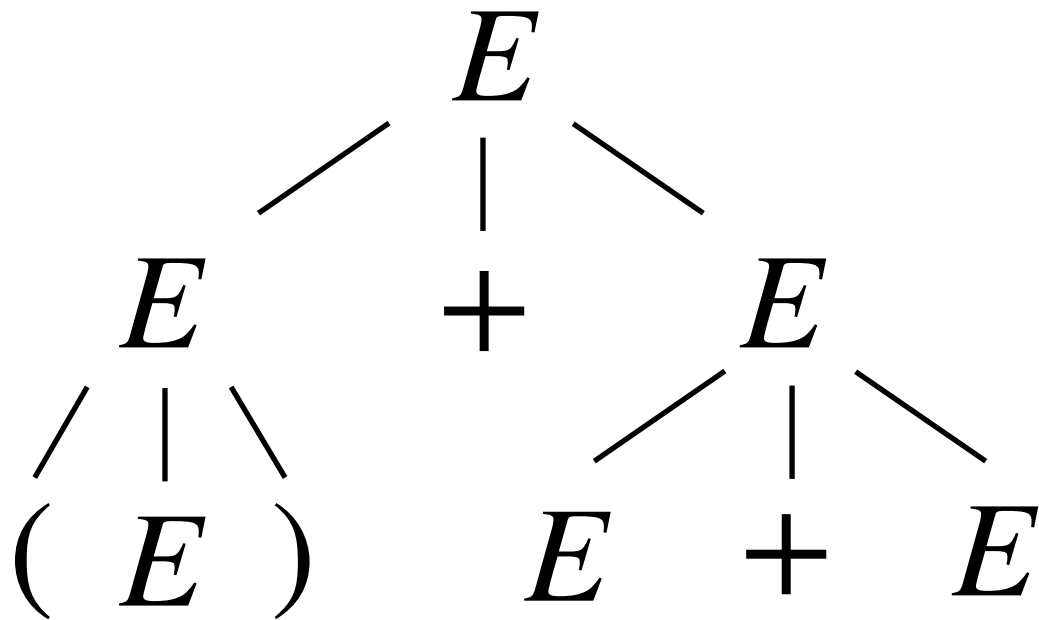
$$\Rightarrow (2) + E - E + E$$

$$\Rightarrow (2) + 5 - E + E$$

$$\Rightarrow (2) + 5 - 3 + E$$

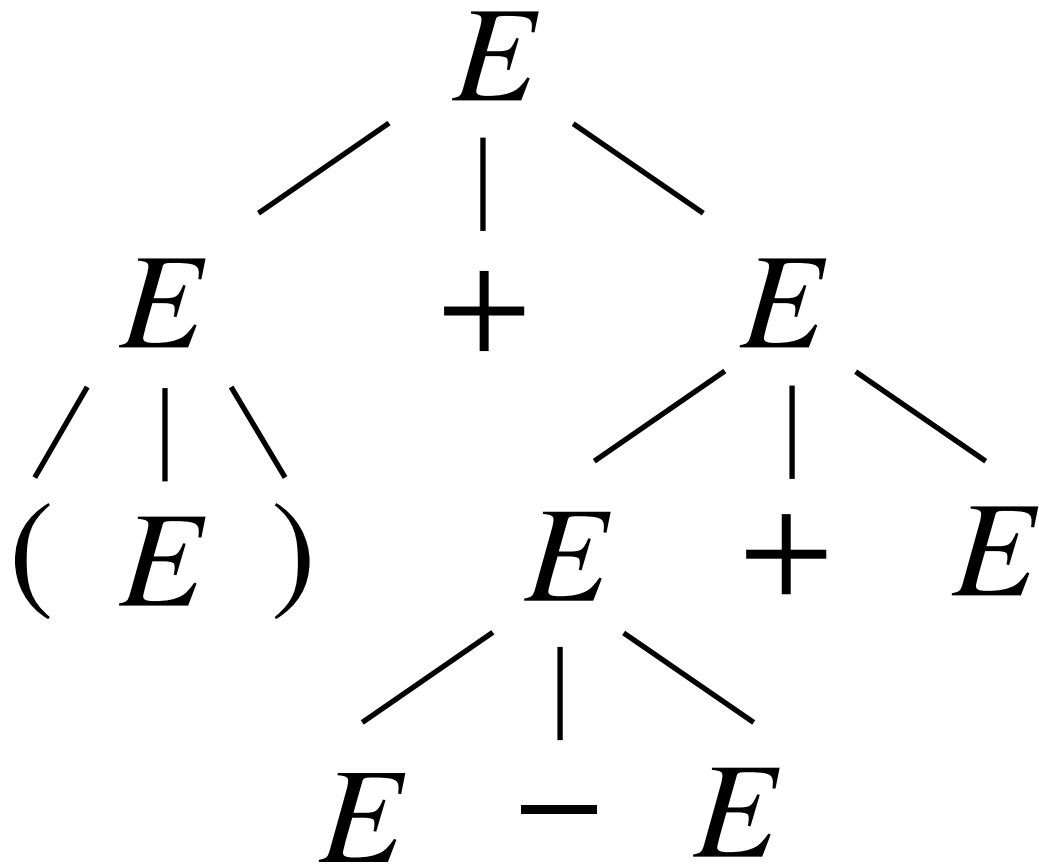
$$\Rightarrow (2) + 5 - 3 + 1$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + E + E \\
 &\Rightarrow (E) + E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$E \Rightarrow E + E$$

$$\Rightarrow E + E + E$$

$$\Rightarrow (E) + E + E$$

$$\Rightarrow (E) + E - E + E$$

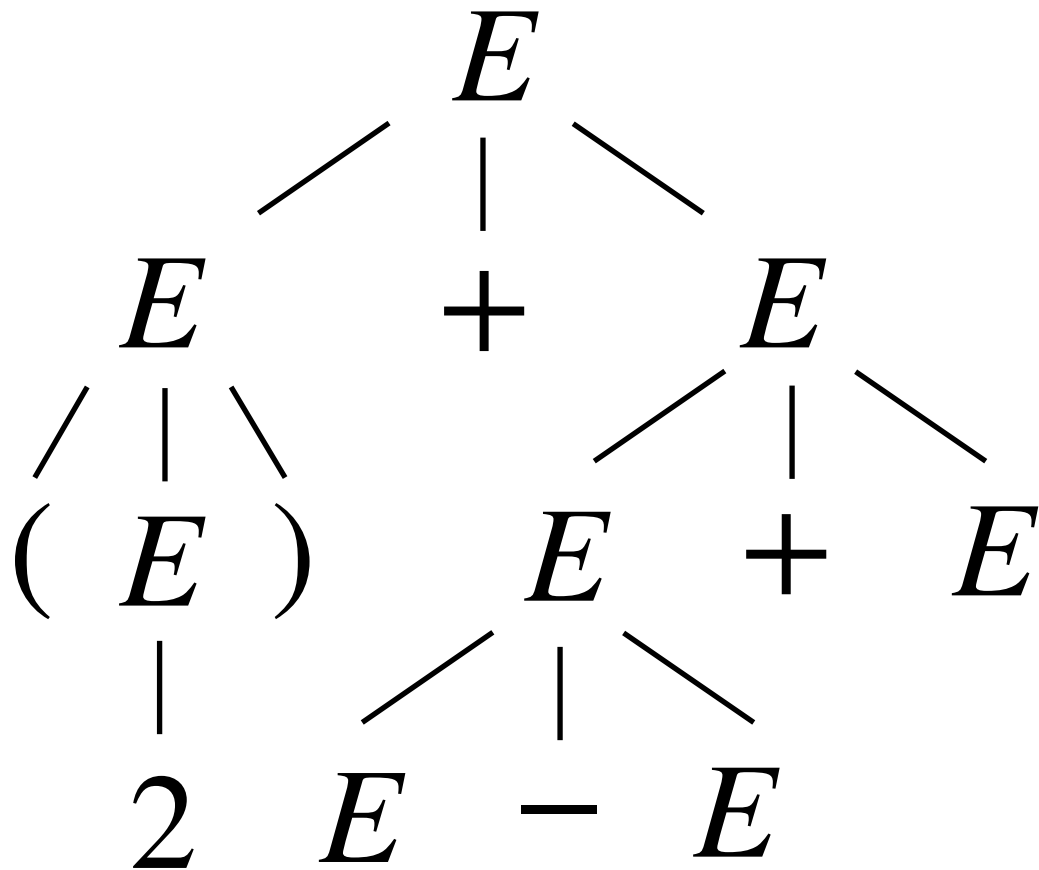
$$\Rightarrow (2) + E - E + E$$

$$\Rightarrow (2) + 5 - E + E$$

$$\Rightarrow (2) + 5 - 3 + E$$

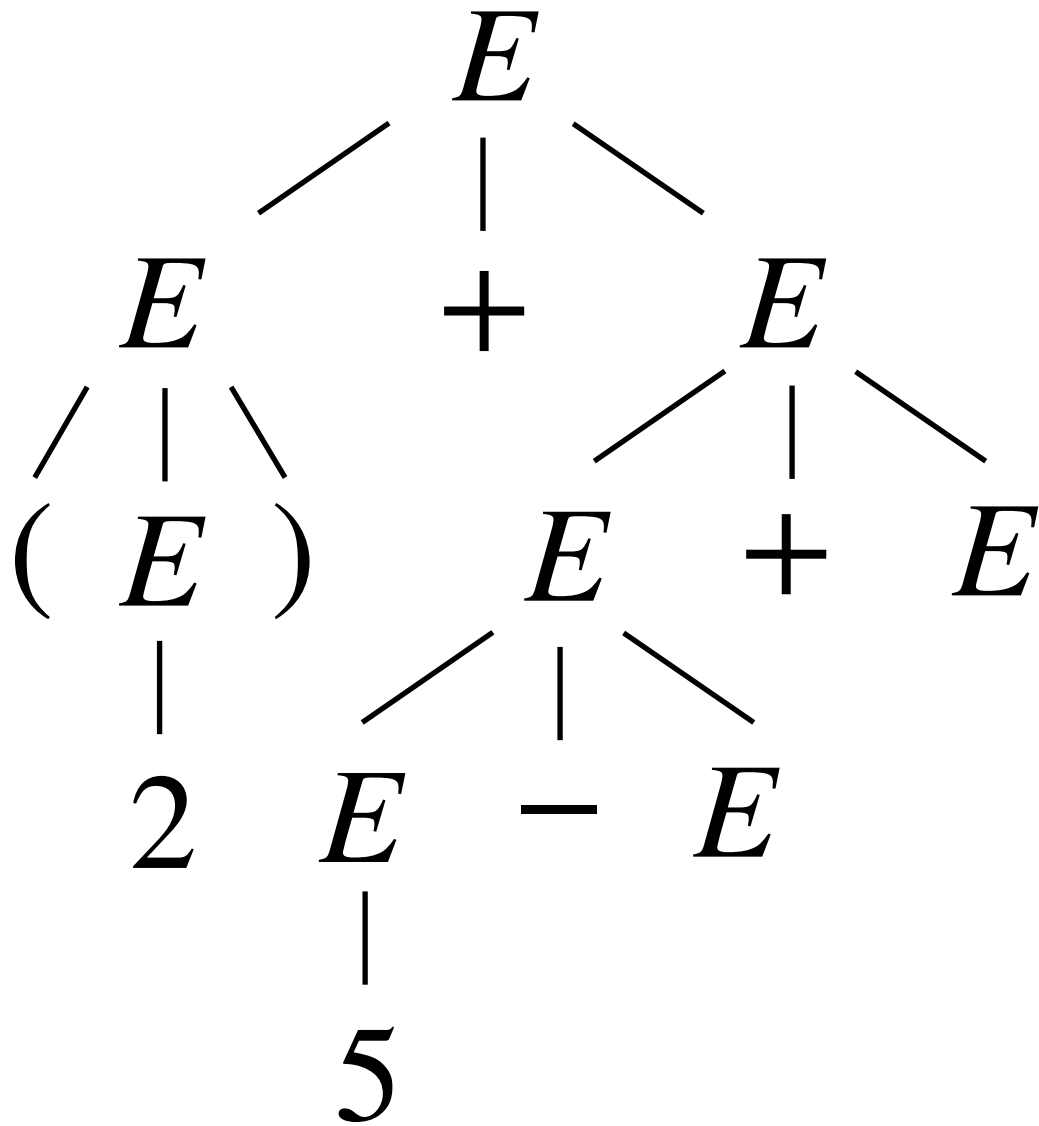
$$\Rightarrow (2) + 5 - 3 + 1$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



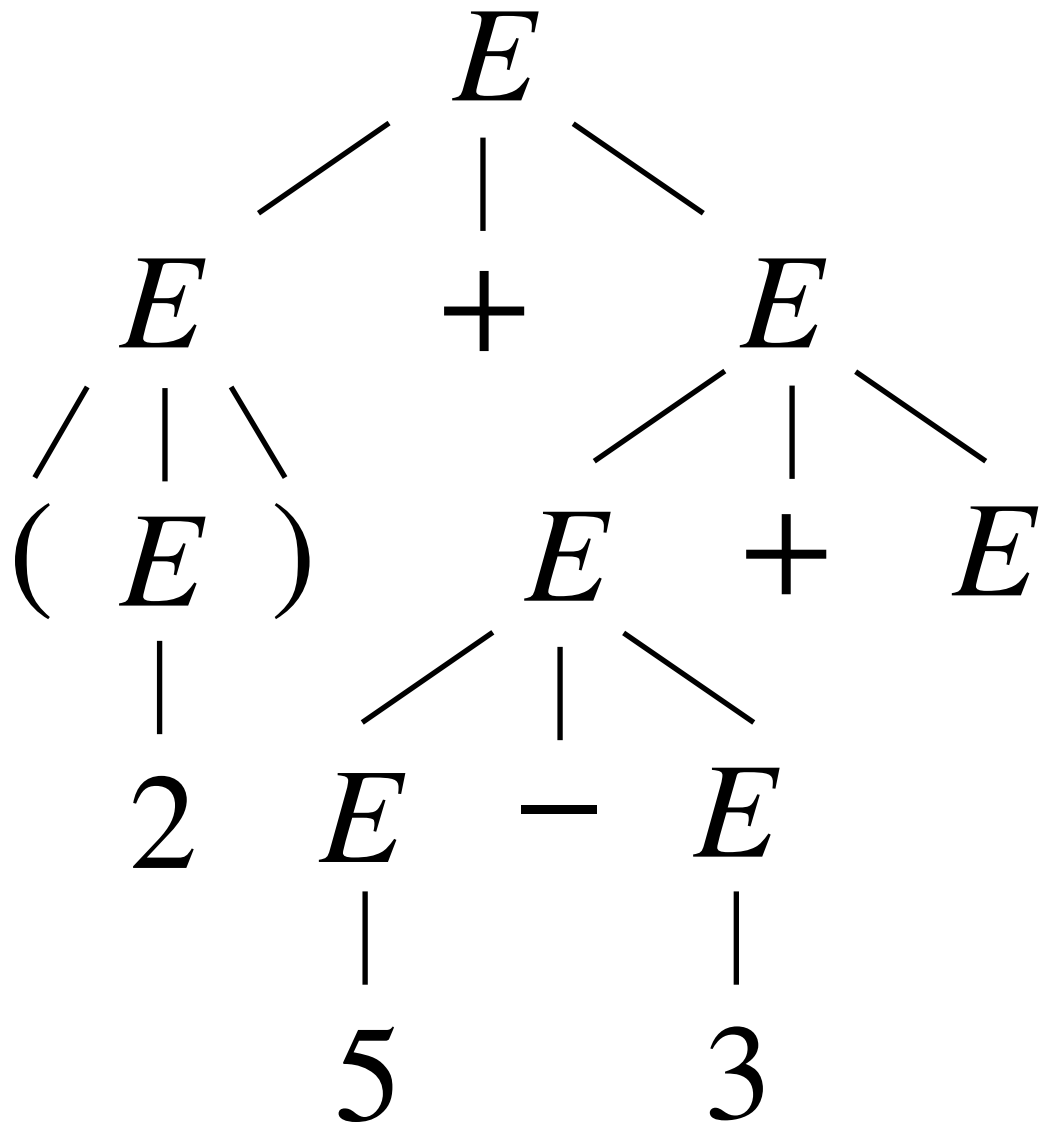
$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + E + E \\
 &\Rightarrow (E) + E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



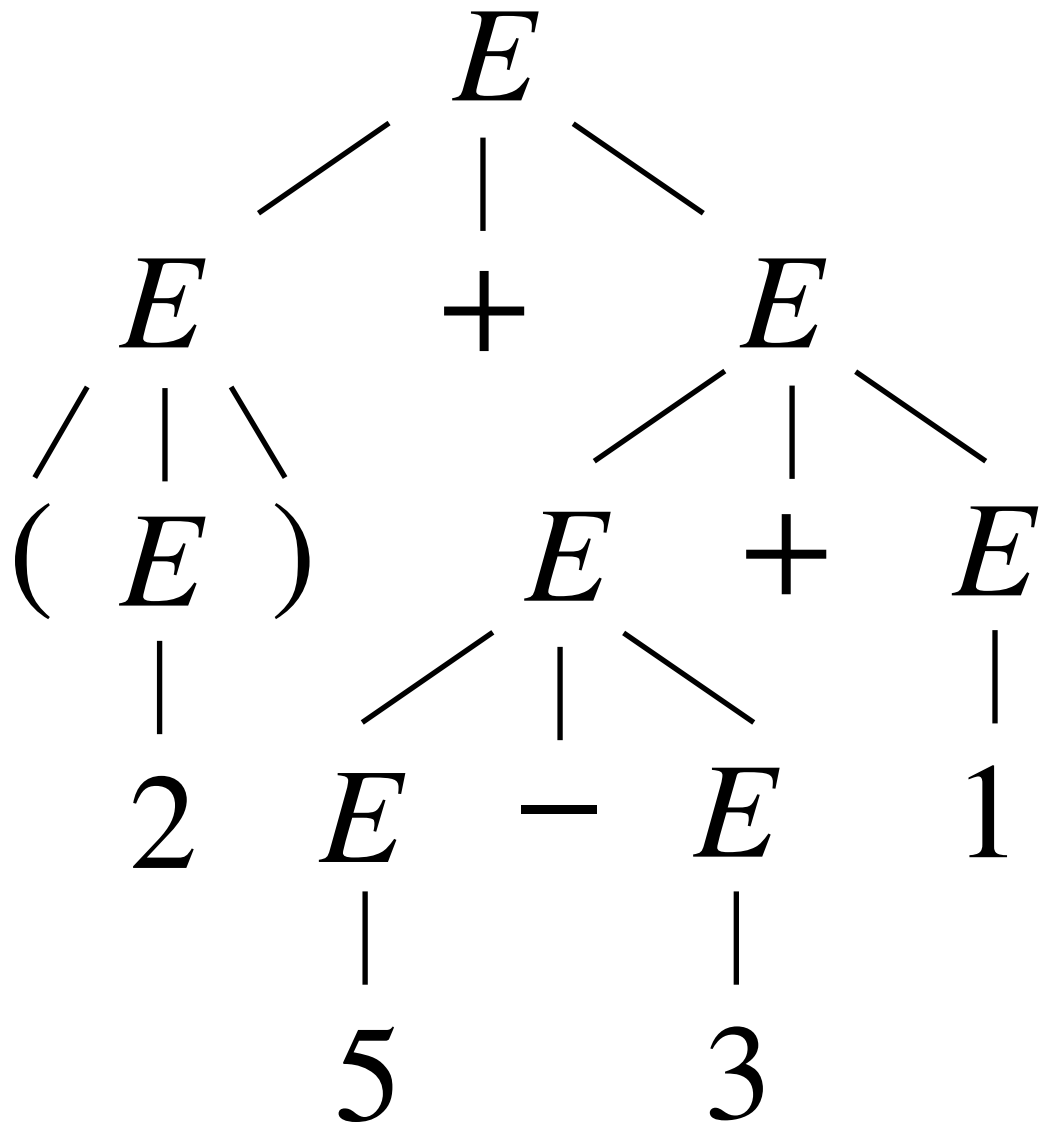
$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + E + E \\
 &\Rightarrow (E) + E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + E + E \\
 &\Rightarrow (E) + E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw a different parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E + E \\
 &\Rightarrow E + E + E \\
 &\Rightarrow (E) + E + E \\
 &\Rightarrow (E) + E - E + E \\
 &\Rightarrow (2) + E - E + E \\
 &\Rightarrow (2) + 5 - E + E \\
 &\Rightarrow (2) + 5 - 3 + E \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Fixing ambiguity

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:
 - E.g., different associativity (***x-y-z***): (***x-y***)-***z*** vs ***x***-(***y-z***)

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:
 - E.g., different associativity (***x-y-z***): (***x-y***)-***z*** vs ***x***-(***y-z***)
 - E.g., different precedence (***x-y*z***): (***x-y***)****z*** vs ***x***-(***y*z***)

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:
 - E.g., different associativity (***x-y-z***): (***x-y***)-***z*** vs ***x***-(***y-z***)
 - E.g., different precedence (***x-y*z***): (***x-y***)****z*** vs ***x***-(***y*z***)
- Two main approaches to fixing ambiguity:

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:
 - E.g., different associativity (***x-y-z***): (***x-y***)-***z*** vs ***x***-(***y-z***)
 - E.g., different precedence (***x-y*z***): (***x-y***)****z*** vs ***x***-(***y*z***)
- Two main approaches to fixing ambiguity:
 - ***Refactoring the grammar***. Many grammars may exist for the same CFL, some could be ambiguous while others are not.

Fixing ambiguity

- Ambiguity in a CFG used for parsing code is *terrible*.
- The same ***syntax*** may have different ***semantics***:
 - E.g., different associativity (***x-y-z***): (***x-y***)-***z*** vs ***x***-(***y-z***)
 - E.g., different precedence (***x-y*z***): (***x-y***)****z*** vs ***x***-(***y*z***)
- Two main approaches to fixing ambiguity:
 - ***Refactoring the grammar***. Many grammars may exist for the same CFL, some could be ambiguous while others are not.
 - Some BNF-style grammars allow special parsing commands to denote associativity/precedence (but this requires tool support).

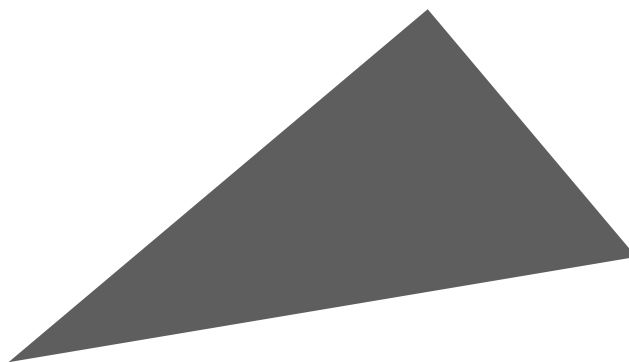
Refactoring our expression grammar

- Stratify expressions into expressions (E) and terms (T), only permitting expressions to recur on the left side of operations.

$$E \rightarrow E * T \mid E - T \mid E + T \mid T$$

$$T \rightarrow (E) \mid n \quad (\text{where } n \in \mathbb{N})$$

- Recurring on the left corresponds to left-associativity because the resulting parse tree will skew/slant left.



Try an example. Draw the parse tree for $(2)+5-3+1$:

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$| E * T$$

$$T \rightarrow (E) | n$$

(where $n \in \mathbb{N}$)

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$| E * T$$

$$T \rightarrow (E) \mid n$$

(where $n \in \mathbb{N}$)

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$E \Rightarrow E + T$$

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$| E * T$$

$$T \rightarrow (E) \mid n$$

(where $n \in \mathbb{N}$)

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$E \rightarrow T \qquad \begin{array}{l} E \Rightarrow E + T \\ \qquad \Rightarrow E - T + T \end{array}$$

$$| E + T$$

$$| E - T$$

$$| E * T$$

$$T \rightarrow (E) \mid n$$

(where $n \in \mathbb{N}$)

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T \\
 \Rightarrow (T) + T - T + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T \\
 \Rightarrow (T) + T - T + T \\
 \Rightarrow (2) + T - T + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T \\
 \Rightarrow (T) + T - T + T \\
 \Rightarrow (2) + T - T + T \\
 \Rightarrow (2) + 5 - T + T
 \end{array}$$

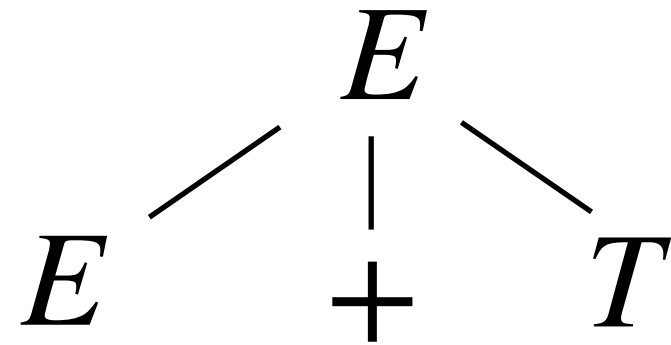
Try an example. Draw the parse tree for $(2)+5-3+1$:

$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T \\
 \Rightarrow (T) + T - T + T \\
 \Rightarrow (2) + T - T + T \\
 \Rightarrow (2) + 5 - T + T \\
 \Rightarrow (2) + 5 - 3 + T
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:

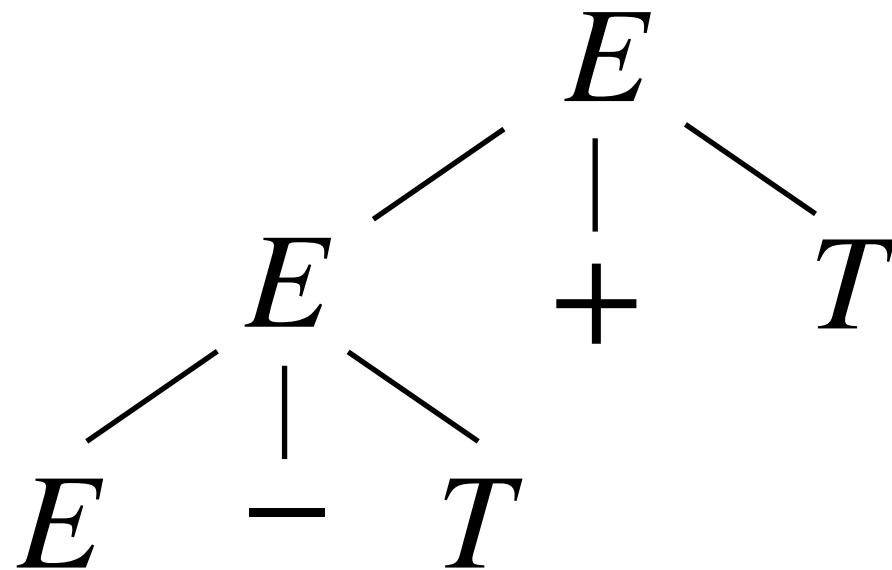
$$\begin{array}{l}
 E \rightarrow T \\
 \quad | E + T \\
 \quad | E - T \\
 \quad | E * T \\
 T \rightarrow (E) \mid n \\
 \text{(where } n \in \mathbb{N})
 \end{array}
 \qquad
 \begin{array}{l}
 E \Rightarrow E + T \\
 \Rightarrow E - T + T \\
 \Rightarrow E + T - T + T \\
 \Rightarrow T + T - T + T \\
 \Rightarrow (E) + T - T + T \\
 \Rightarrow (T) + T - T + T \\
 \Rightarrow (2) + T - T + T \\
 \Rightarrow (2) + 5 - T + T \\
 \Rightarrow (2) + 5 - 3 + T \\
 \Rightarrow (2) + 5 - 3 + 1
 \end{array}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



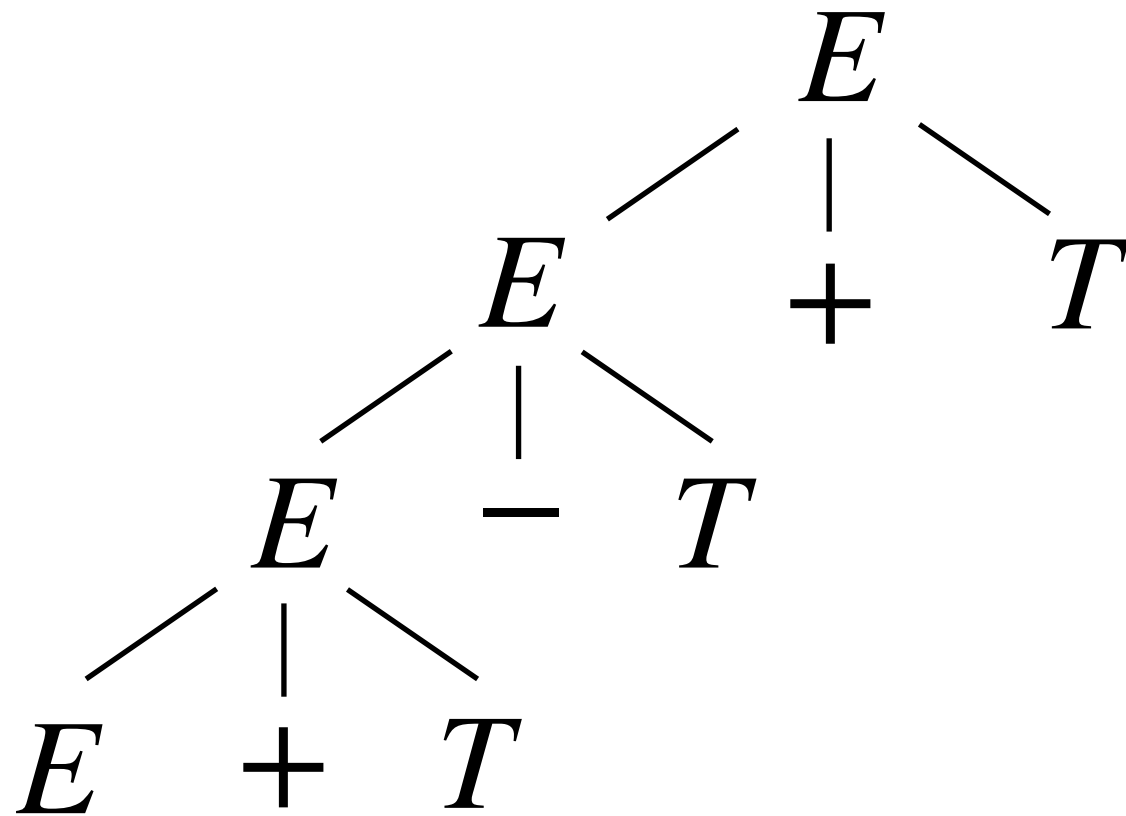
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



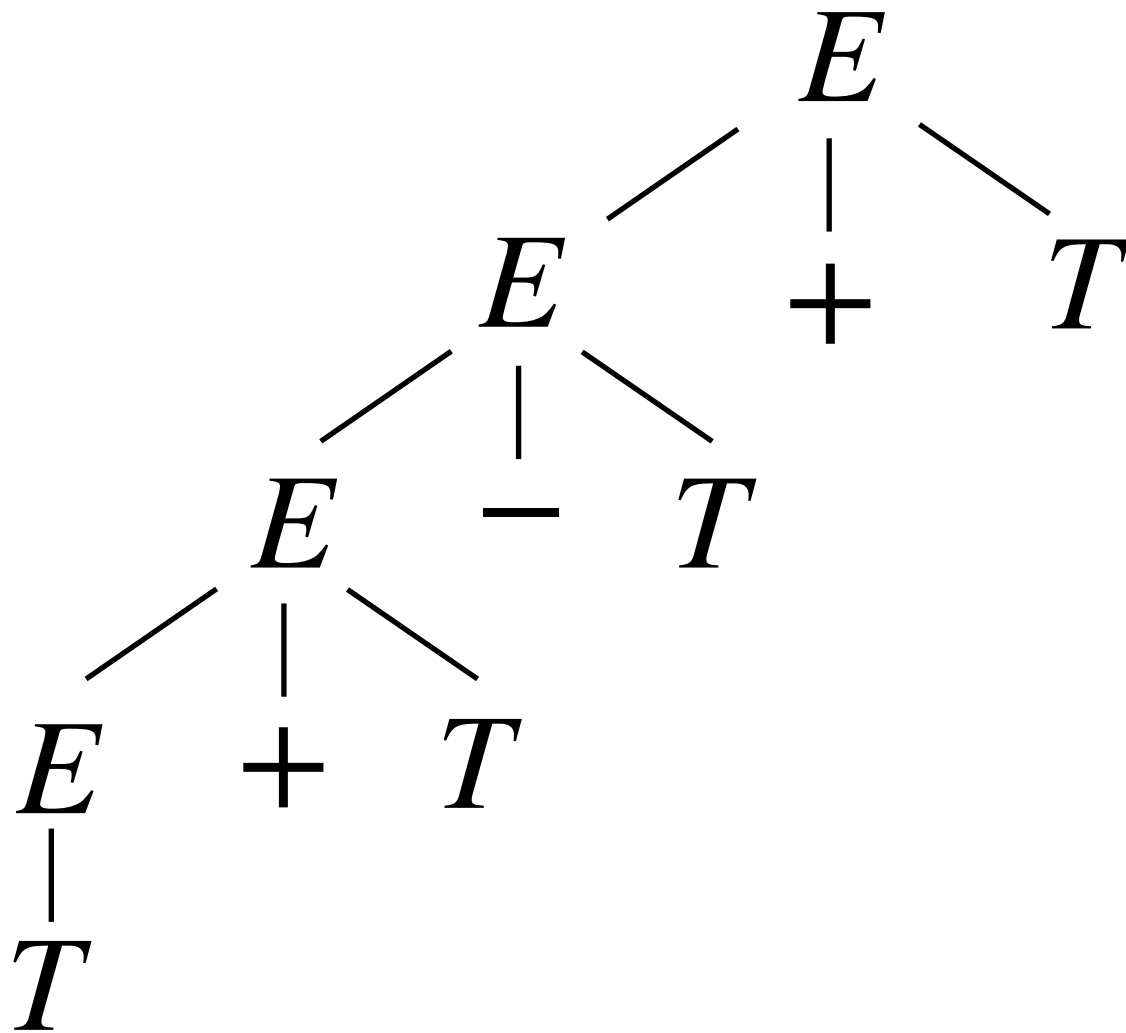
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



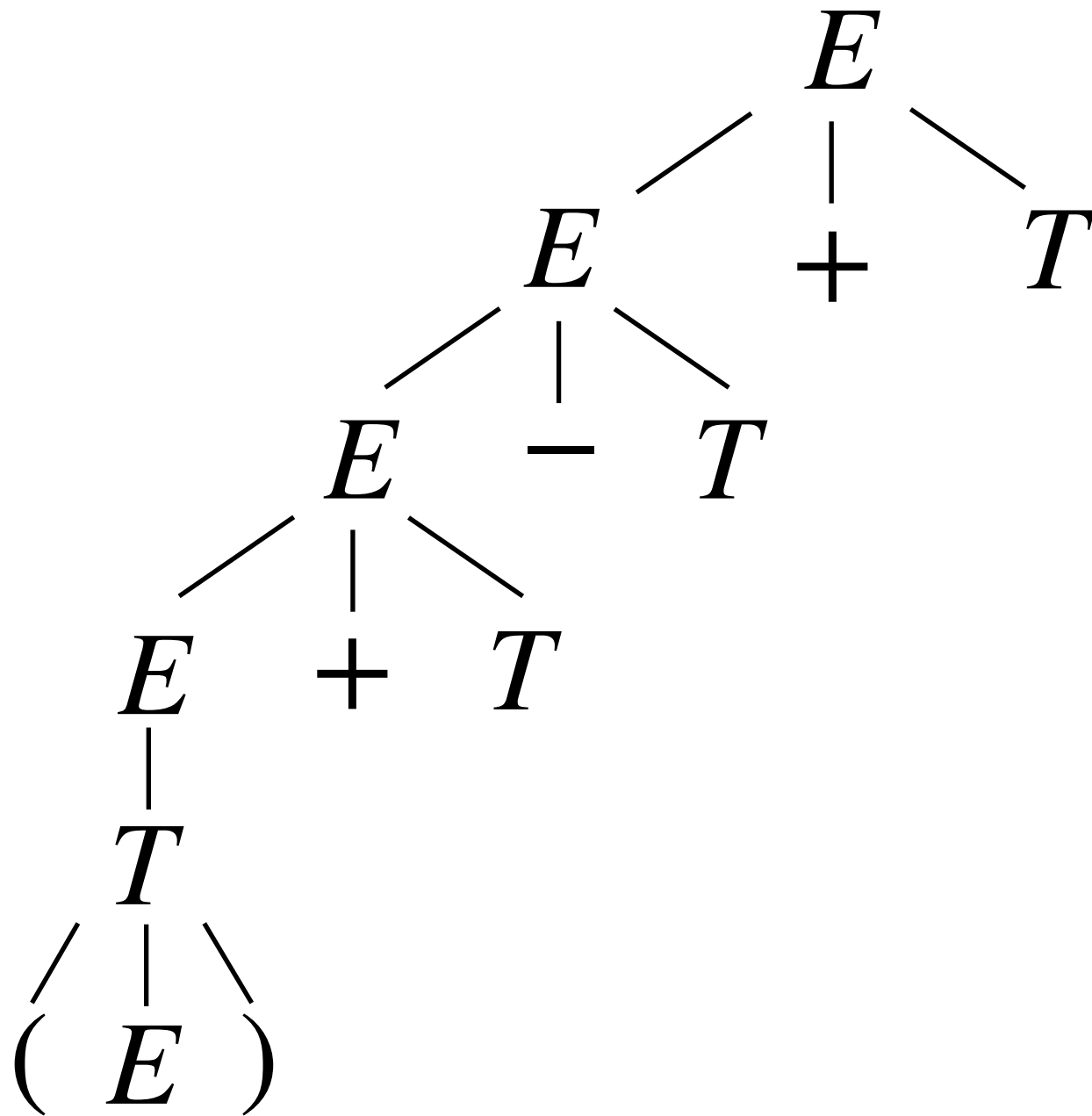
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



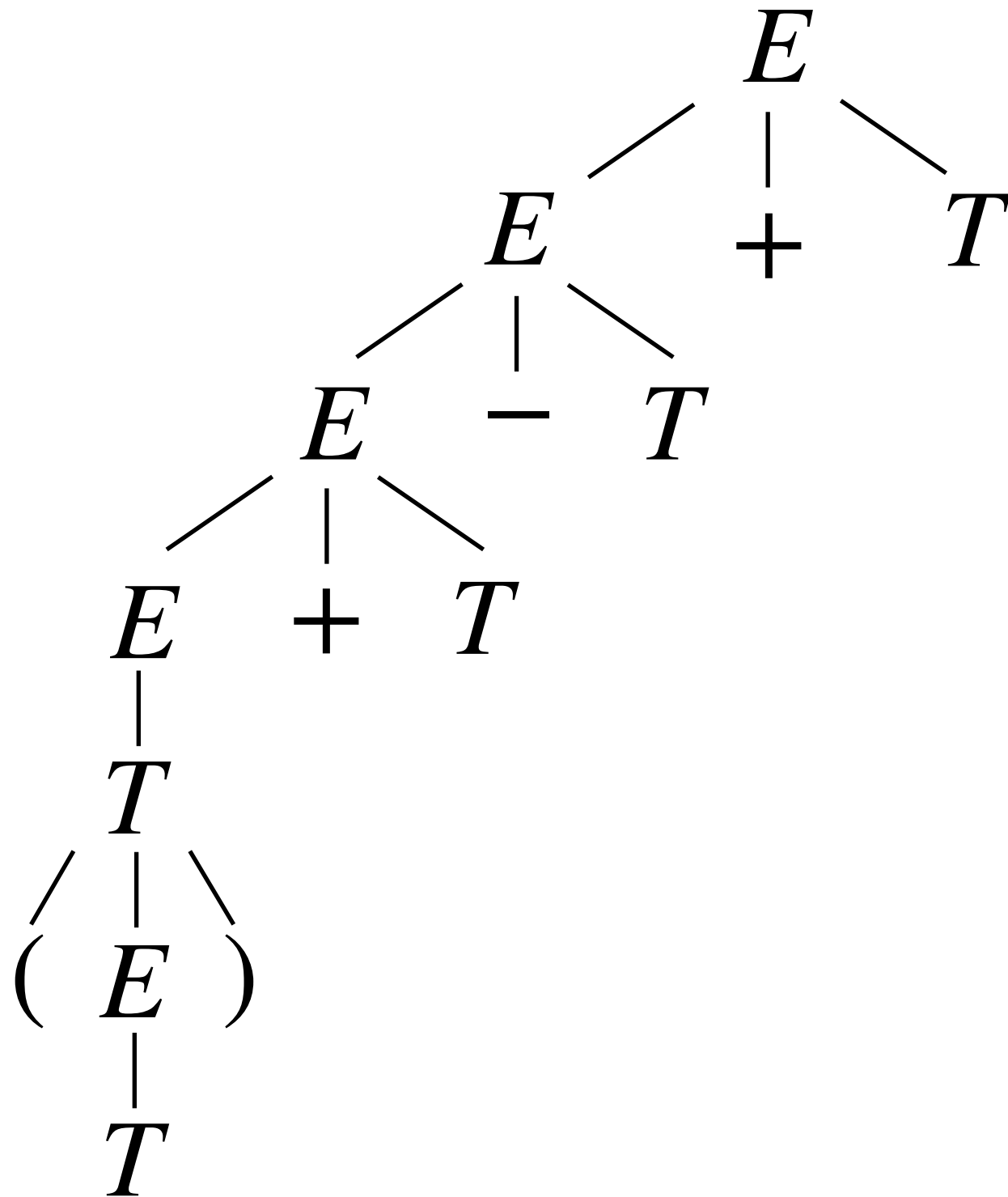
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



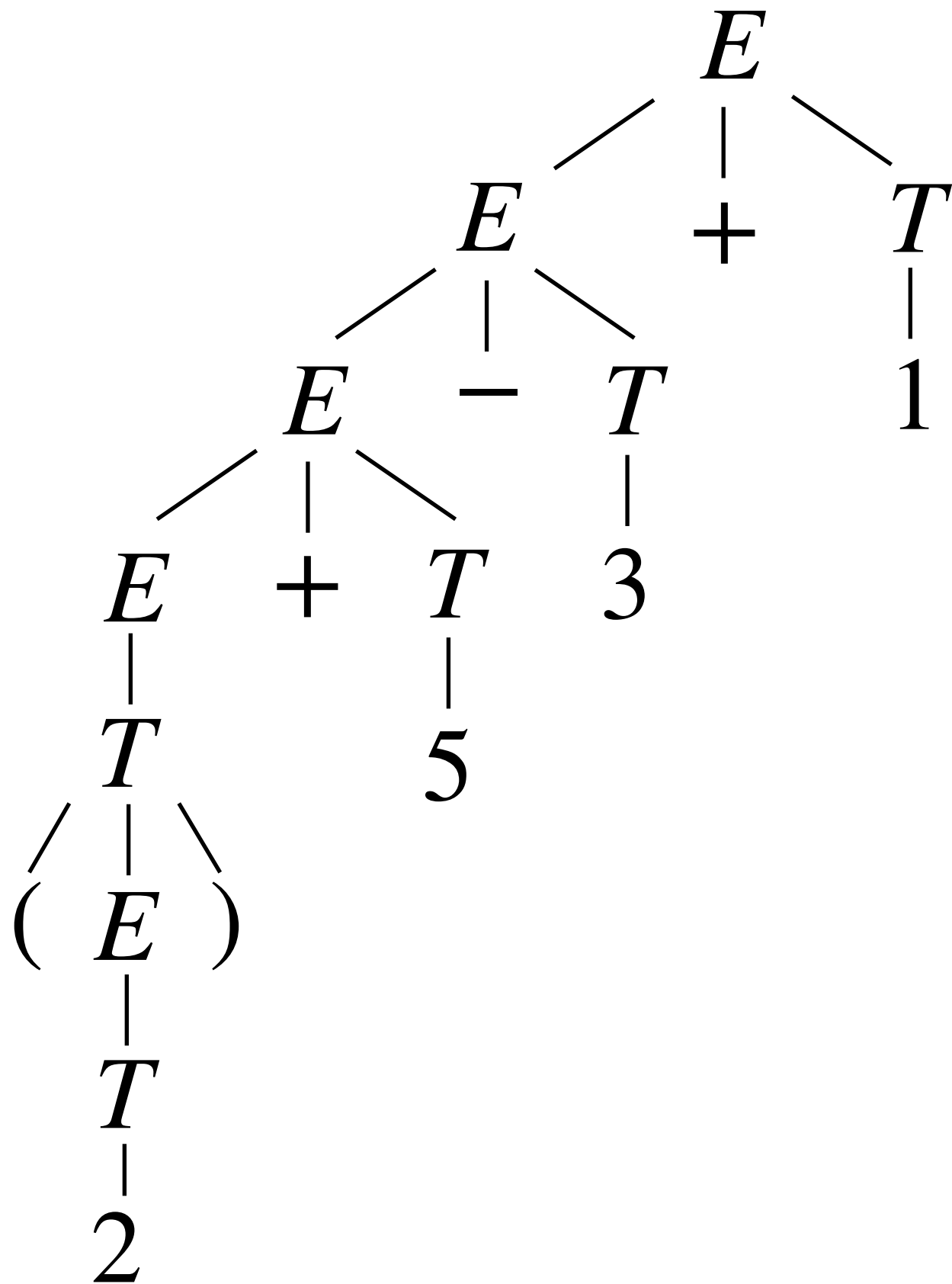
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Try an example. Draw the parse tree for $(2)+5-3+1$:



$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

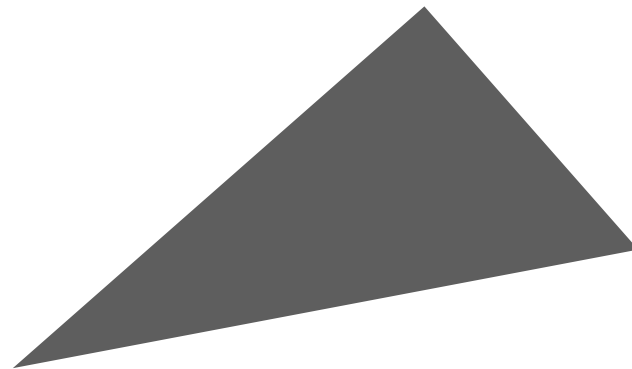
Try an example. Draw the parse tree for $(2)+5-3+1$:



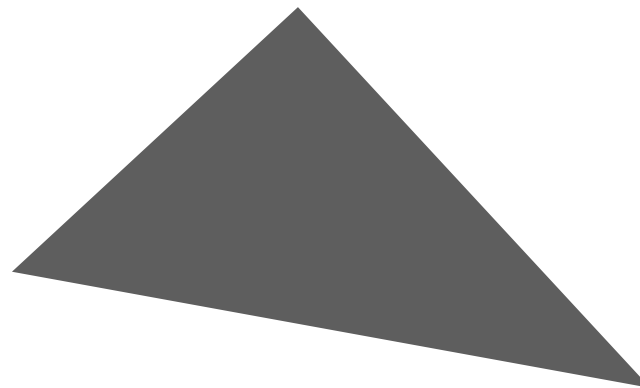
$$\begin{aligned}
 E &\Rightarrow E + T \\
 &\Rightarrow E - T + T \\
 &\Rightarrow E + T - T + T \\
 &\Rightarrow T + T - T + T \\
 &\Rightarrow (E) + T - T + T \\
 &\Rightarrow (T) + T - T + T \\
 &\Rightarrow (2) + T - T + T \\
 &\Rightarrow (2) + 5 - T + T \\
 &\Rightarrow (2) + 5 - 3 + T \\
 &\Rightarrow (2) + 5 - 3 + 1
 \end{aligned}$$

Refactoring our expression grammar

- Use left-recursive productions for ***left associativity***. The left side of the parse-tree will be deeper and at higher precedence!



- Use right-recursive productions for ***right associativity***. The right side of the parse-tree will be at greater depth, and thus at higher precedence!



Refactoring our expression grammar

Refactoring our expression grammar

- What about operators of different precedence? E.g., **4-3*5**

Refactoring our expression grammar

- What about operators of different precedence? E.g., **4-3*5**
- Stratify expressions further into expressions (E), terms (T), ***and factors (F)***, only permitting recurrence on the left side of operations and strictly ordering E, T, and F.

$$E \rightarrow E - T \mid E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid n \quad (\text{where } n \in \mathbb{N})$$

Refactoring our expression grammar

- What about operators of different precedence? E.g., **4-3*5**
- Stratify expressions further into expressions (E), terms (T), ***and factors (F)***, only permitting recurrence on the left side of operations and strictly ordering E, T, and F.

$$E \rightarrow E - T \mid E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid n \quad (\text{where } n \in \mathbb{N})$$

- Stratification of nonterminals in a grammar can be used to control the precedence of operations or forms in the resulting AST. Operations introduced by the same nonterminal will have equal precedence (e.g., addition and subtraction).

Try an example. Add exponentiation (^) to this grammar.

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$T \rightarrow T * F | F$$

$$F \rightarrow (E) | n$$

(where $n \in \mathbb{N}$)

Try an example. Add exponentiation (^) to this grammar.

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$T \rightarrow T * F | F$$

$$F \rightarrow A^{\wedge} F | A$$

$$A \rightarrow (E) | n \quad (\text{where } n \in \mathbb{N})$$

Implementing parsers

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.
 - E.g., **LL(k)**, SLR(k), LR(k), LALR(k), ...

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.
 - E.g., **LL(k)**, SLR(k), LR(k), LALR(k), ...
 - ***LL(k) parsing*** processes **L**eft-to-right, obtains the **L**eftmost derivation, and may look ahead **k** tokens.

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.
 - E.g., **LL(k)**, SLR(k), LR(k), LALR(k), ...
 - ***LL(k) parsing*** processes **L**eft-to-right, obtains the **L**eftmost derivation, and may look ahead **k** tokens.
- There are two major classes of parsing algorithms: ***bottom-up***, and ***top-down***.

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.
 - E.g., **LL(k)**, SLR(k), LR(k), LALR(k), ...
 - ***LL(k) parsing*** processes **L**eft-to-right, obtains the **L**eftmost derivation, and may look ahead **k** tokens.
- There are two major classes of parsing algorithms: ***bottom-up***, and ***top-down***.
 - ***Recursive descent parsing*** is an *intuitive* and elegant approach to top-down parsing that yields *good errors*.

Implementing parsers

- There are many parsing algorithms (for turning strings into ASTs or failing), each with distinct advantages and disadvantages.
 - E.g., **LL(k)**, SLR(k), LR(k), LALR(k), ...
 - ***LL(k) parsing*** processes **L**eft-to-right, obtains the **L**eftmost derivation, and may look ahead **k** tokens.
- There are two major classes of parsing algorithms: ***bottom-up***, and ***top-down***.
 - ***Recursive descent parsing*** is an *intuitive* and elegant approach to top-down parsing that yields *good errors*.
 - Other parsing algorithms are mostly bottom-up.

Top-down and bottom-up parsers

Top-down and bottom-up parsers

- ***Bottom-up*** (i.e., ***shift-reduce***) parsing starts with the string of terminals, and coalesces substrings into nonterminals.

Top-down and bottom-up parsers

- ***Bottom-up*** (i.e., ***shift-reduce***) parsing starts with the string of terminals, and coalesces substrings into nonterminals.
 - This attempts to find a derivation *in reverse*!

Top-down and bottom-up parsers

- ***Bottom-up*** (i.e., ***shift-reduce***) parsing starts with the string of terminals, and coalesces substrings into nonterminals.
 - This attempts to find a derivation *in reverse*!
 - Somewhat complex; requires tool support; bad errors.

Top-down and bottom-up parsers

- **Bottom-up** (i.e., **shift-reduce**) parsing starts with the string of terminals, and coalesces substrings into nonterminals.
 - This attempts to find a derivation *in reverse*!
 - Somewhat complex; requires tool support; bad errors.
- **Top-down** parsing starts with the grammar's start symbol and attempts to discover a parse tree top-down—in forward order.

Top-down and bottom-up parsers

- **Bottom-up** (i.e., **shift-reduce**) parsing starts with the string of terminals, and coalesces substrings into nonterminals.
 - This attempts to find a derivation *in reverse*!
 - Somewhat complex; requires tool support; bad errors.
- **Top-down** parsing starts with the grammar's start symbol and attempts to discover a parse tree top-down—in forward order.
 - Recursive descent parsing requires a constrained form of grammar and is written much the way you might if following a grammar ad hoc.

Top-down and bottom-up parsers

- **Bottom-up** (i.e., **shift-reduce**) parsing starts with the string of terminals, and coalesces substrings into nonterminals.
 - This attempts to find a derivation *in reverse*!
 - Somewhat complex; requires tool support; bad errors.
- **Top-down** parsing starts with the grammar's start symbol and attempts to discover a parse tree top-down—in forward order.
 - Recursive descent parsing requires a constrained form of grammar and is written much the way you might if following a grammar ad hoc.
 - Simple, intuitive, fast, gives better error messages.

Recursive descent parsing

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.
 - A position in the string and the next k ***lookahead*** tokens.

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.
 - A position in the string and the next k ***lookahead*** tokens.
 - An incomplete parse-tree (or AST) for the tokens already seen, encoded on the stack.

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.
 - A position in the string and the next k ***lookahead*** tokens.
 - An incomplete parse-tree (or AST) for the tokens already seen, encoded on the stack.
- At each step there are 2 valid cases, otherwise parsing fails.

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.
 - A position in the string and the next k ***lookahead*** tokens.
 - An incomplete parse-tree (or AST) for the tokens already seen, encoded on the stack.
- At each step there are 2 valid cases, otherwise parsing fails.
 - If matching a *terminal*: check lookahead and advance.

Recursive descent parsing

- The CFG defines a set of parse trees. Recursive descent parsing explores this space to find the valid parse tree for the given string top-down and left-to-right. At each step we track:
 - The nonterminal we're parsing next.
 - A position in the string and the next k ***lookahead*** tokens.
 - An incomplete parse-tree (or AST) for the tokens already seen, encoded on the stack.
- At each step there are 2 valid cases, otherwise parsing fails.
 - If matching a *terminal*: check lookahead and advance.
 - If matching a *nonterminal*: pick appropriate production.

Recursive descent parsing

Recursive descent parsing

- The main difficulty in writing a recursive descent parser is selecting the correct production to apply at a nonterminal.

Recursive descent parsing

- The main difficulty in writing a recursive descent parser is selecting the correct production to apply at a nonterminal.
- There are two major approaches to this:

Recursive descent parsing

- The main difficulty in writing a recursive descent parser is selecting the correct production to apply at a nonterminal.
- There are two major approaches to this:
 - ***Backtracking:*** Select a production (arbitrarily) to speculatively apply, but permit parsing to backtrack at a failure. An error only occurs when the whole parse fails.

Recursive descent parsing

- The main difficulty in writing a recursive descent parser is selecting the correct production to apply at a nonterminal.
- There are two major approaches to this:
 - ***Backtracking:*** Select a production (arbitrarily) to speculatively apply, but permit parsing to backtrack at a failure. An error only occurs when the whole parse fails.
 - ***Predictive parsing:*** analyze the grammar ahead of time to determine which lookahead tokens correspond to which production(s)—if only one, no backtracking!

Recursive descent parsing

- The main difficulty in writing a recursive descent parser is selecting the correct production to apply at a nonterminal.
- There are two major approaches to this:
 - ***Backtracking:*** Select a production (arbitrarily) to speculatively apply, but permit parsing to backtrack at a failure. An error only occurs when the whole parse fails.
 - ***Predictive parsing:*** analyze the grammar ahead of time to determine which lookahead tokens correspond to which production(s)—if only one, no backtracking!
- Predictive parsing is much faster but may require refactoring.

Recursive descent parsing for $\{x = 5; y=\{2;\}; (x+y);\}$

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



E

$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



E

$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



$\{$
 $/$
 E

$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



$\begin{matrix} E \\ / \mid \\ \{ L \end{matrix}$

$$E \rightarrow x = E \mid (E + E) \mid \{L\} \mid n \mid x$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



$$\begin{array}{c} E \\ / \quad | \\ \{ \quad L \\ / \\ E \end{array}$$

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



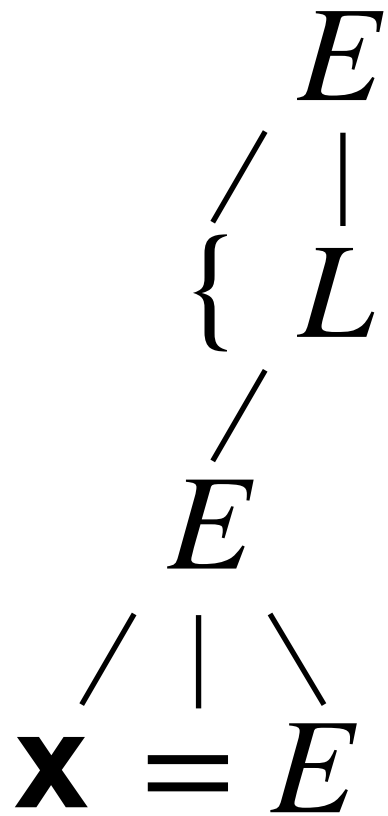
$$\begin{array}{c} E \\ / \quad | \\ \{ \quad L \\ / \\ E \end{array}$$

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

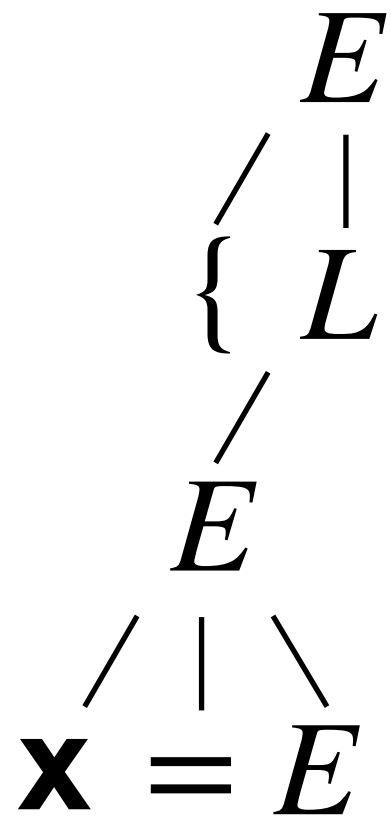


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

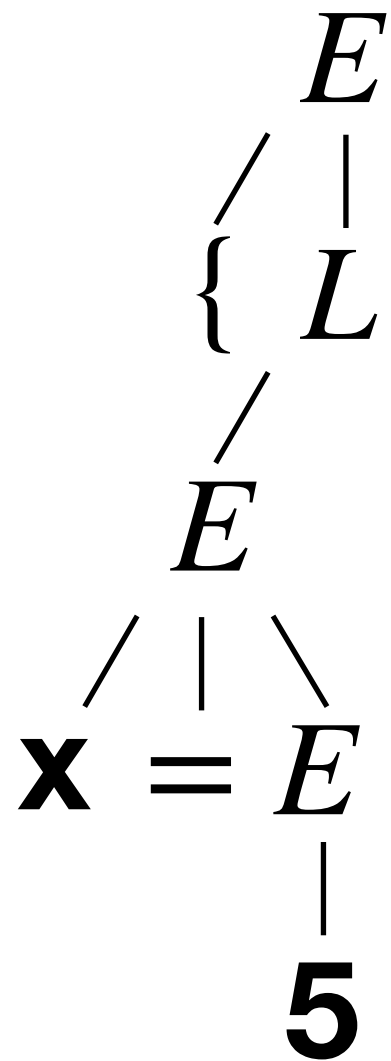


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

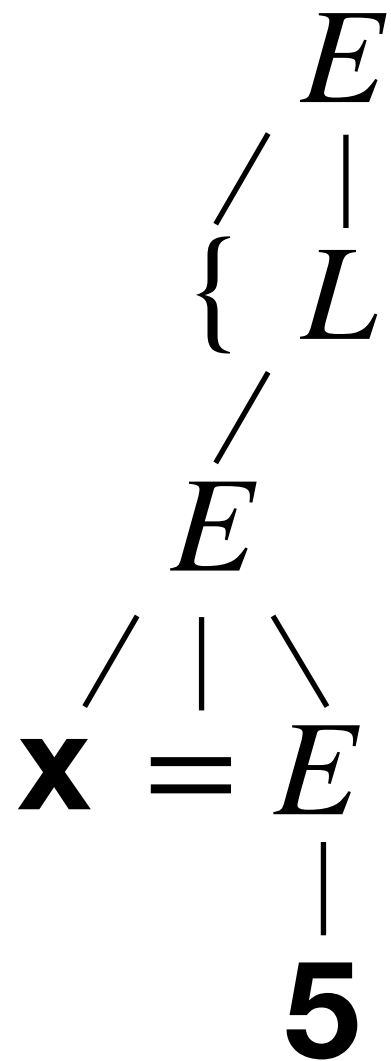


$$\begin{array}{l}
 \uparrow \\
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

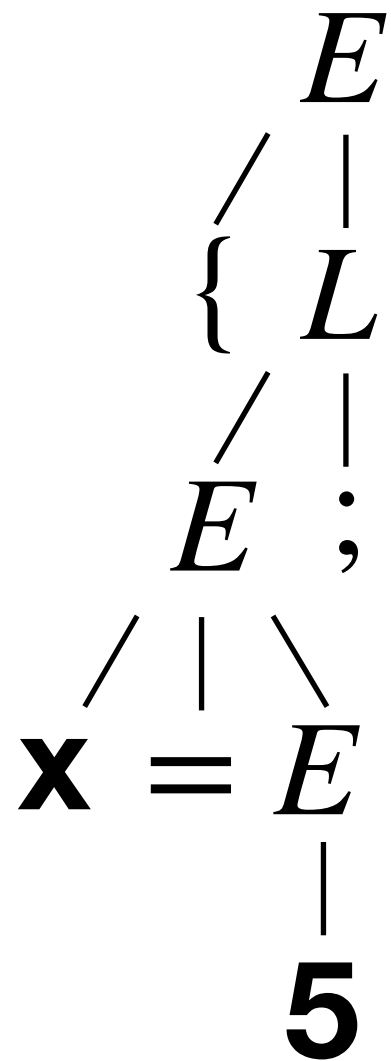


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

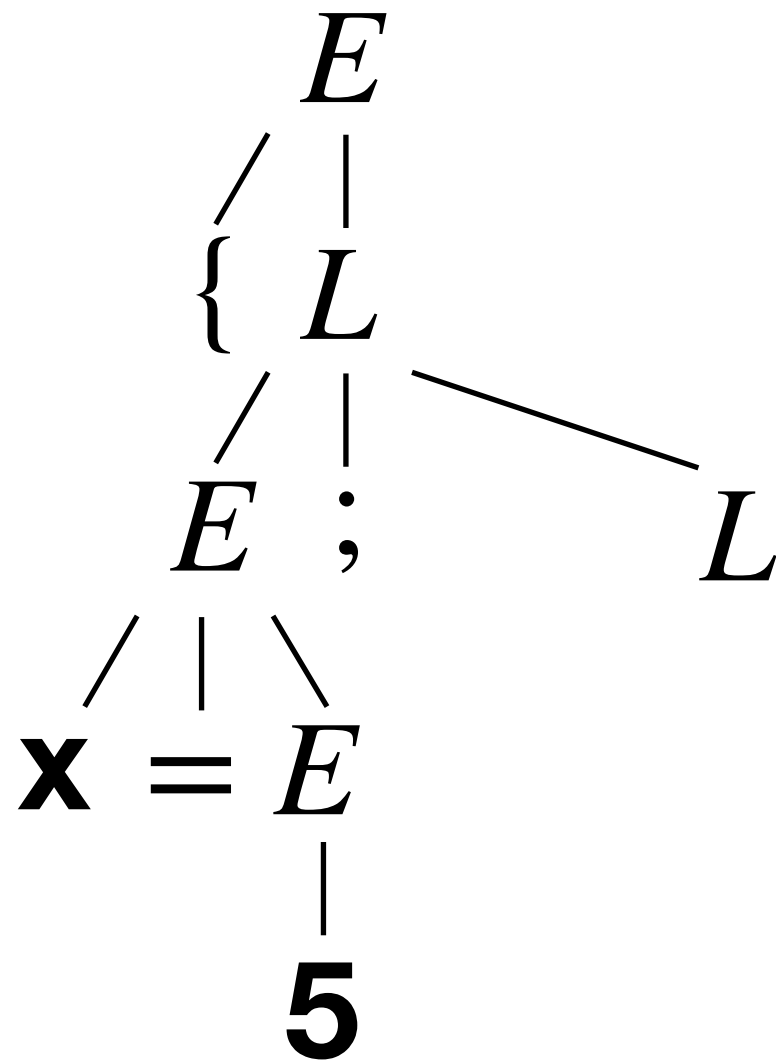


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

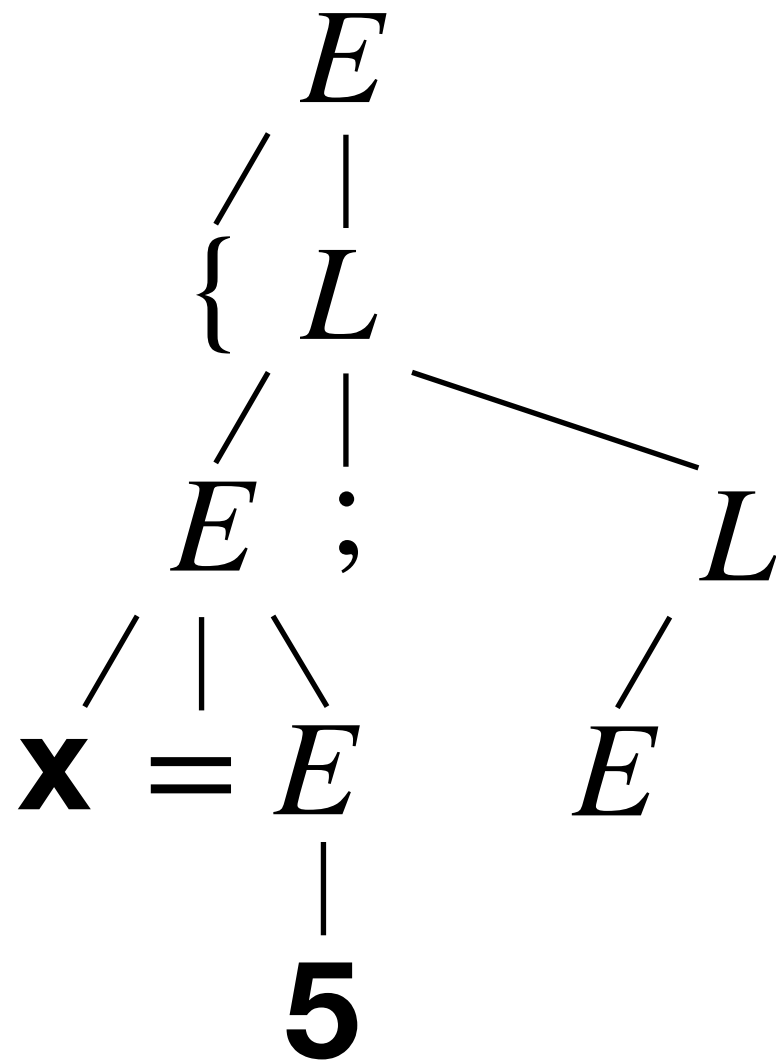


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

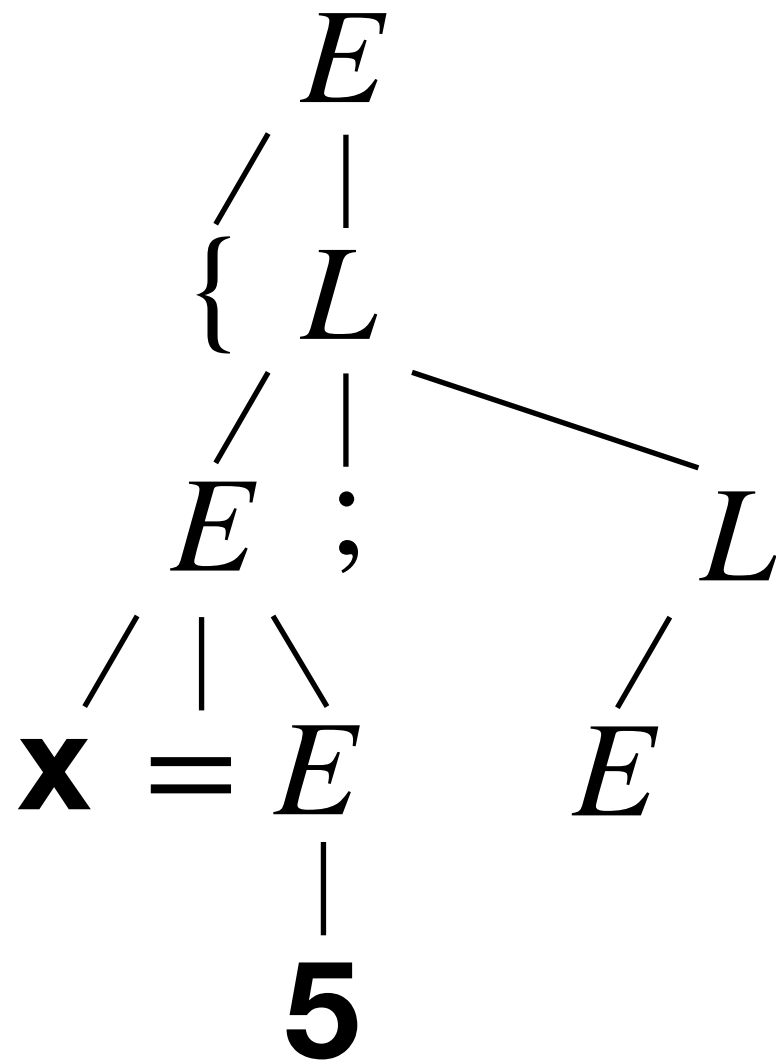


$$E \rightarrow x = E \mid (E + E) \mid \{L\} \mid n \mid x$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y=\{2;\}; (x+y);\}$

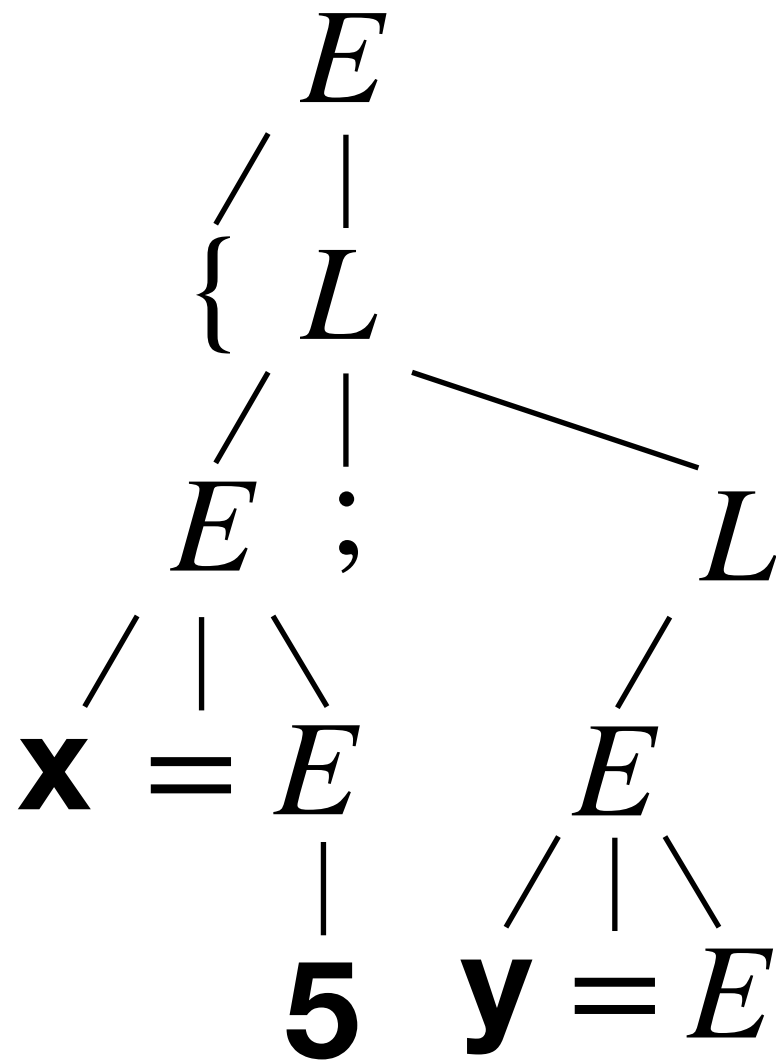


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

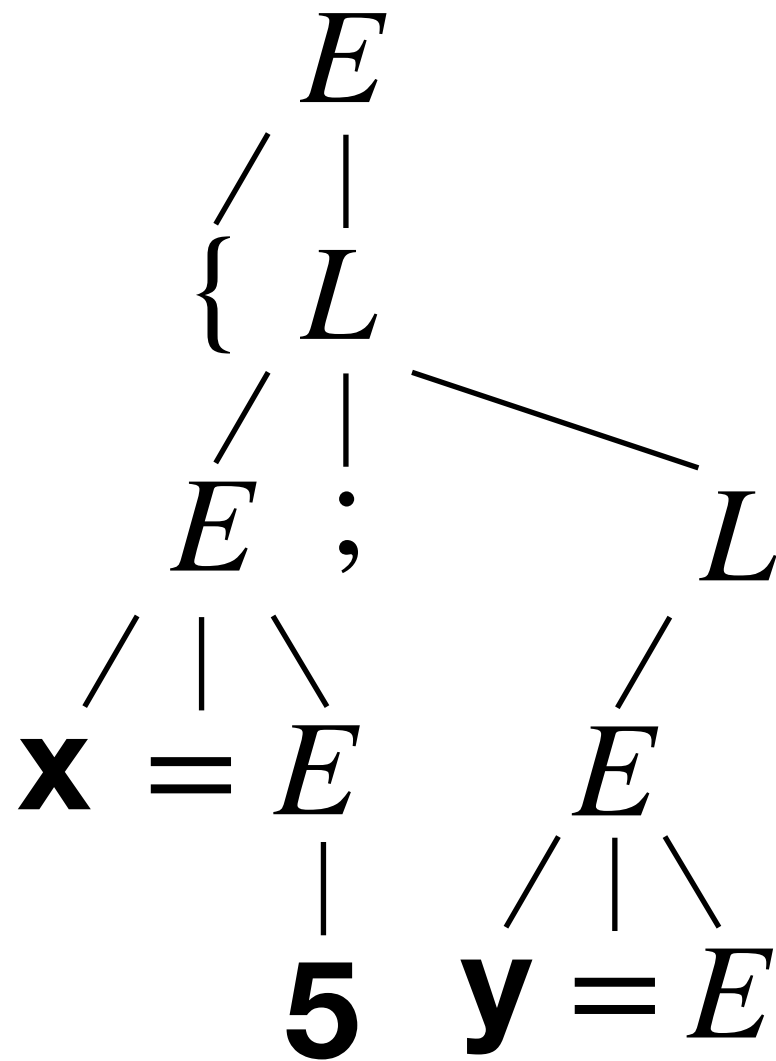


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

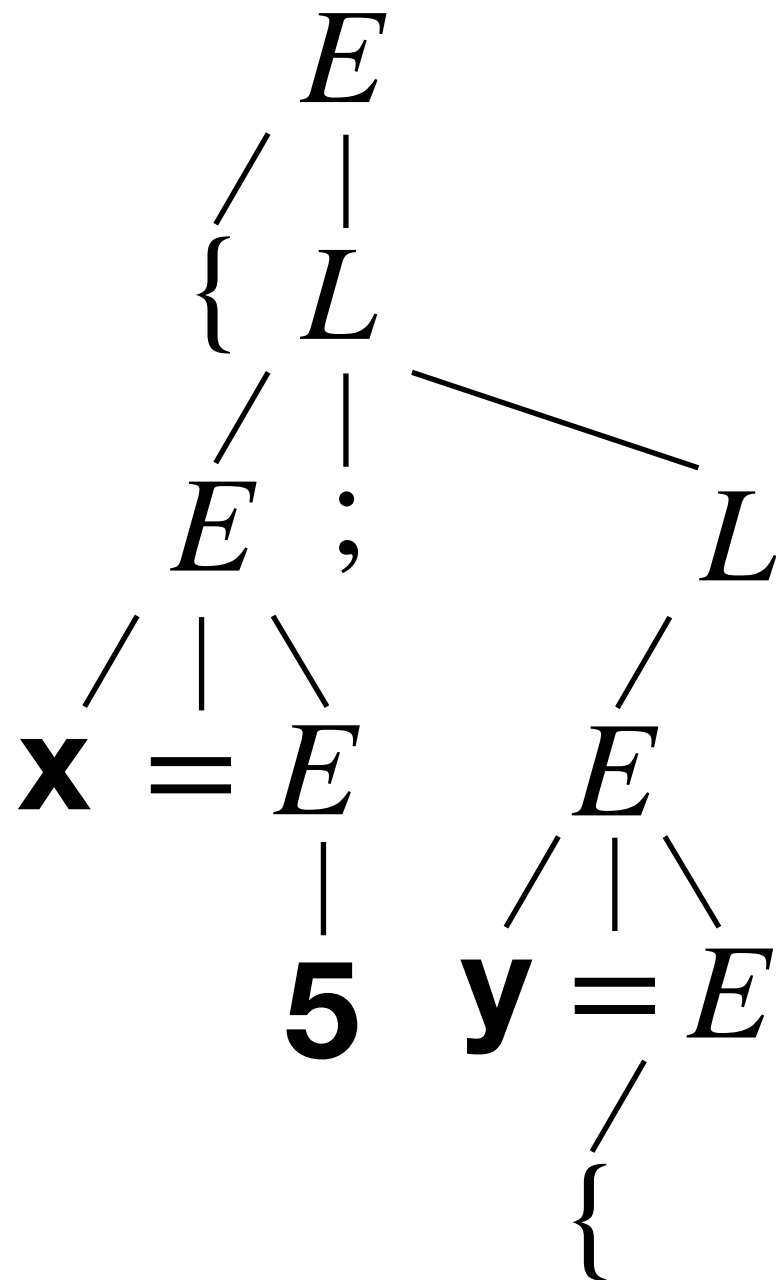


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

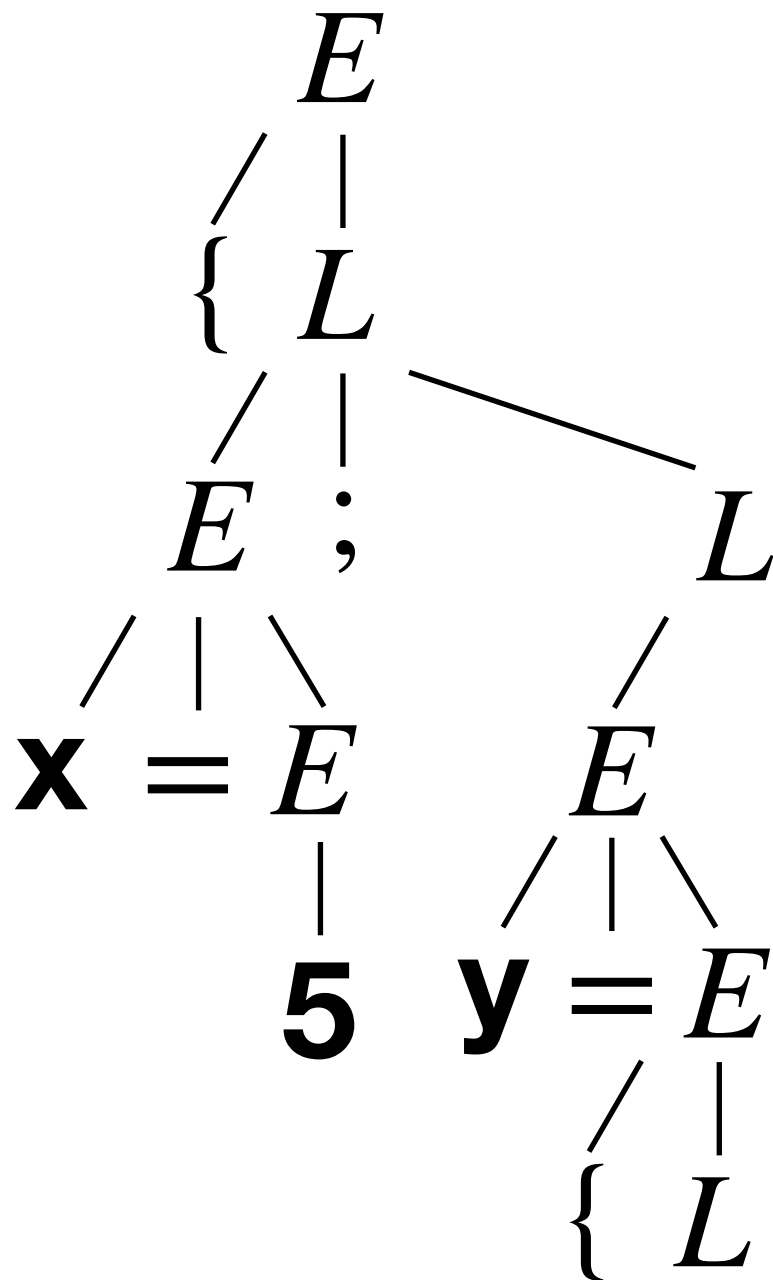
(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$



$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x \\
 L &\rightarrow E; L \mid \epsilon
 \end{aligned}$$

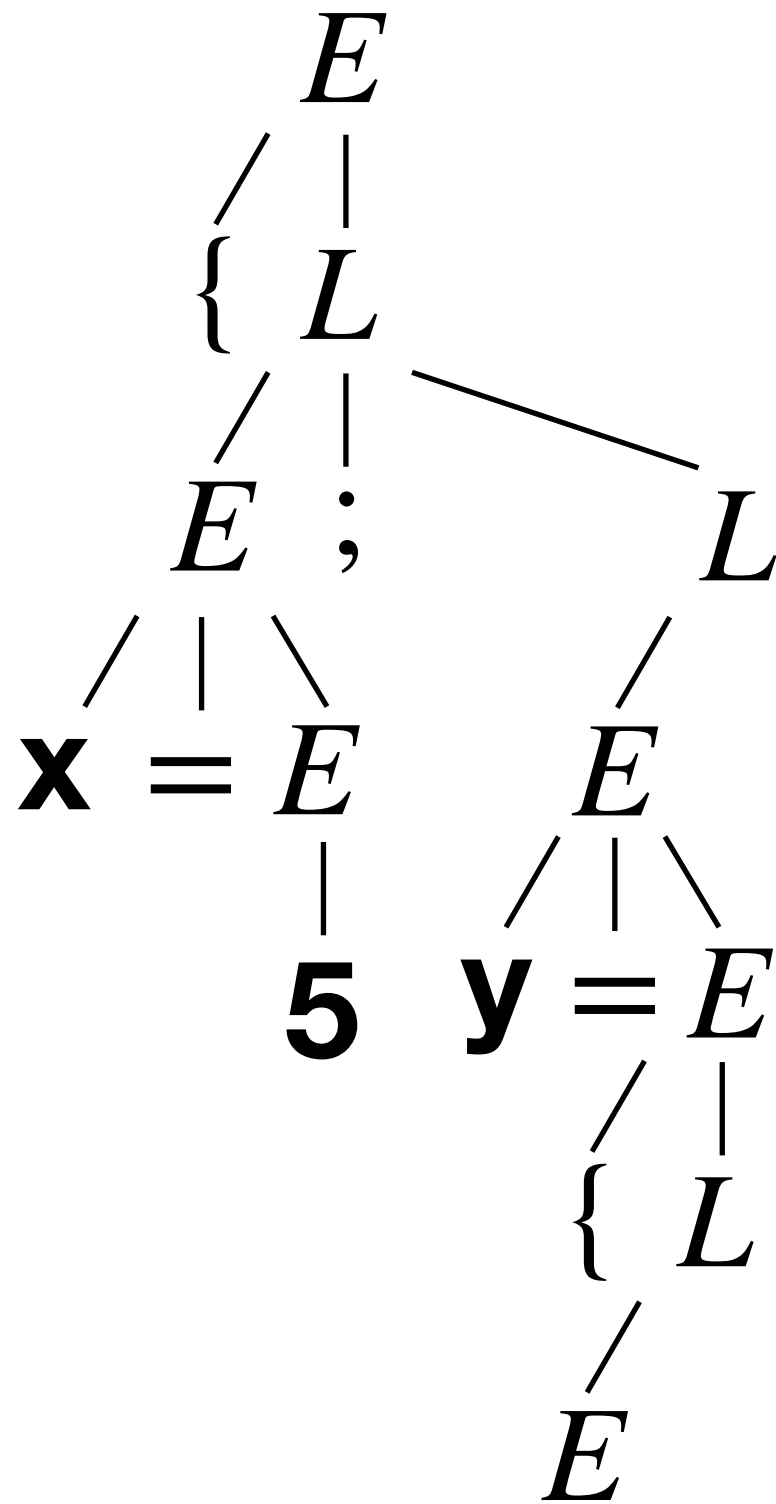
(where x is an ID, and n is a NAT)



$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x \\
 L \rightarrow E; L \mid \epsilon
 \end{array}$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

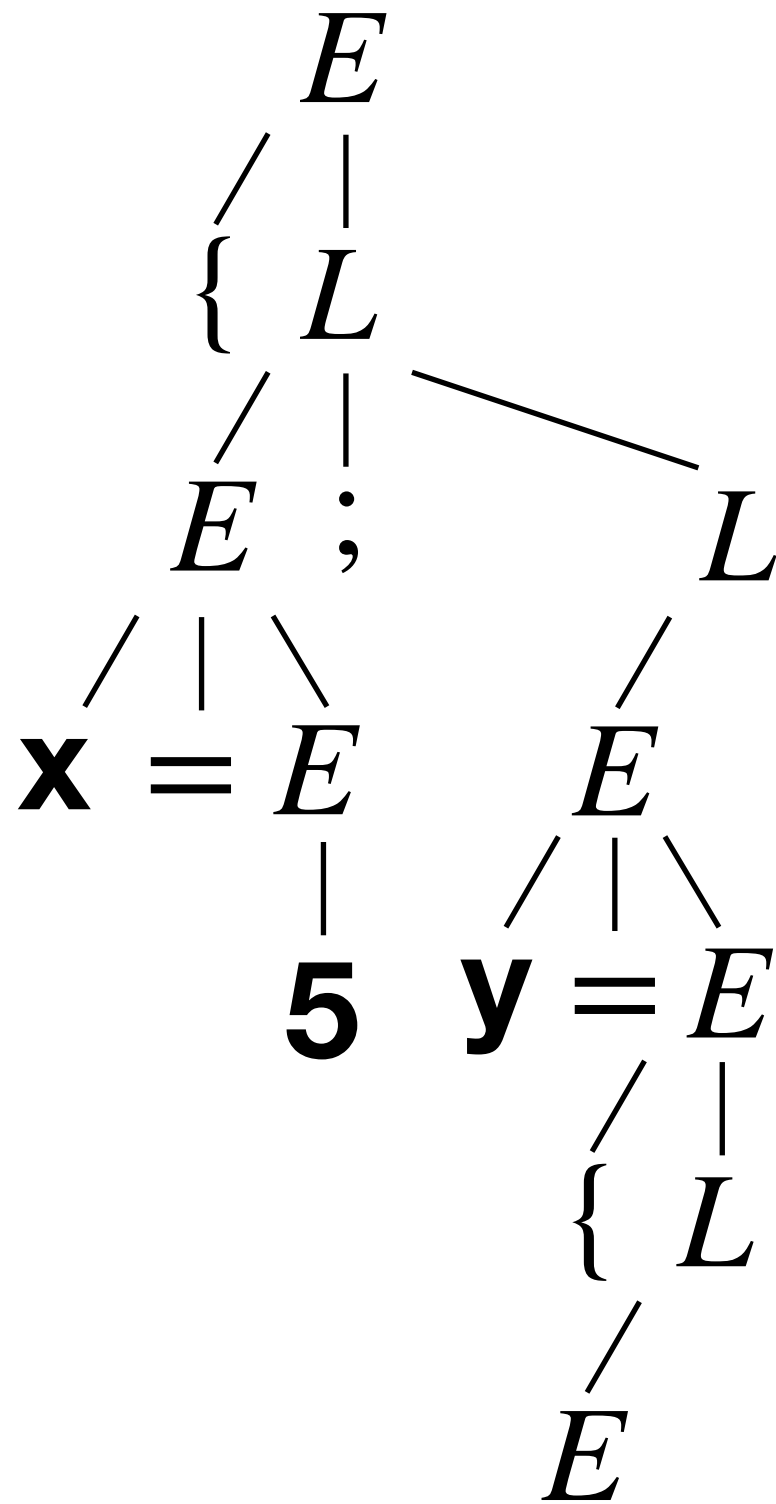


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

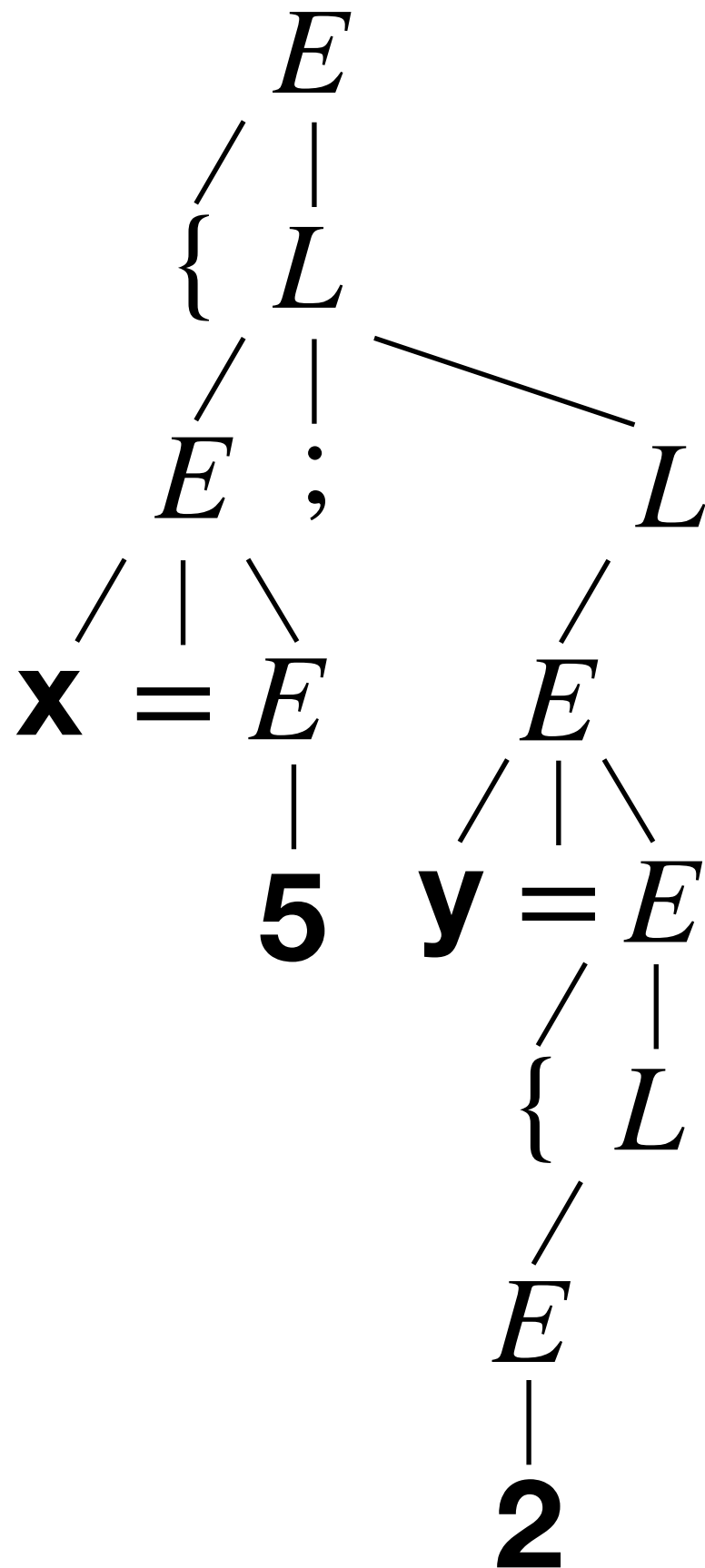


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y); \}$

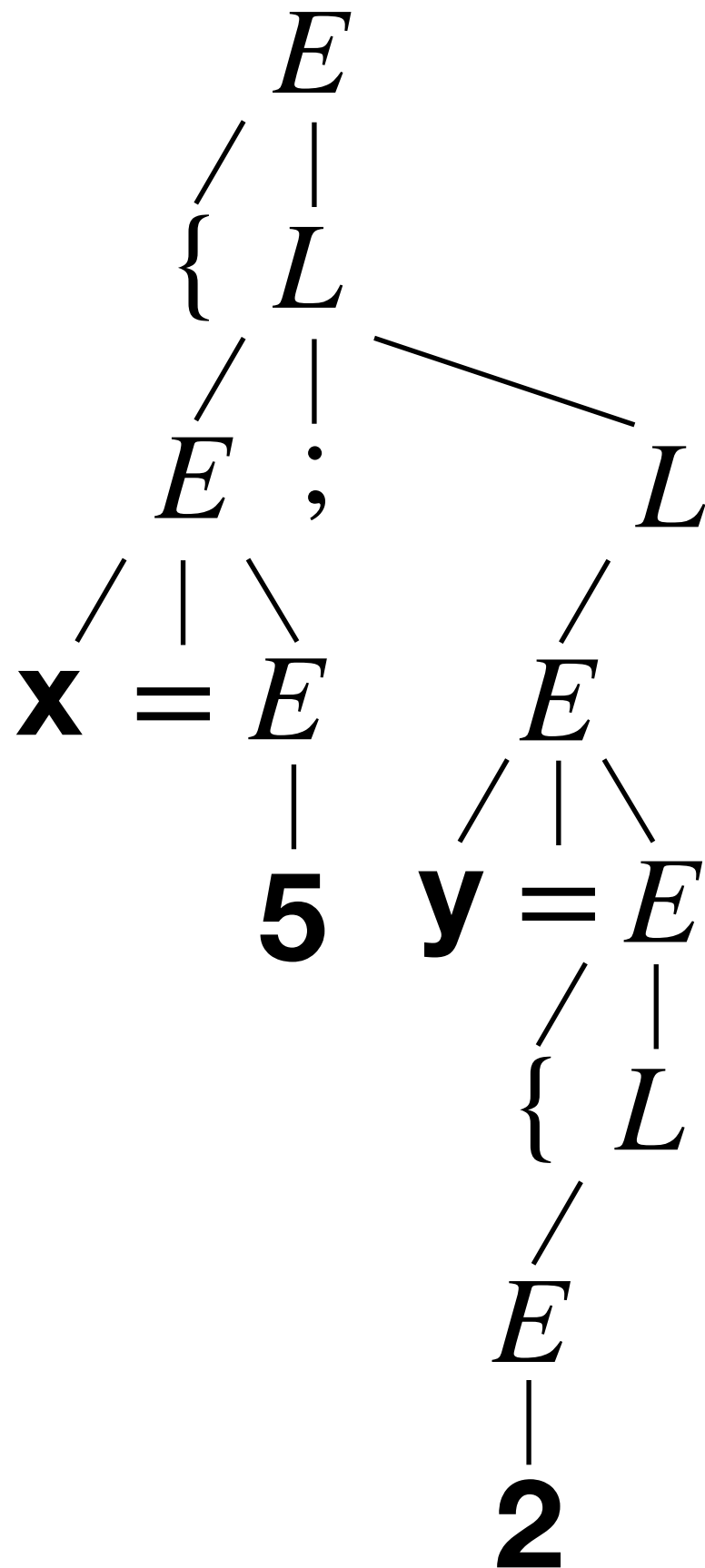


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

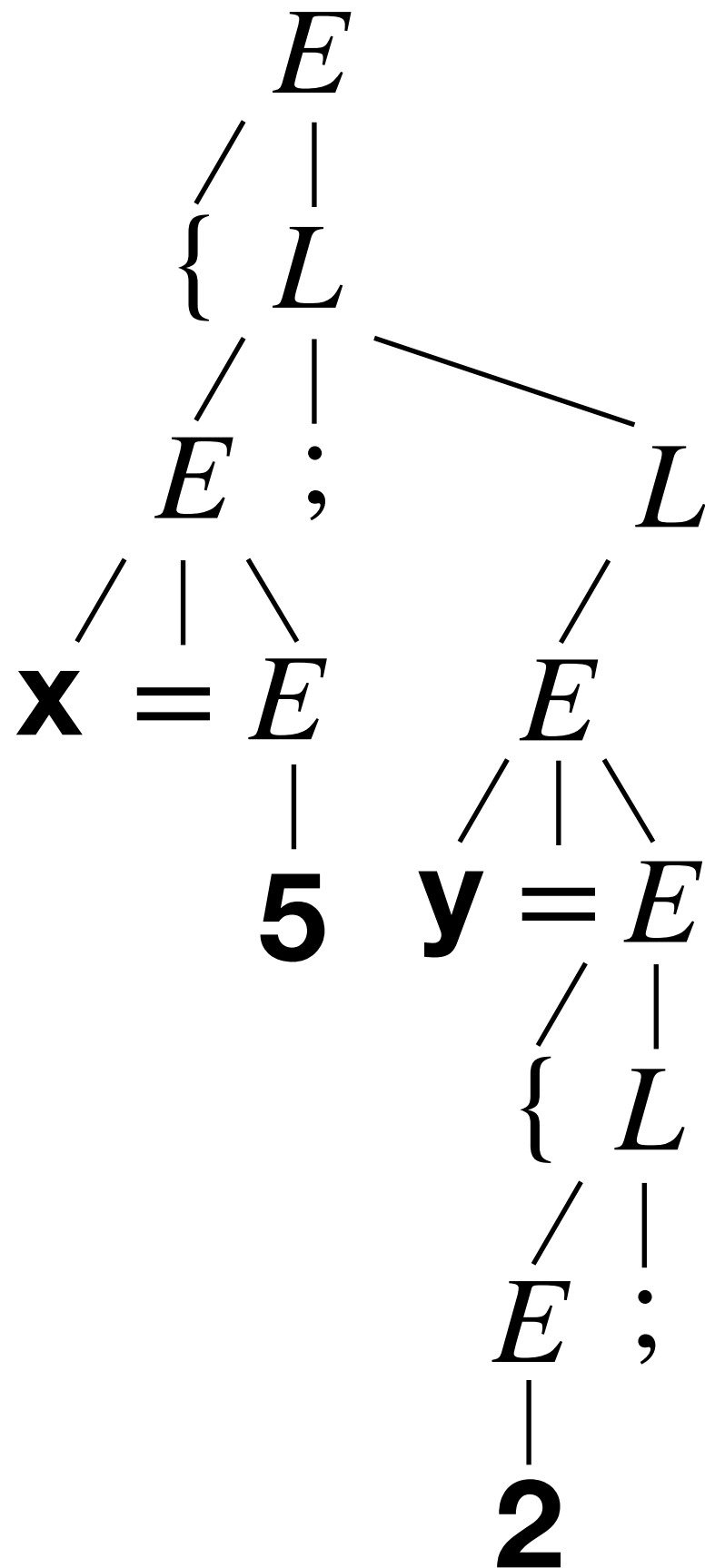


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

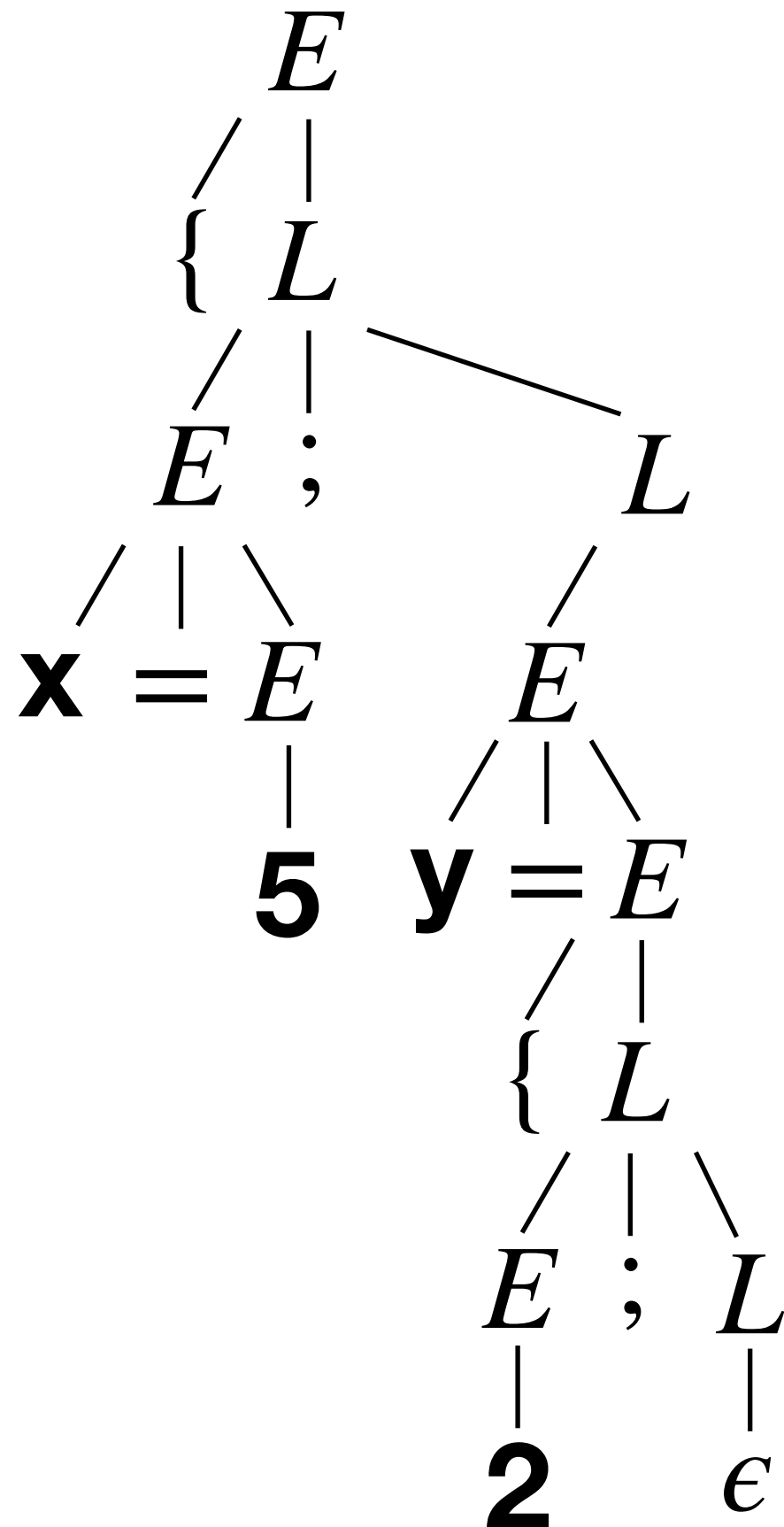
(where x is an ID, and n is a NAT)



(where x is an ID, and n is a NAT)

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

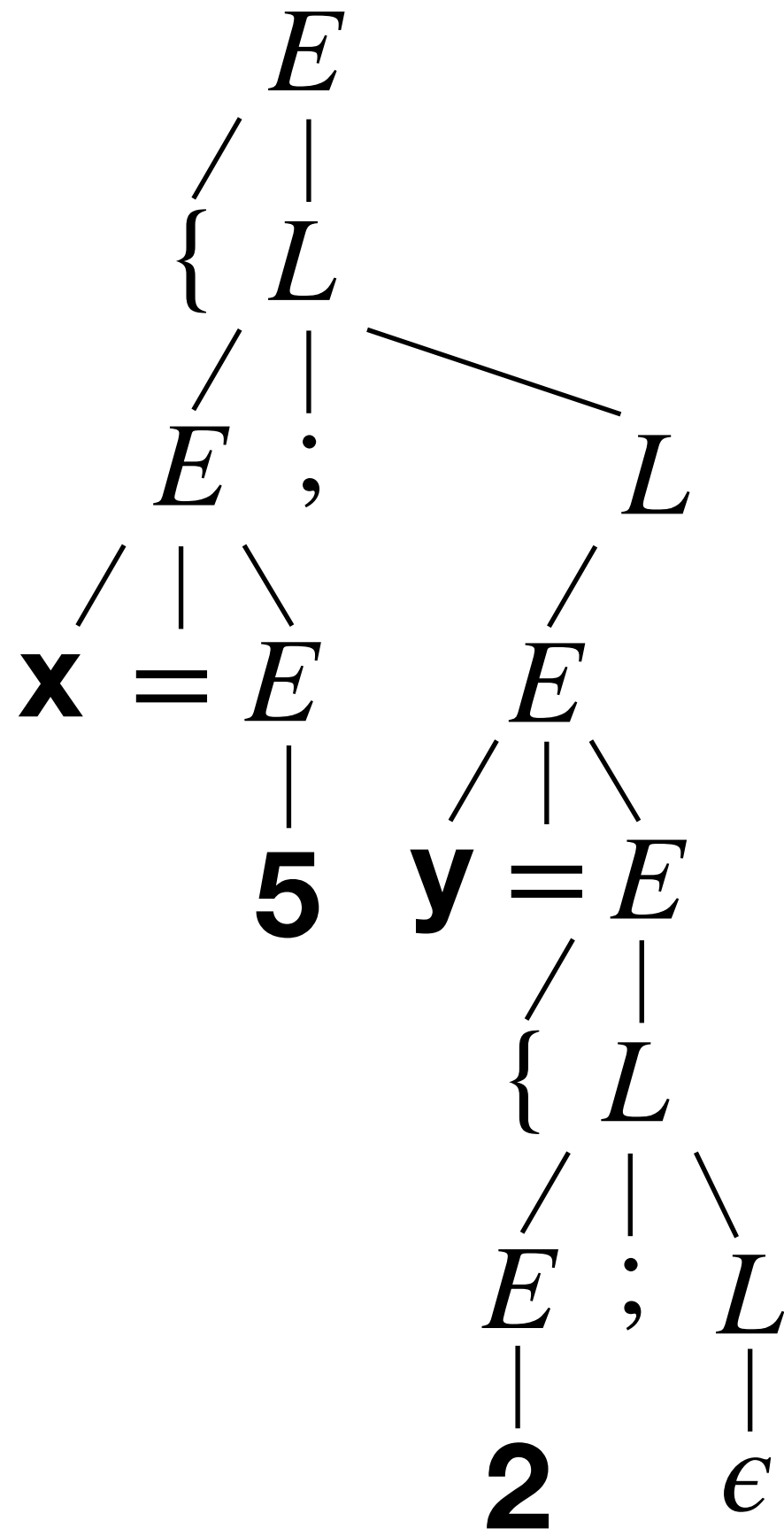


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

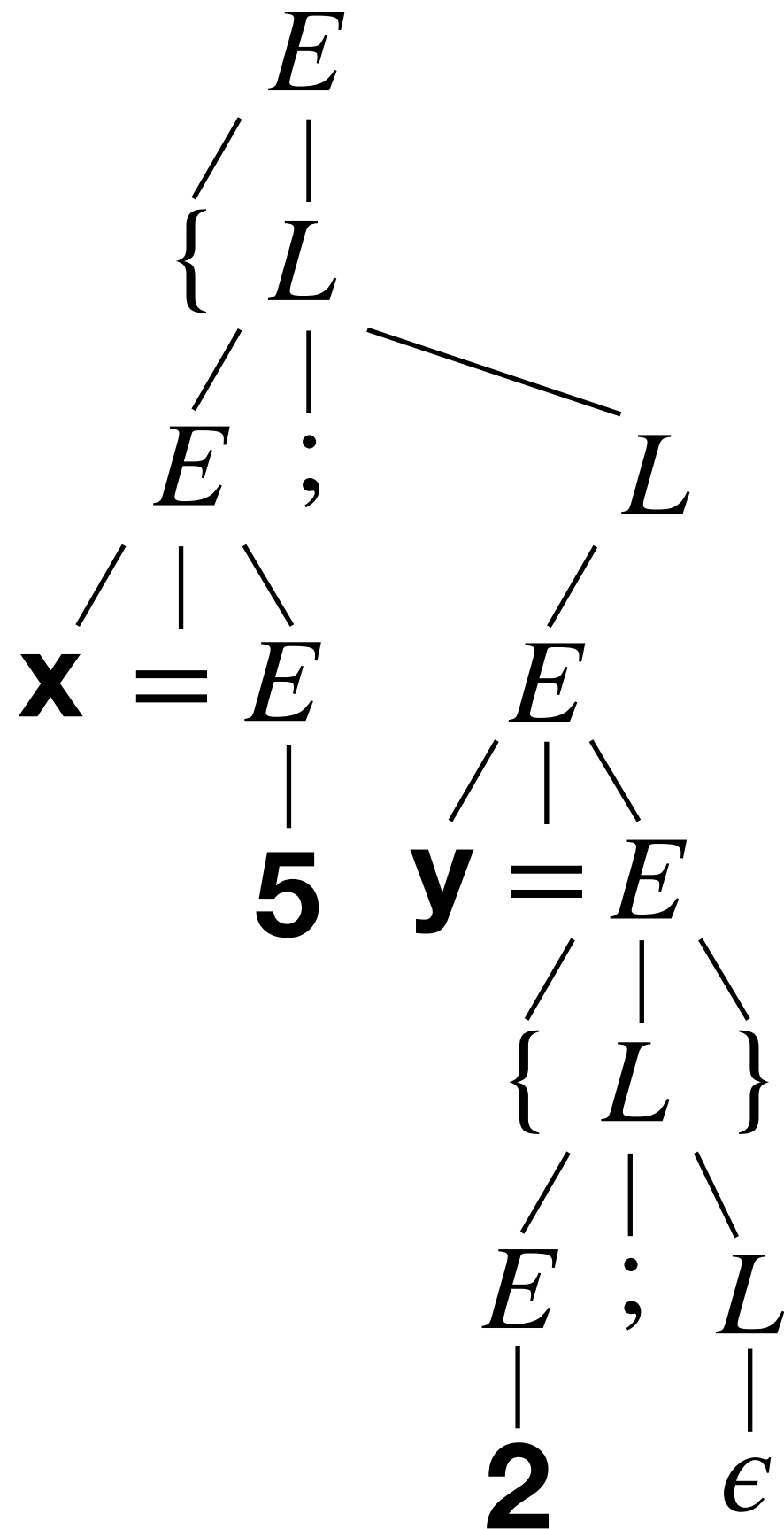


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

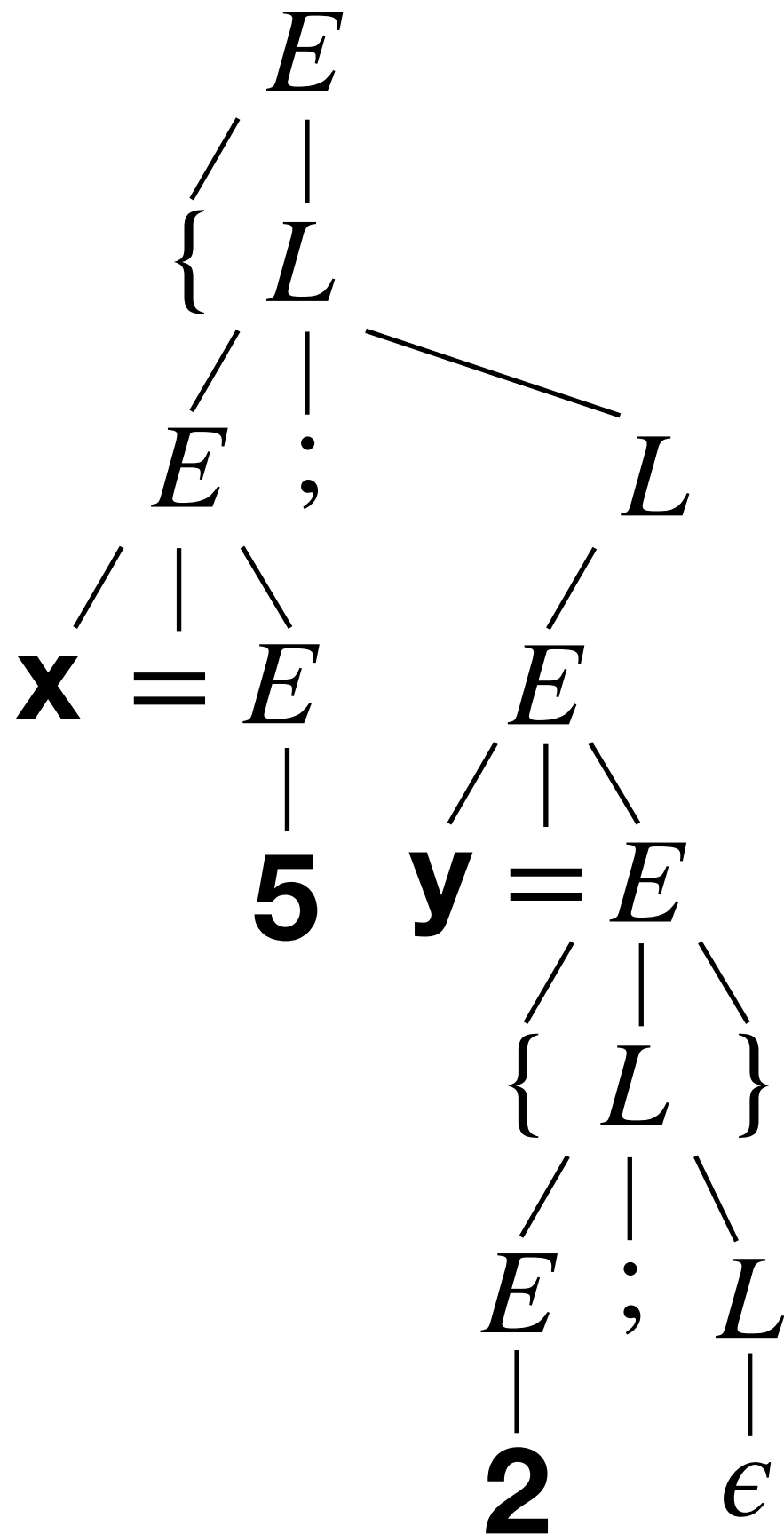


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$

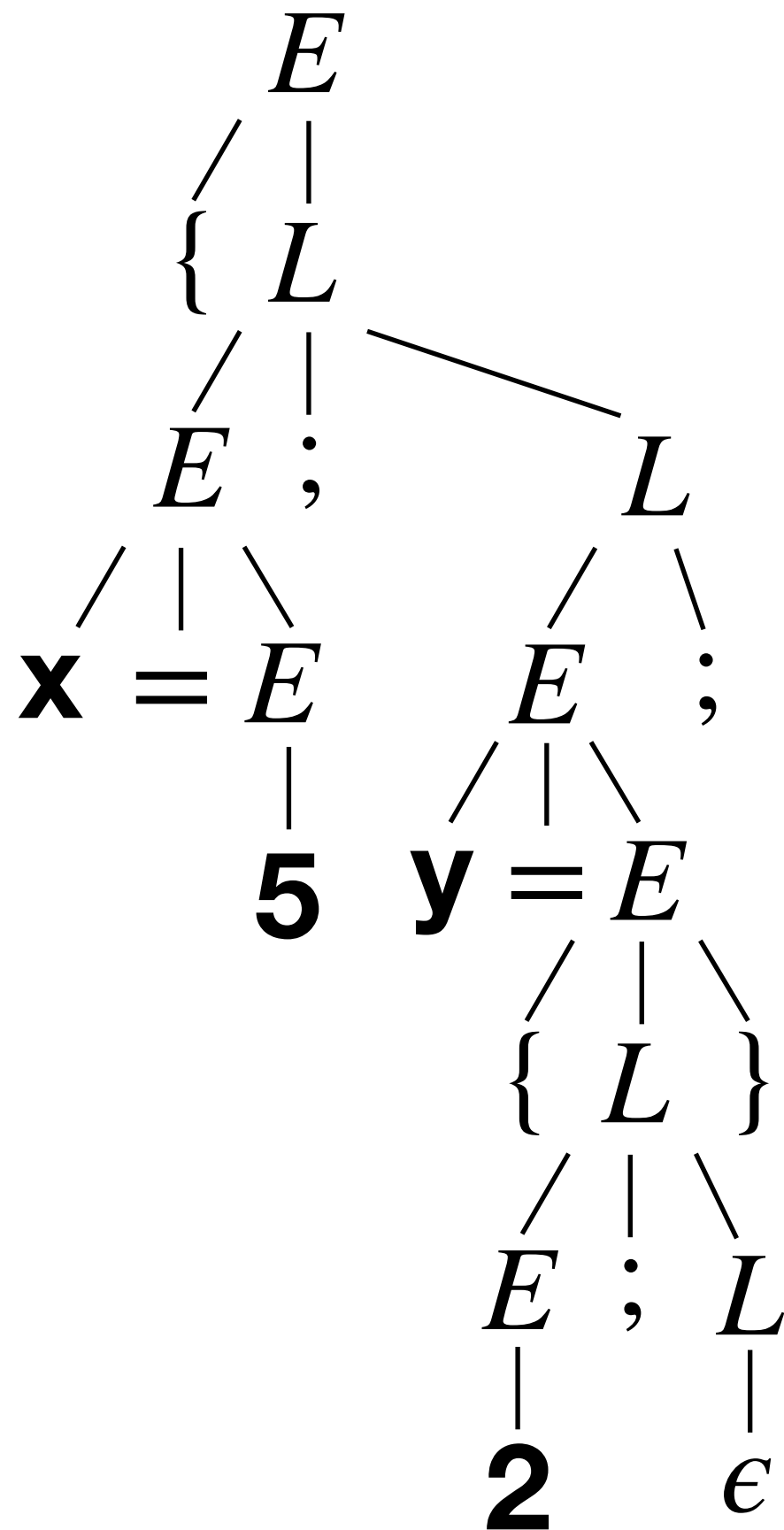


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

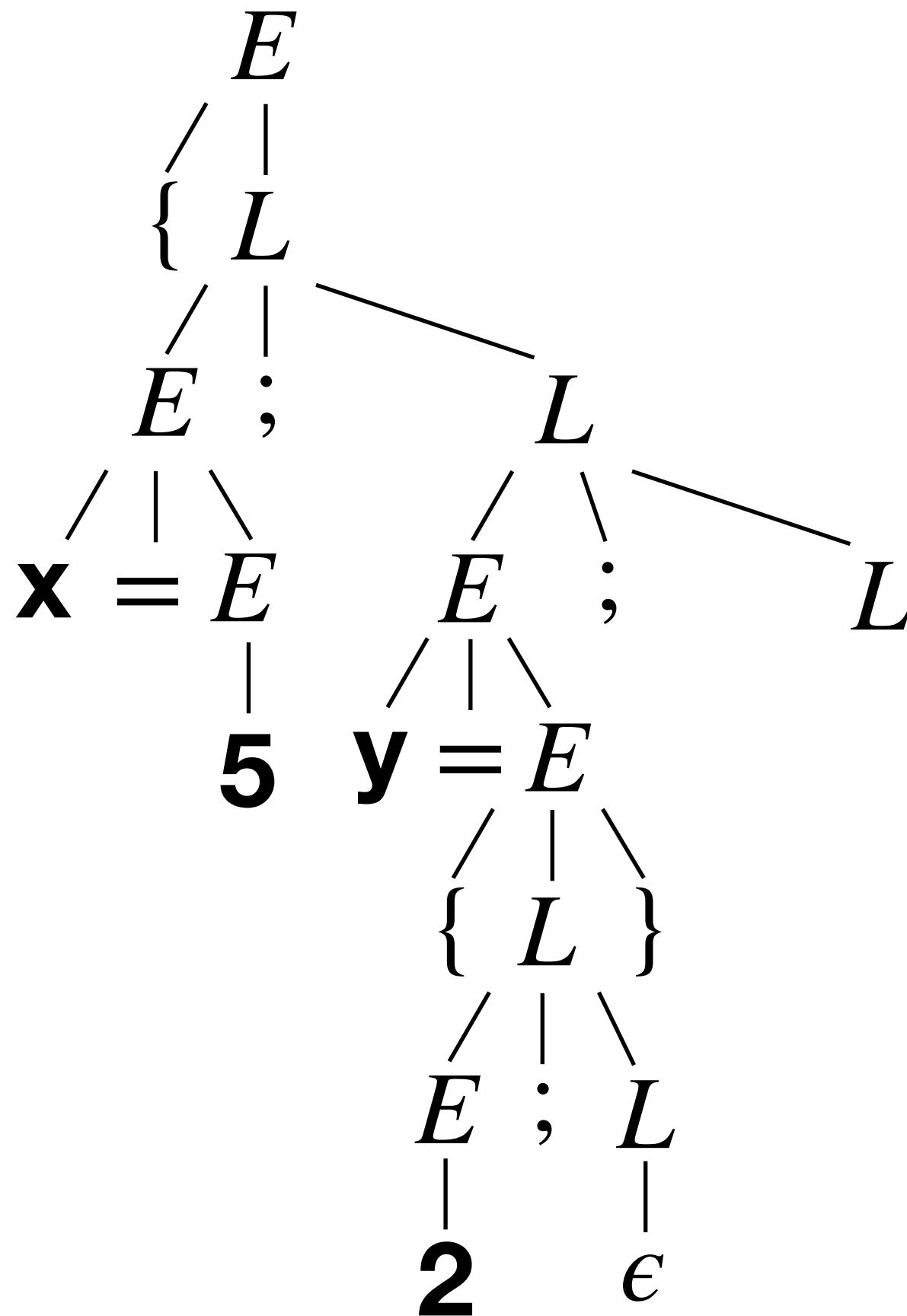


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

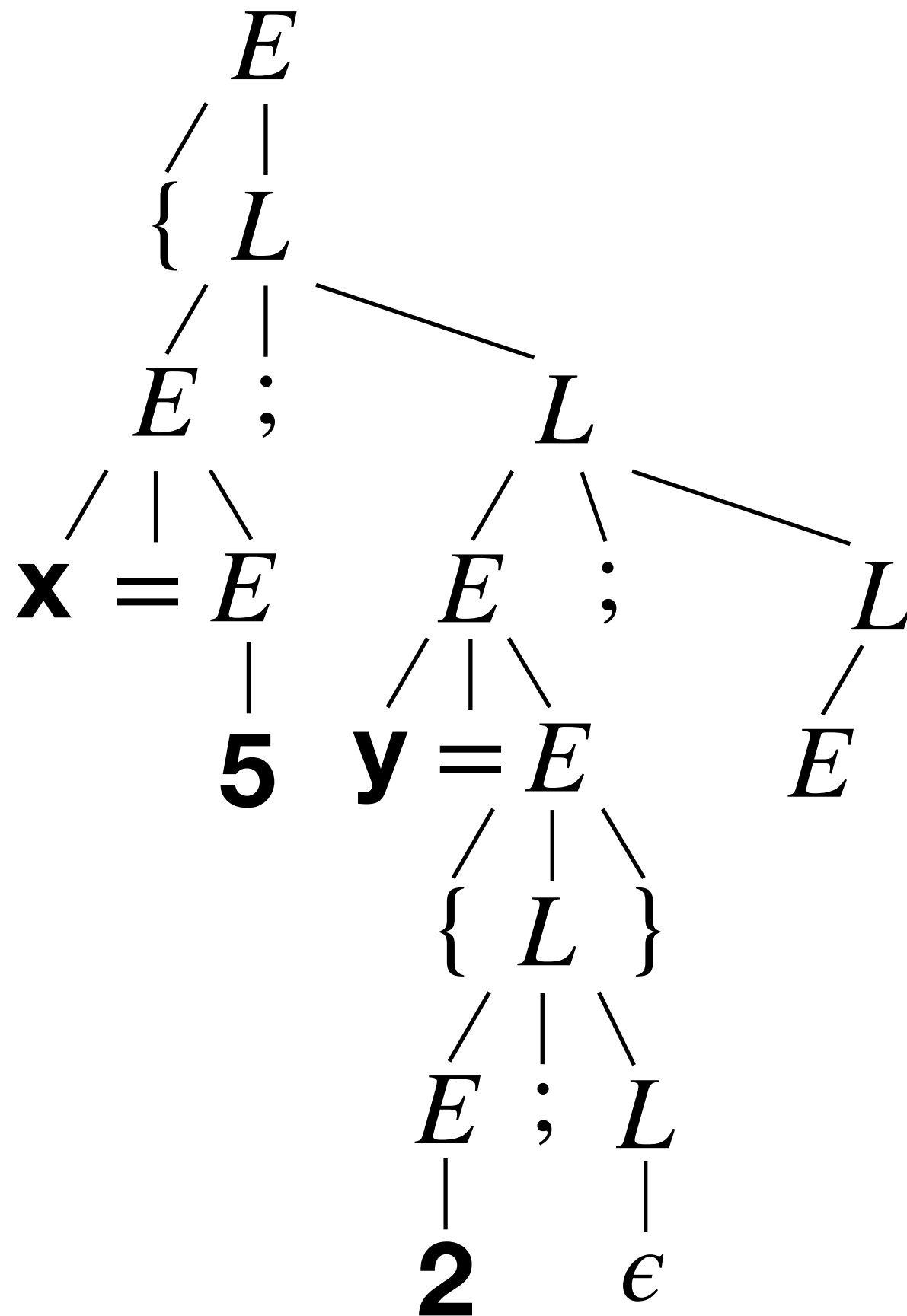


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

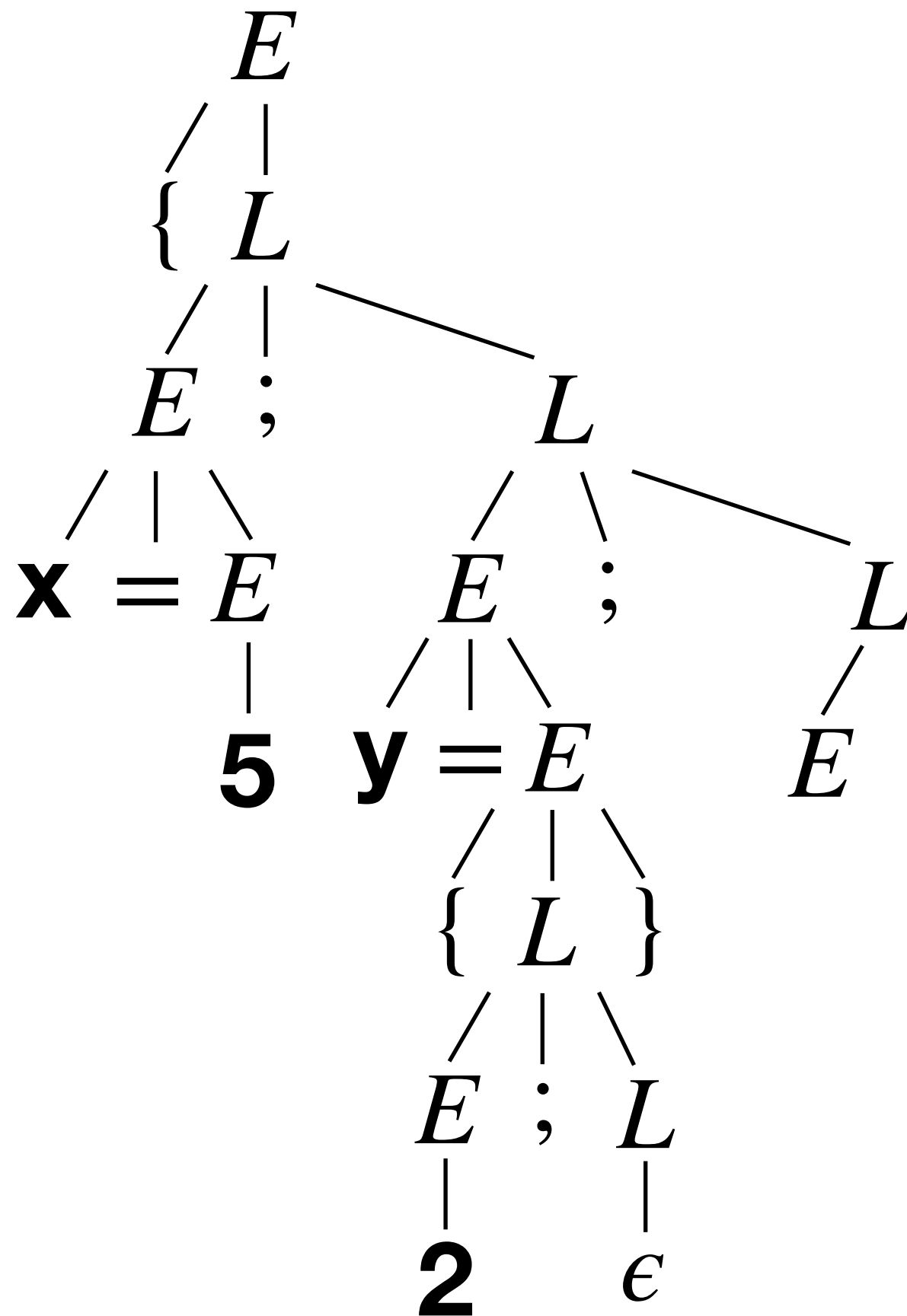


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

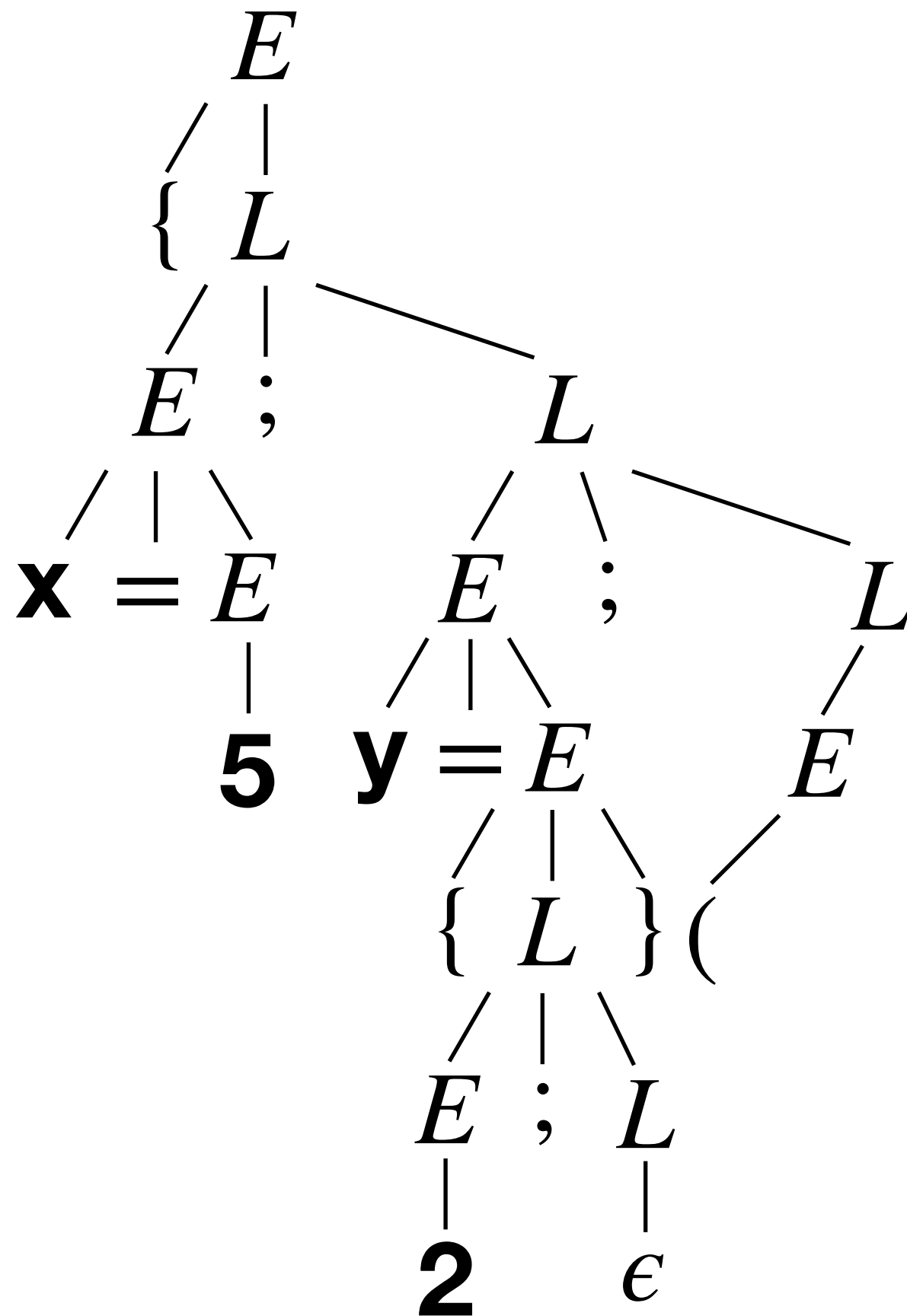


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

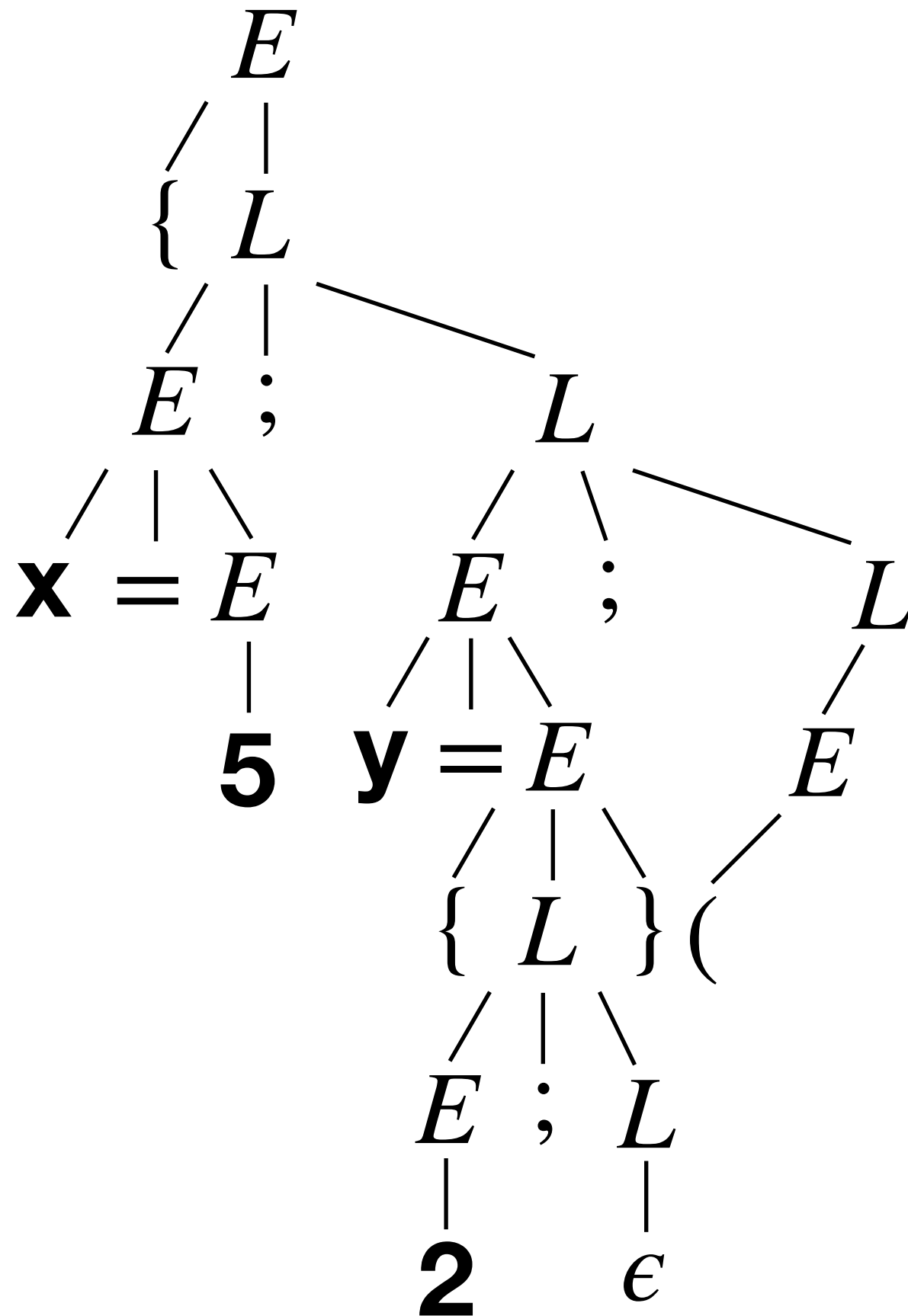


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

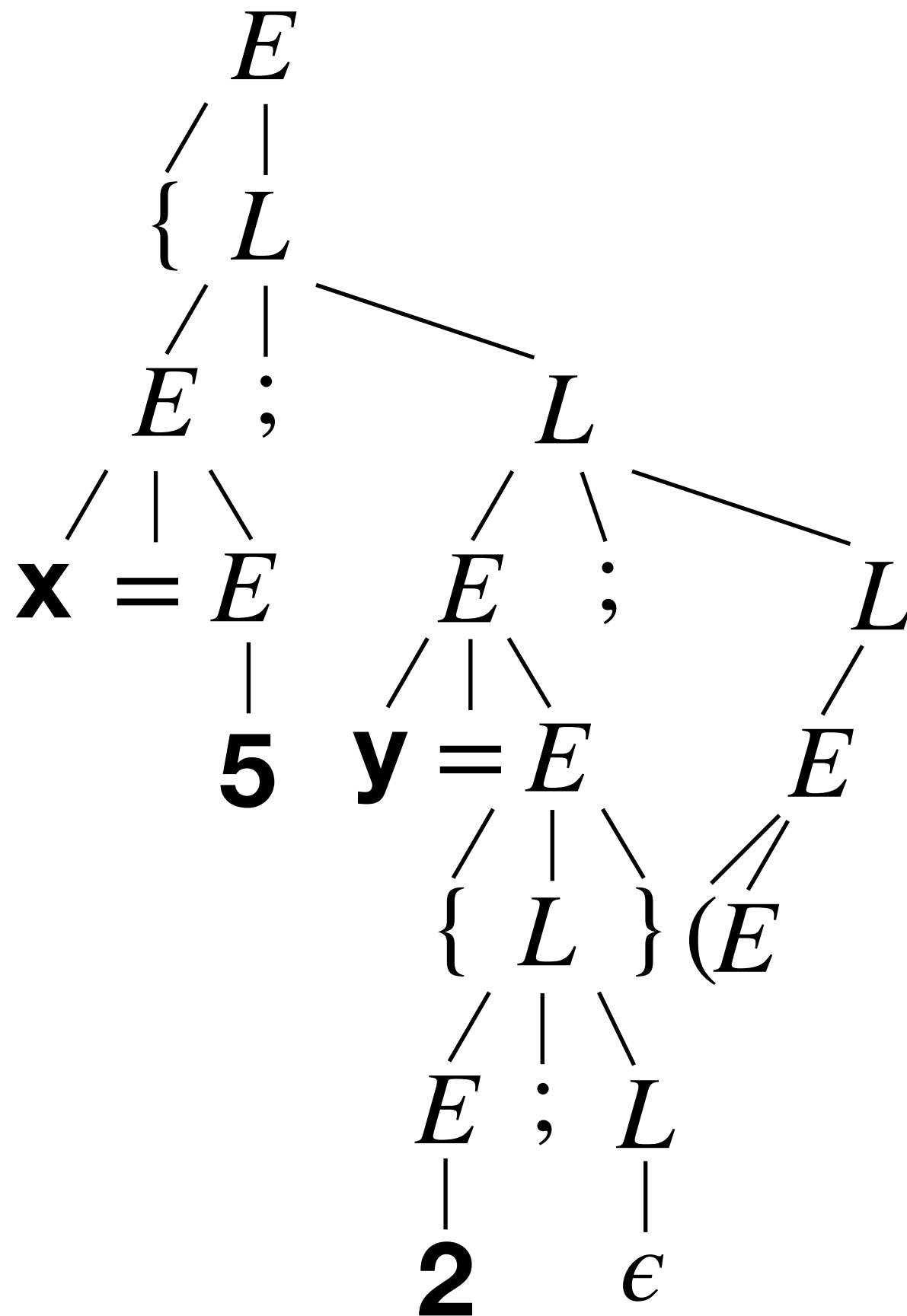


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

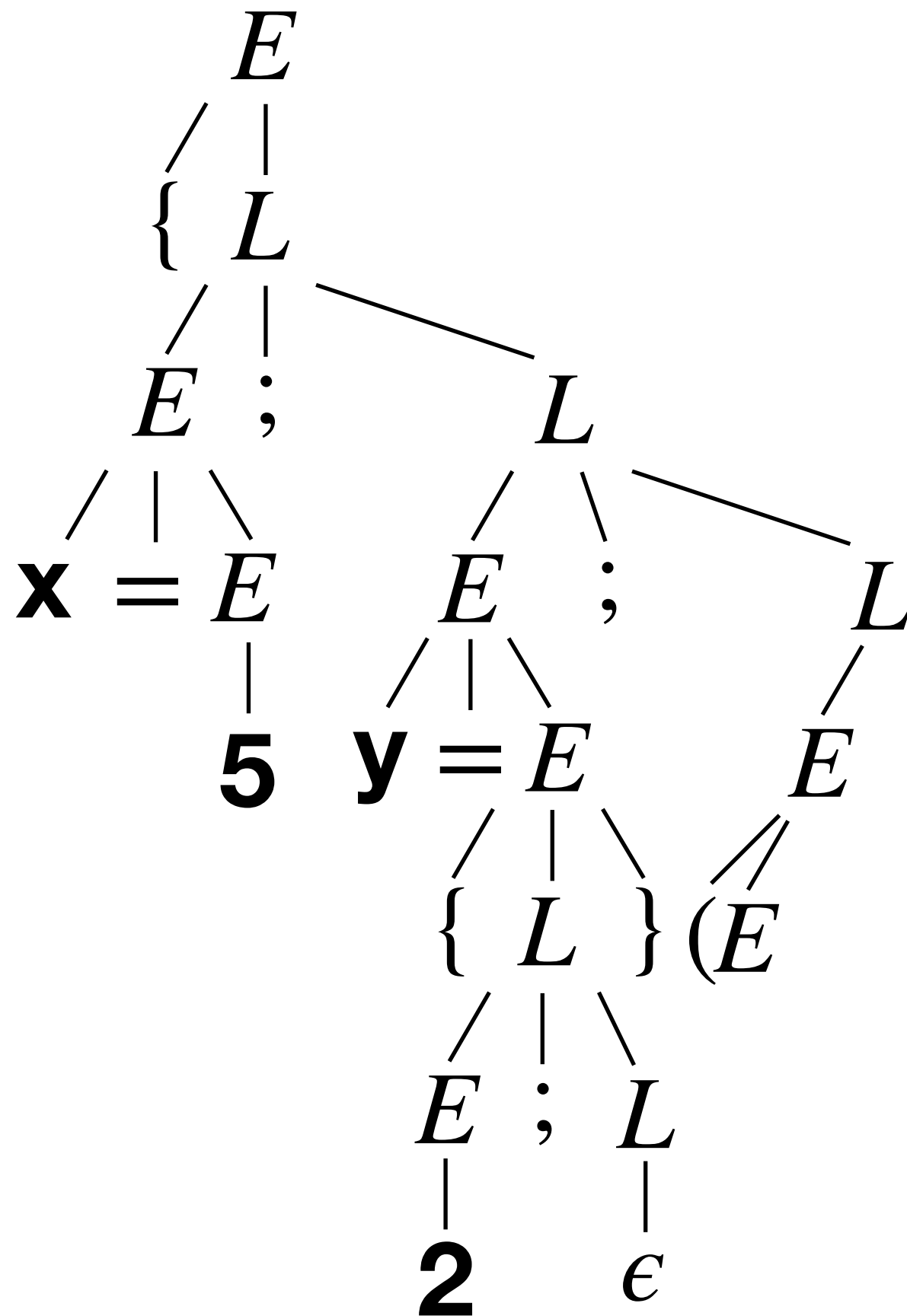


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

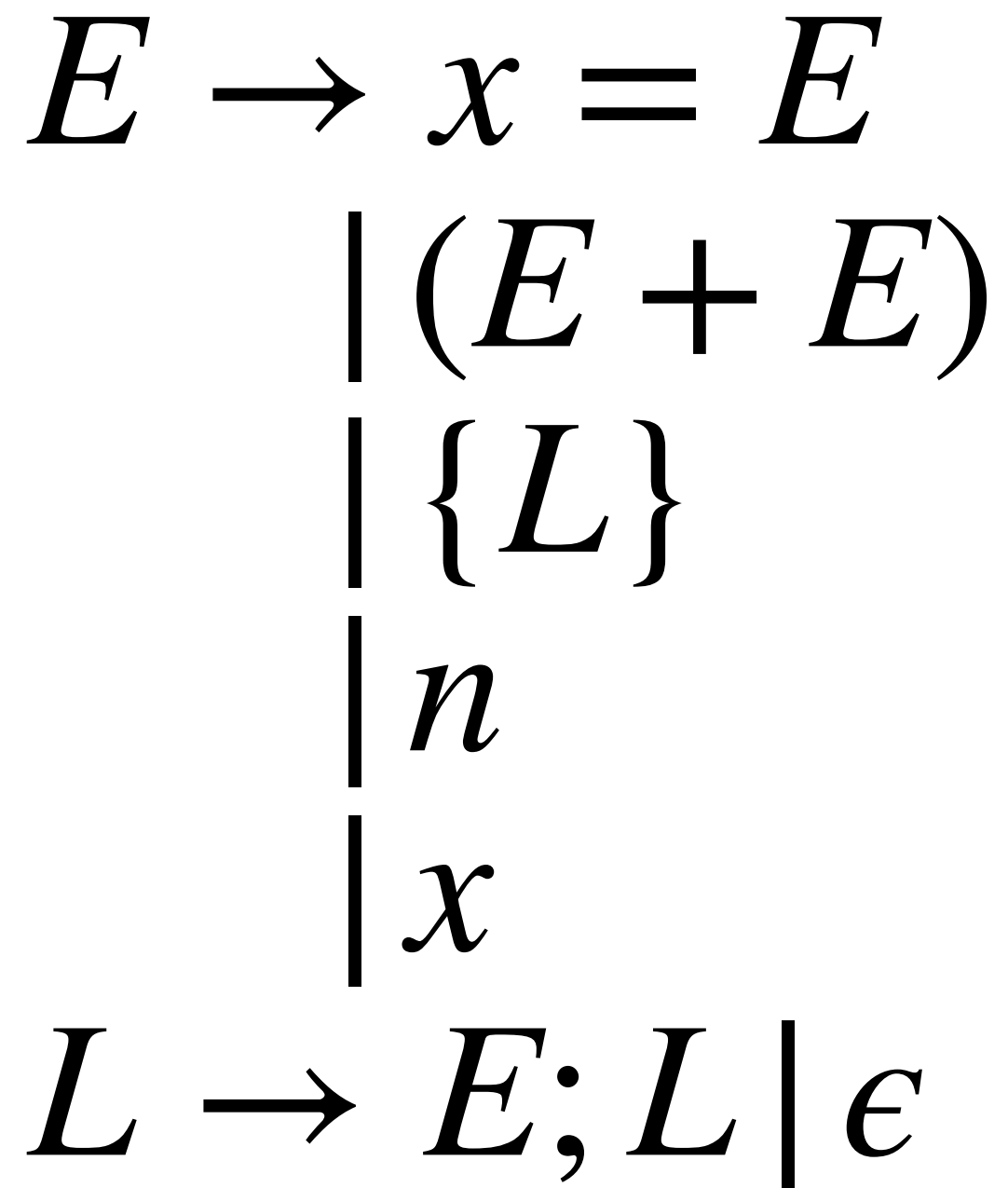
Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

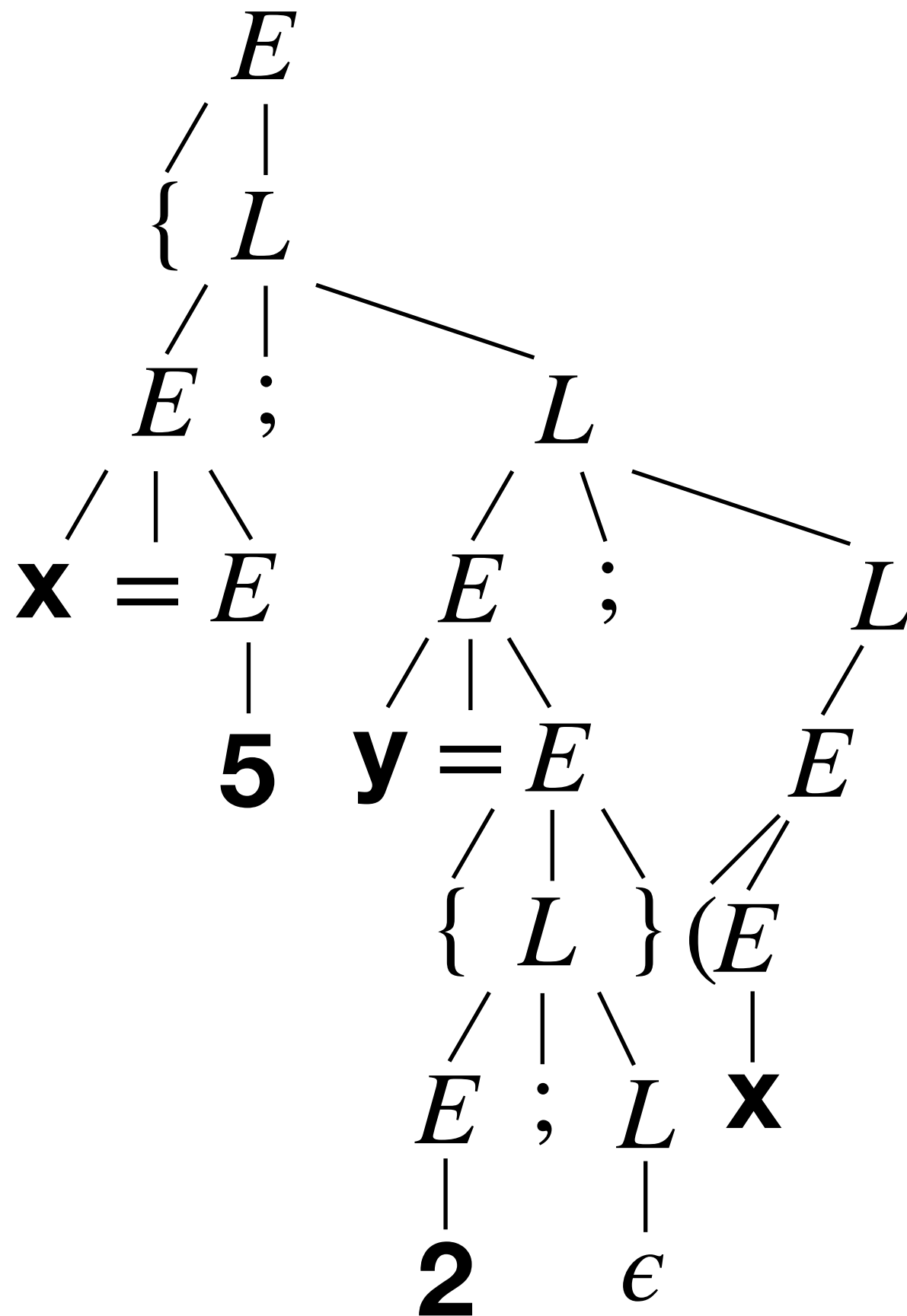
$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)



78

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

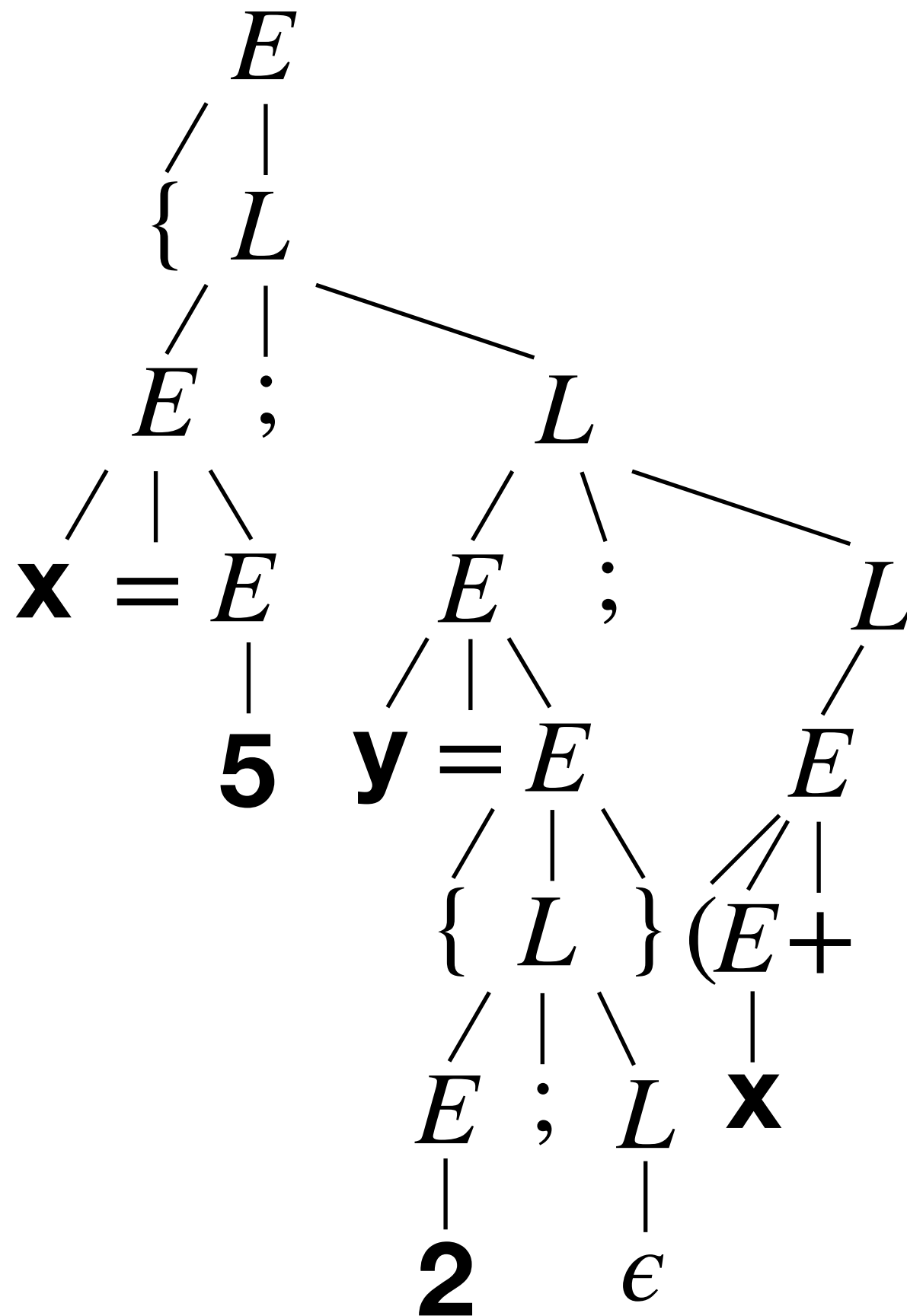


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

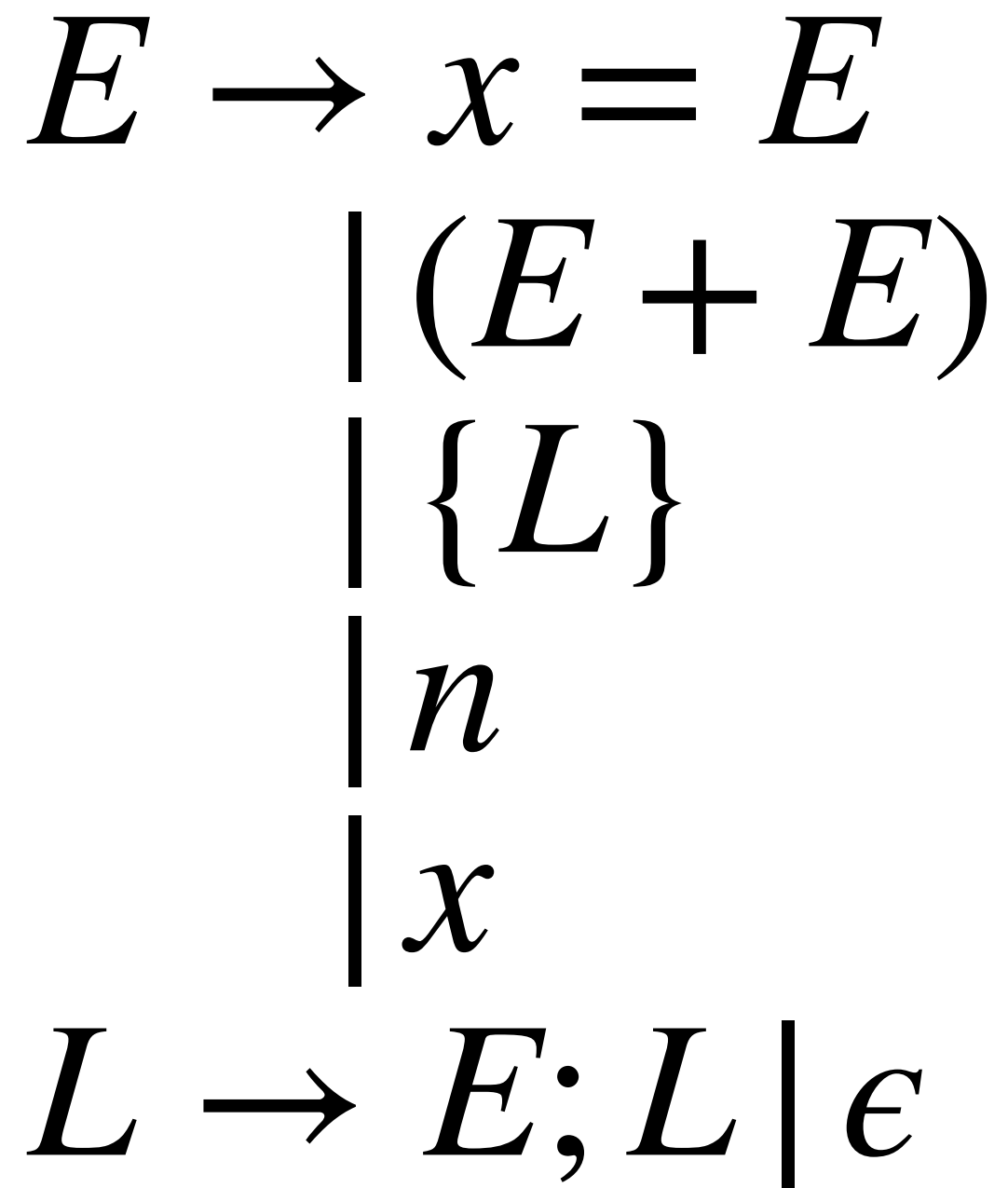
Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

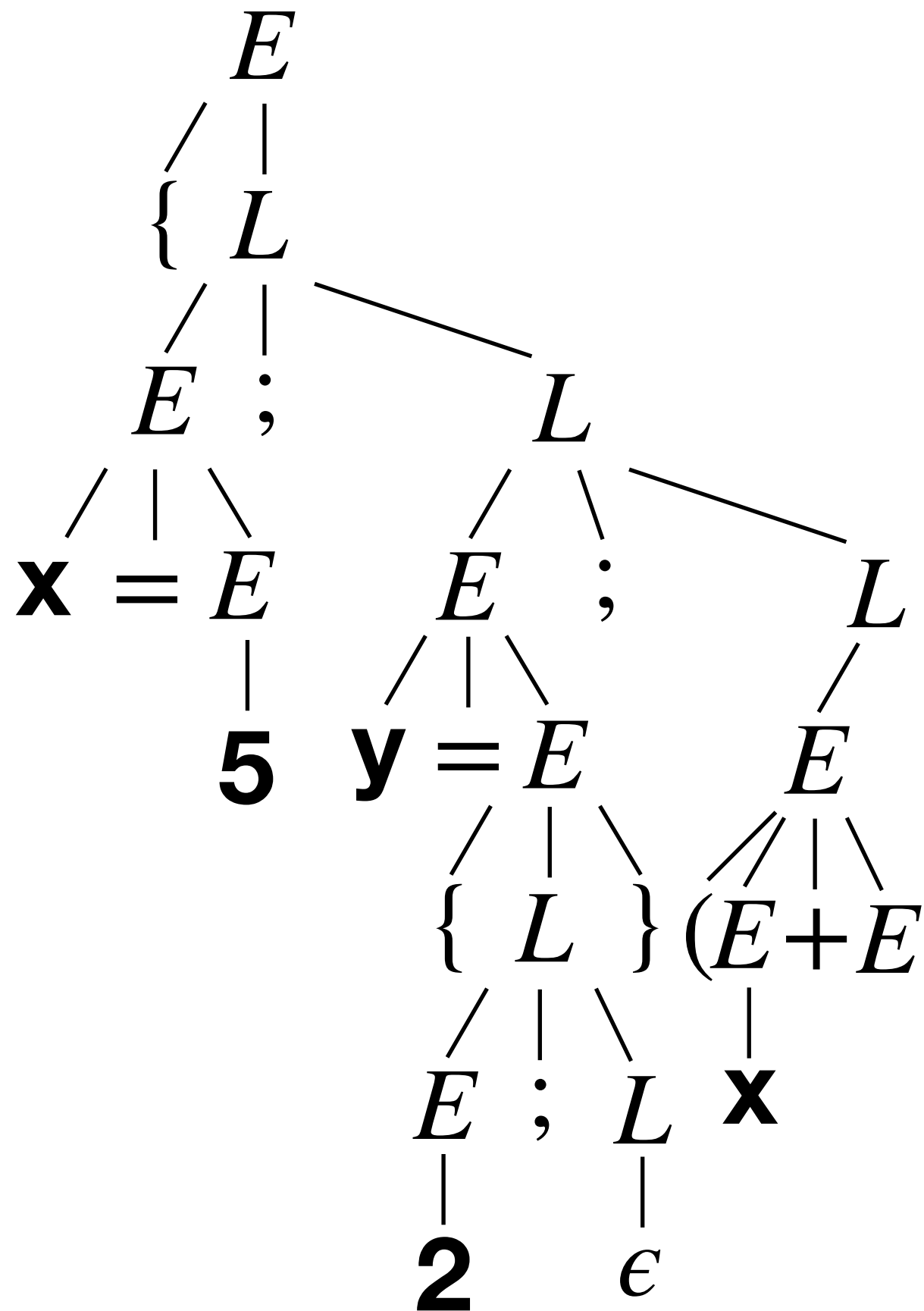
$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)



80

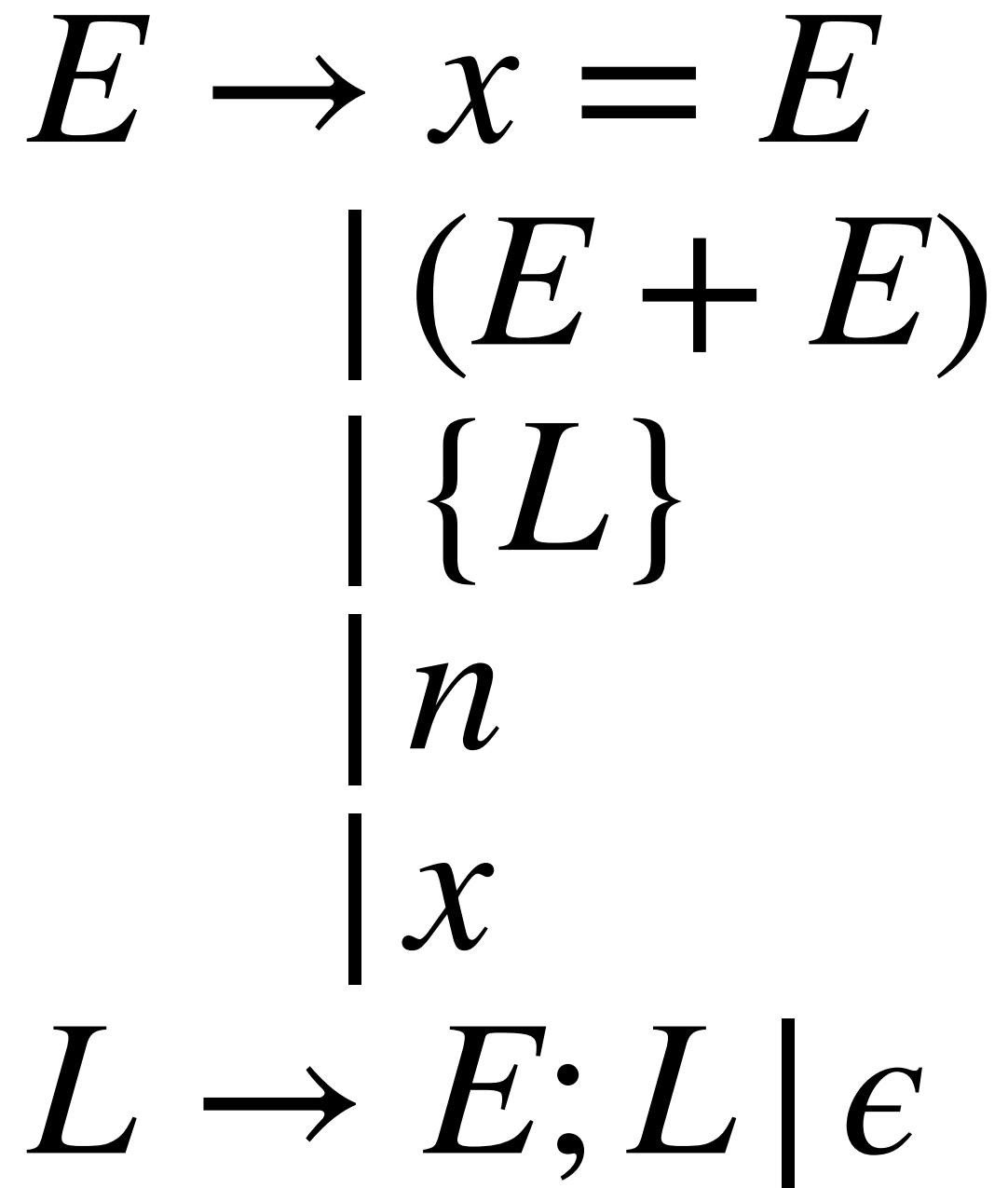
Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{ L \} \\
 \quad | n \\
 \quad | x
 \end{array}$$

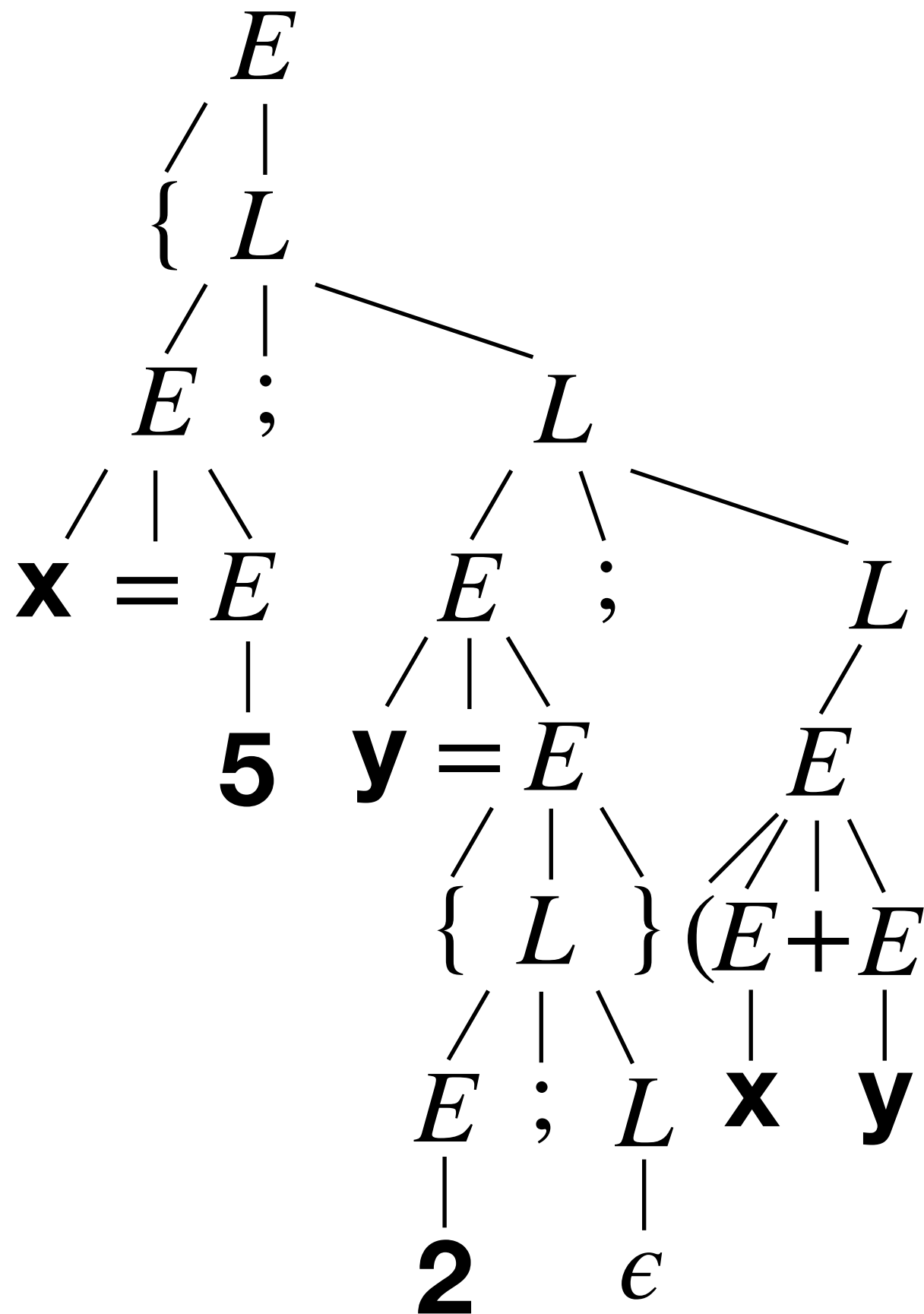
$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)



81

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

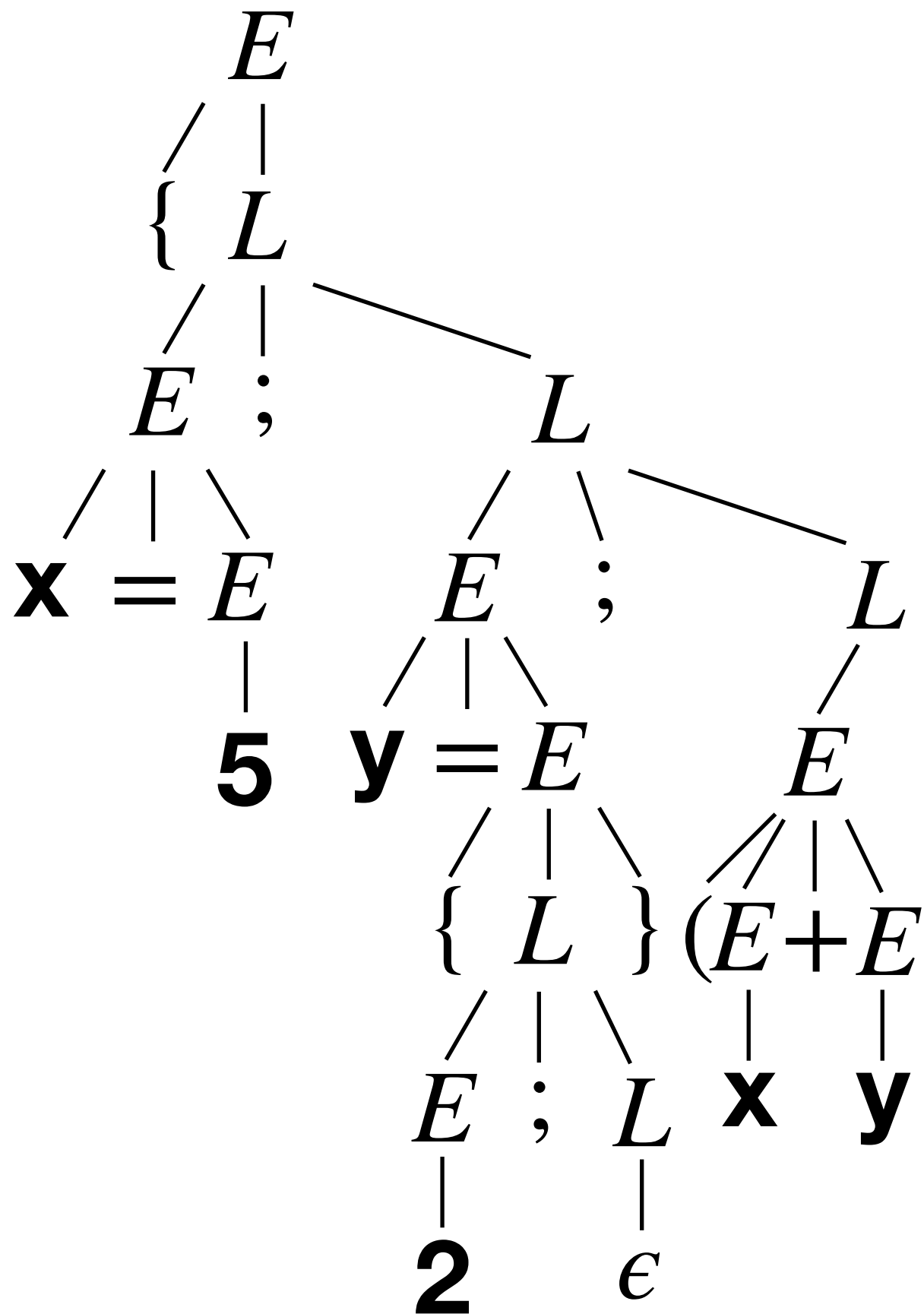


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

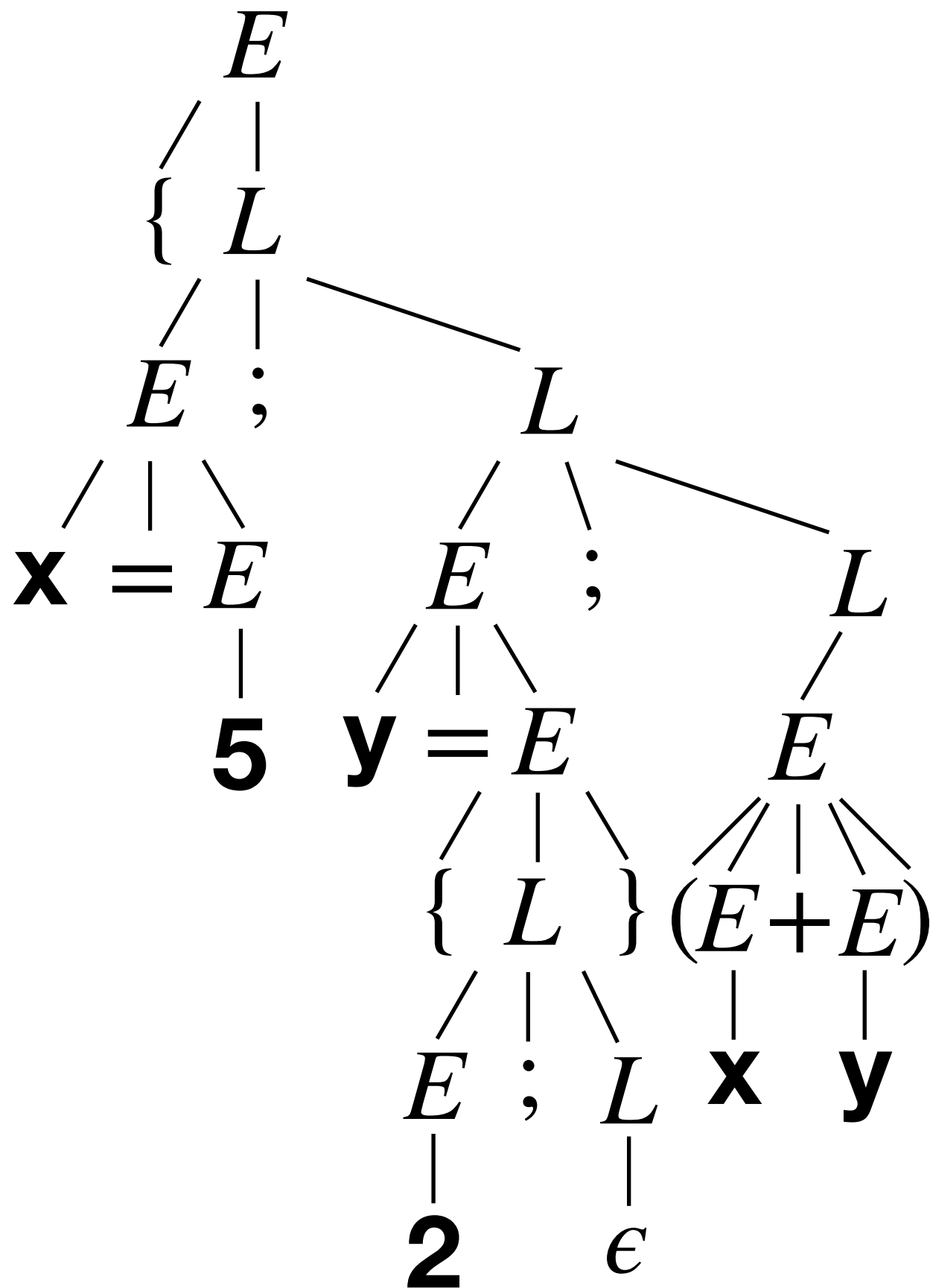


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

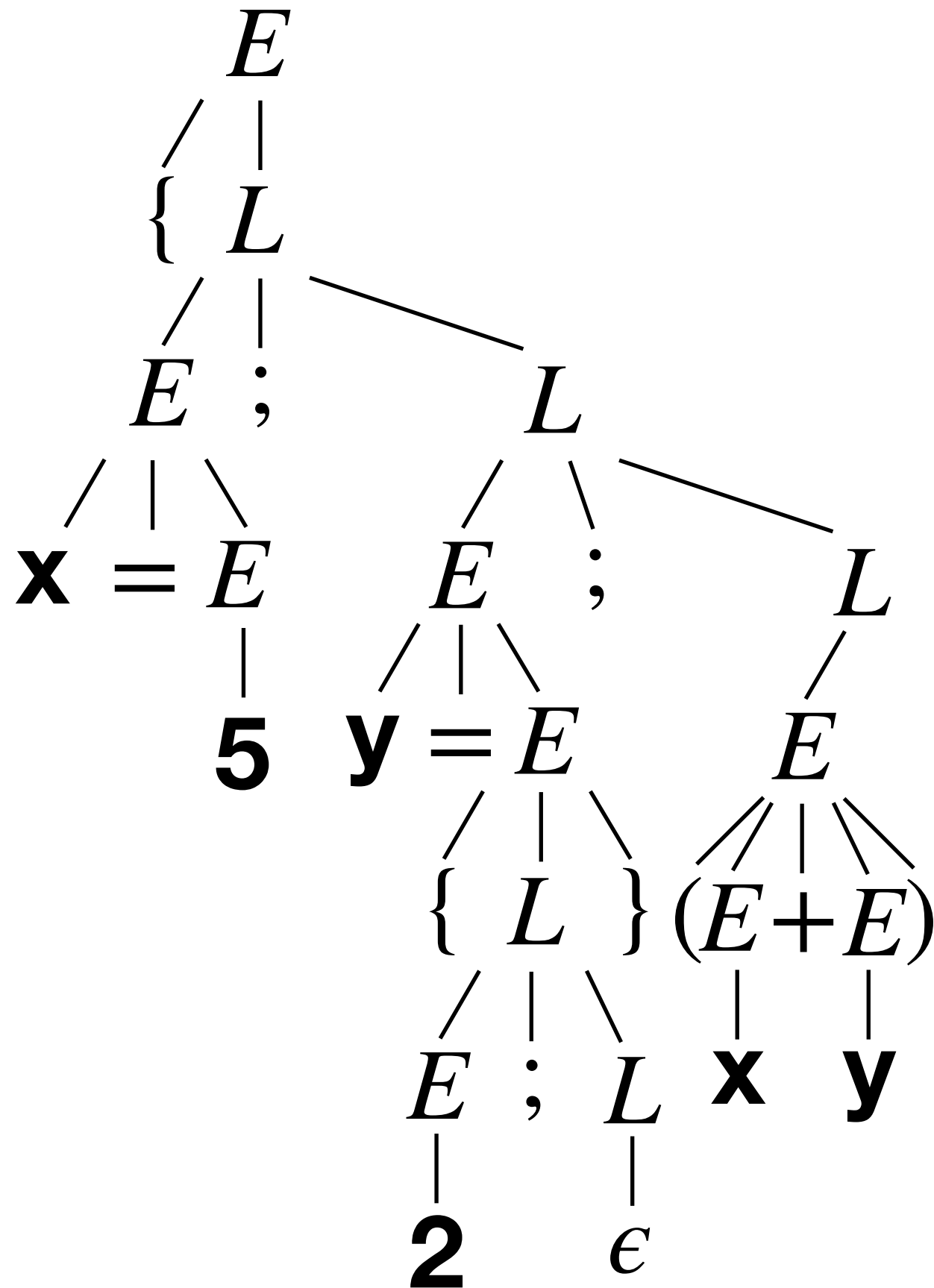


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

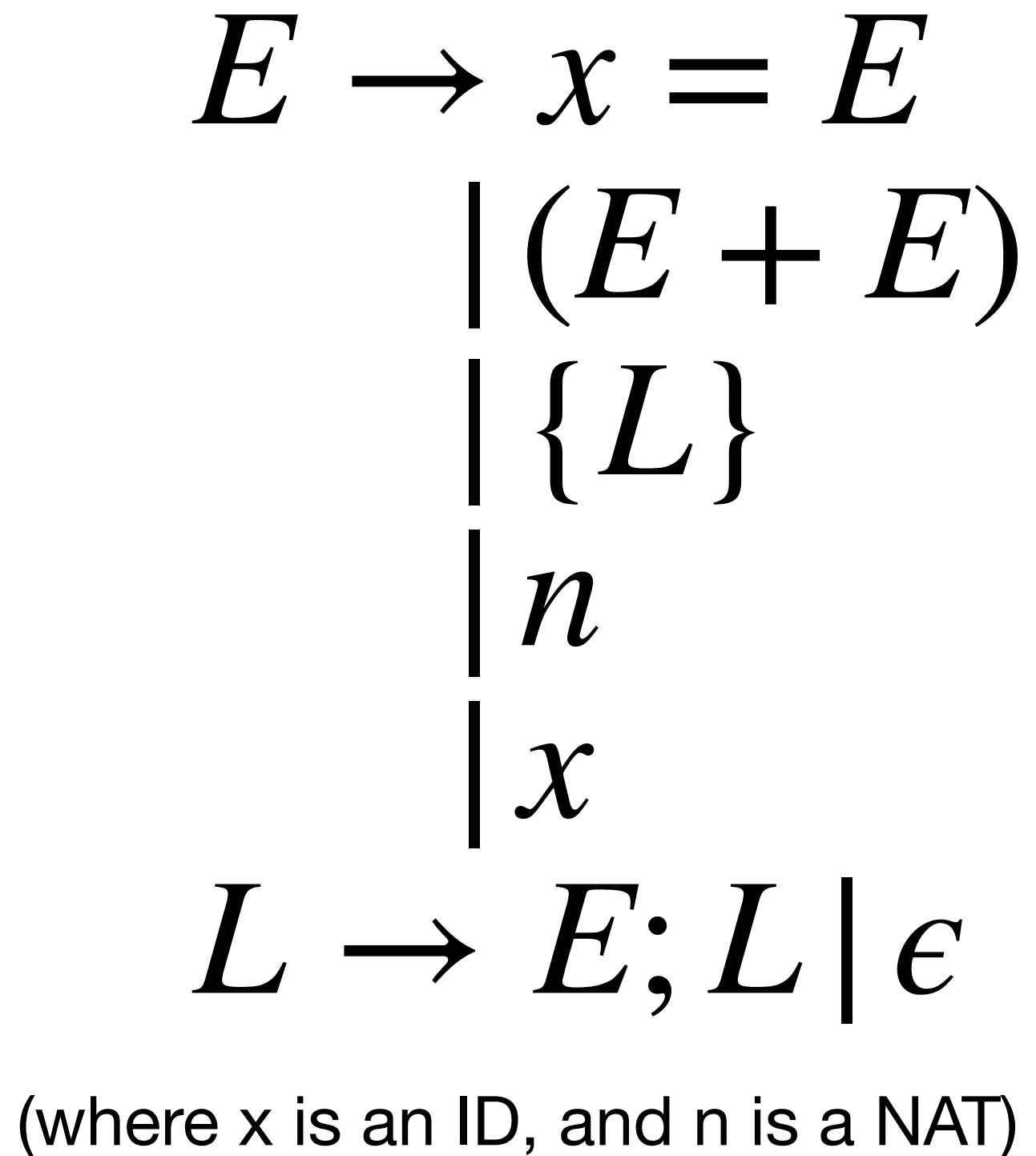
Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



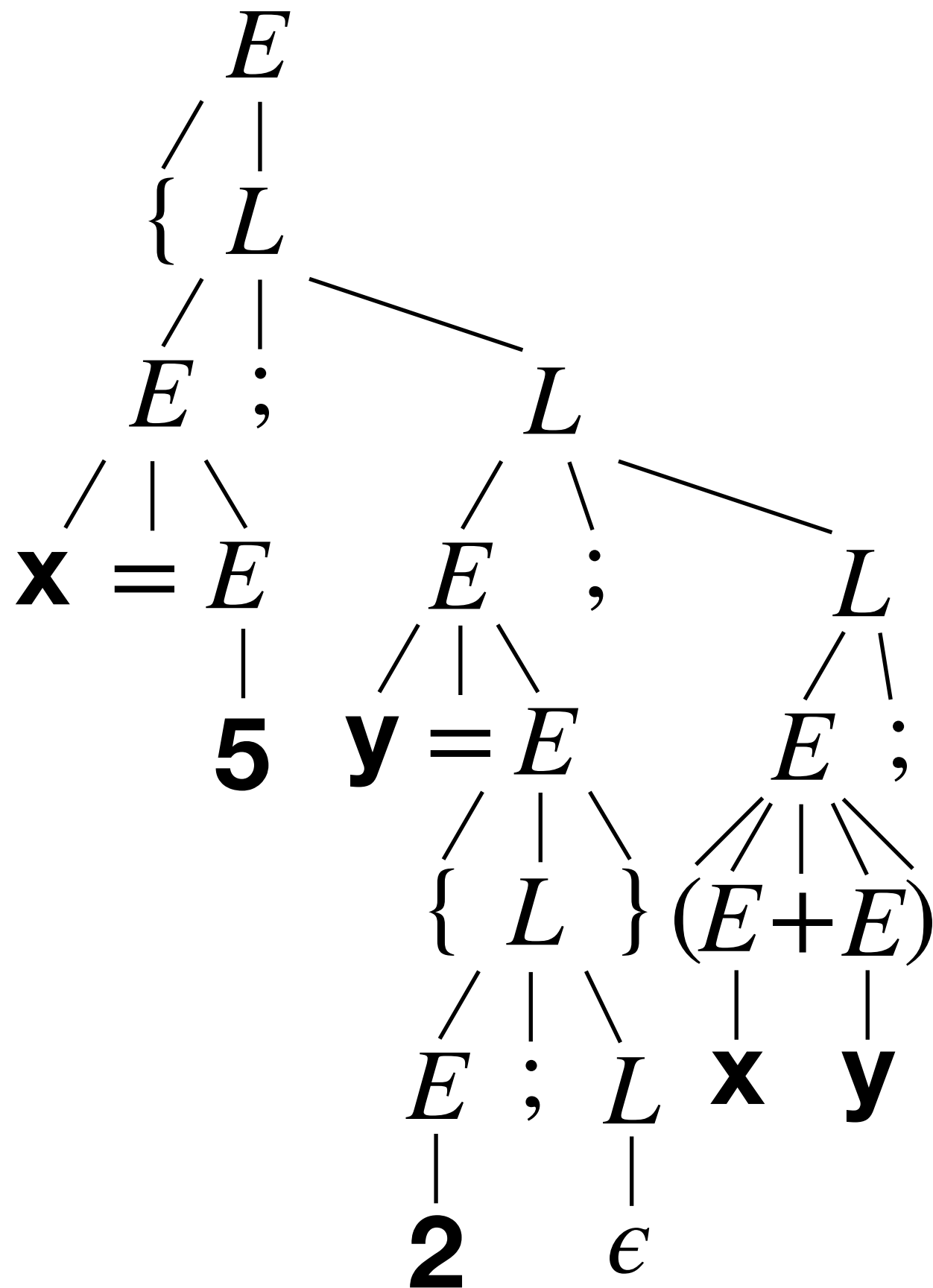
$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)



Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$

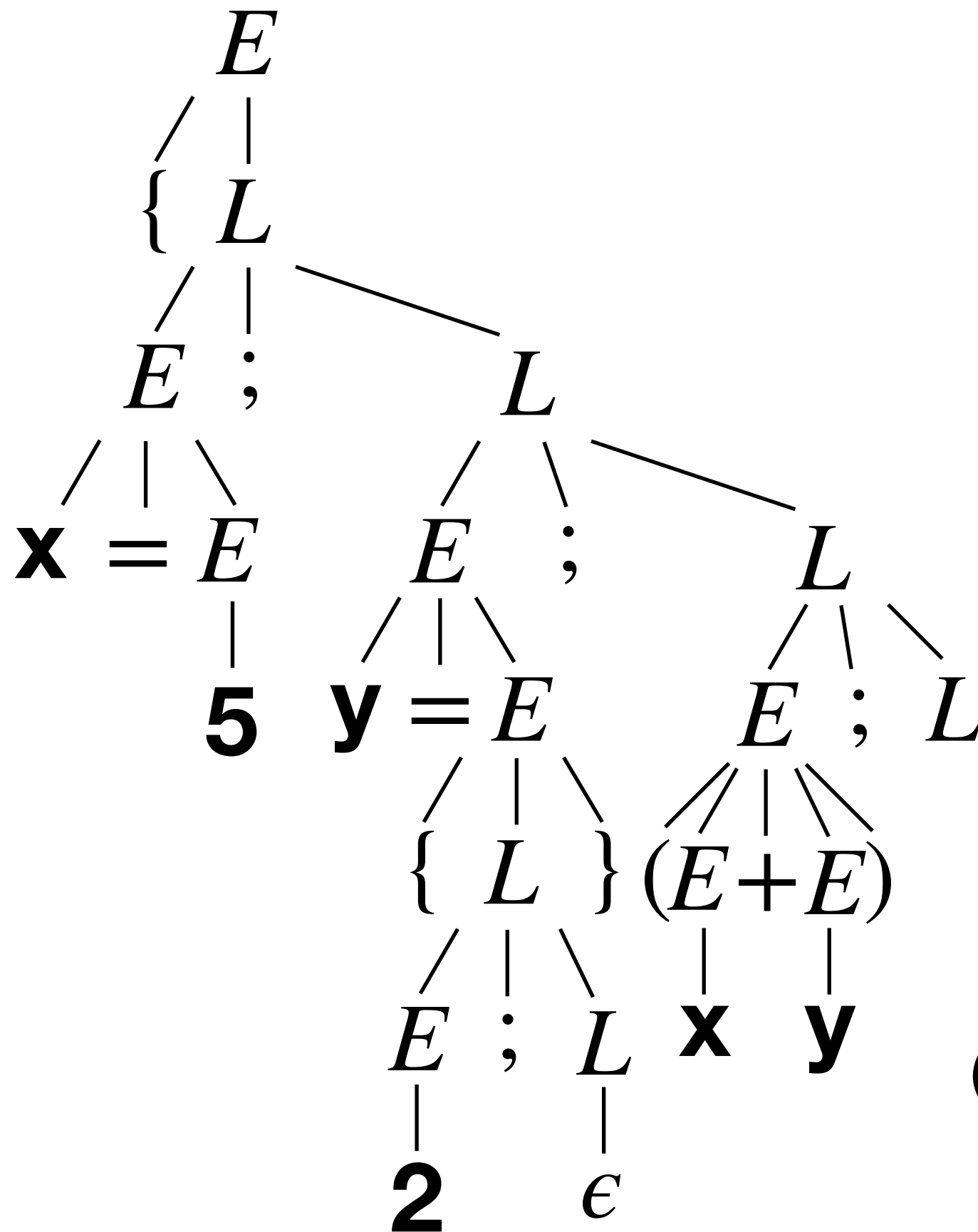


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{ L \} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$

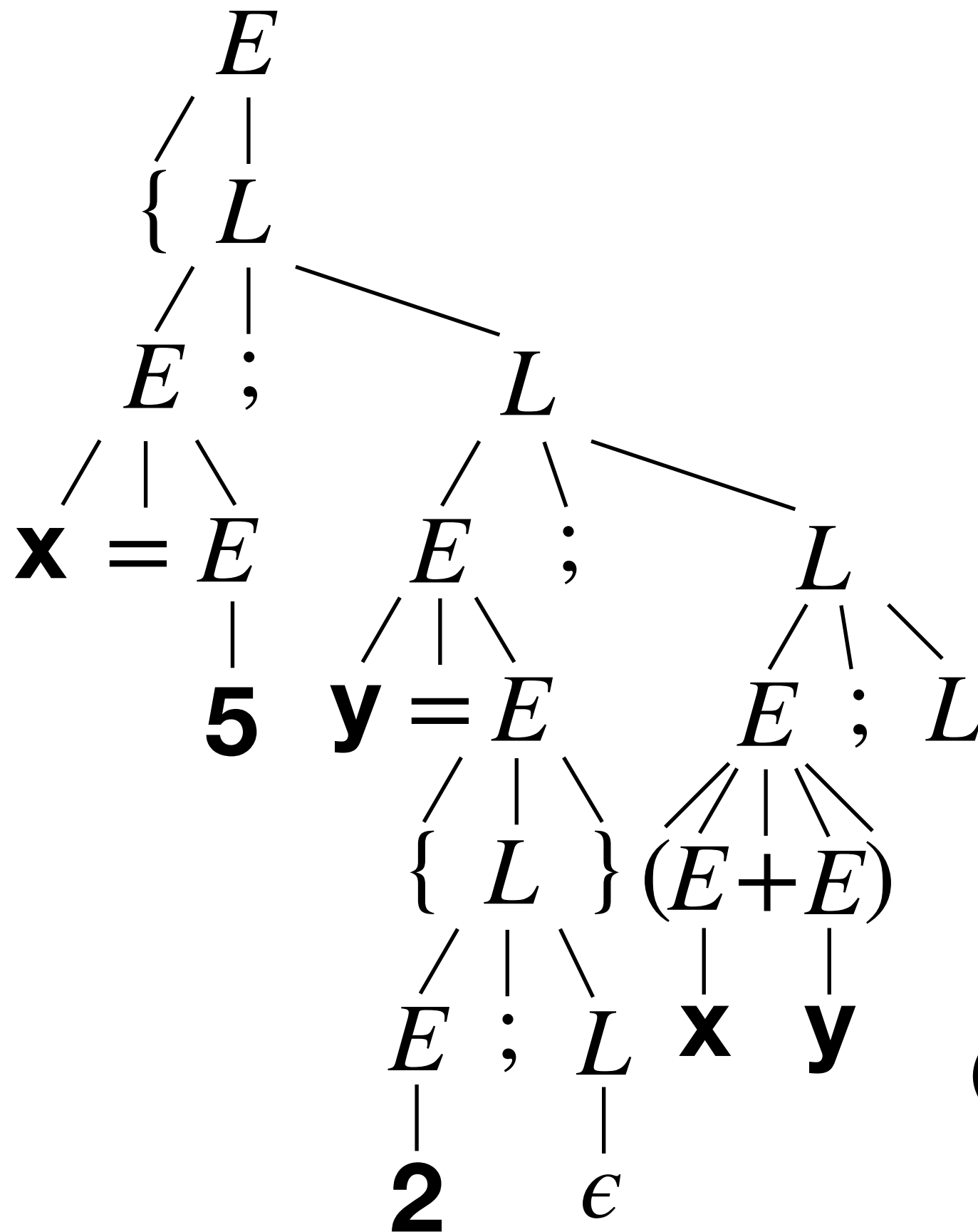


$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{ L \} \\
 \quad | n \\
 \quad | x
 \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

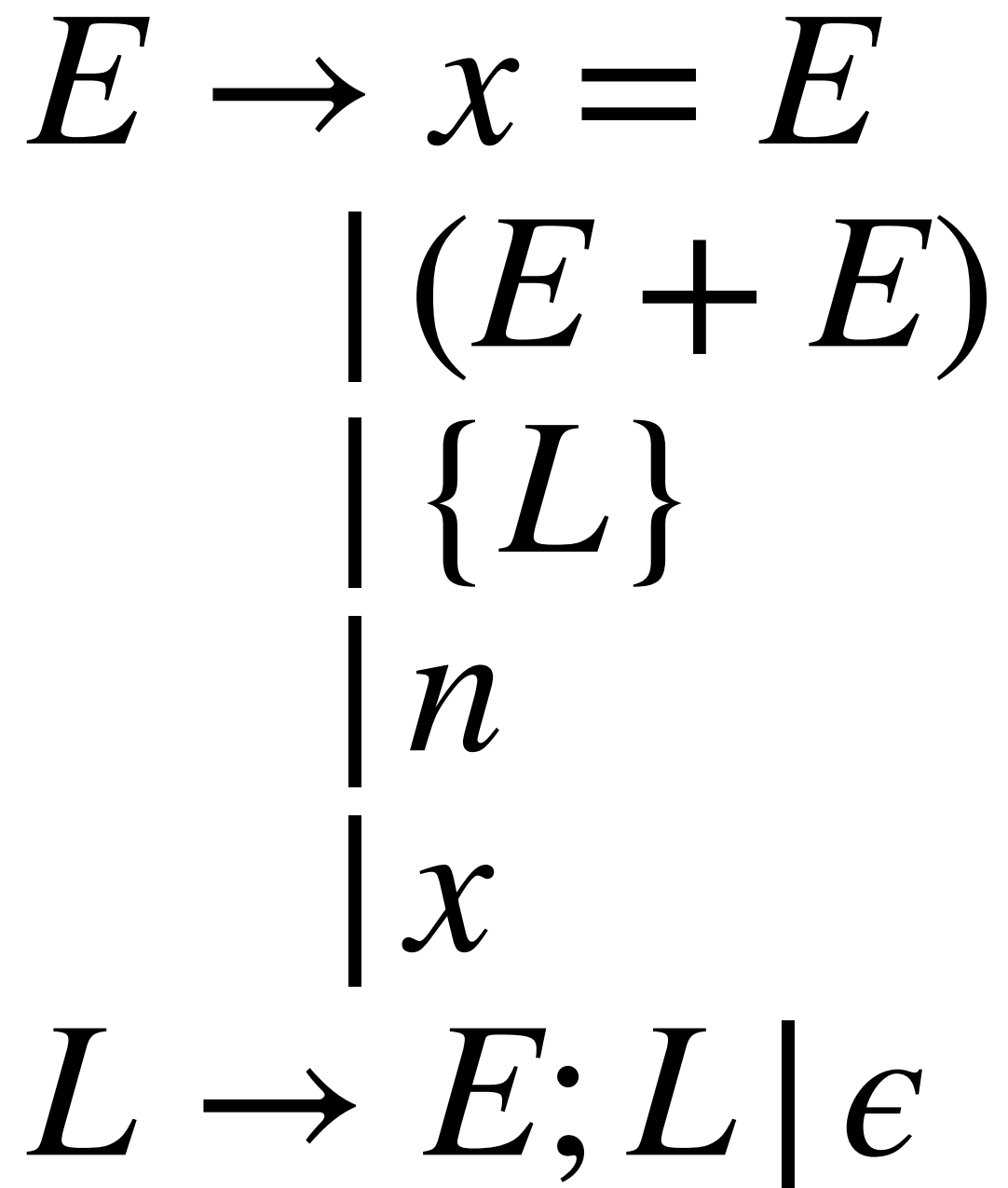
Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$



$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

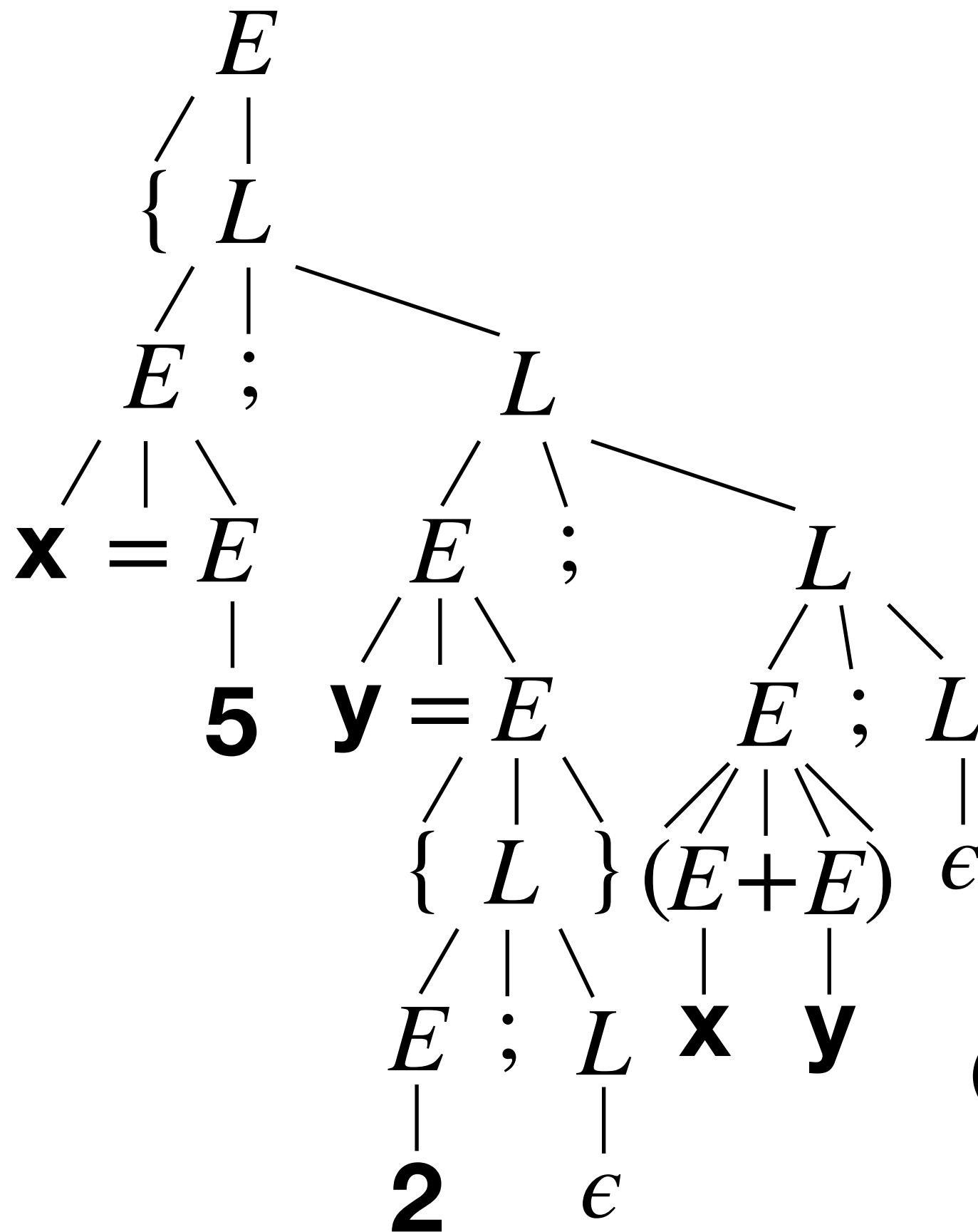
$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)



(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2\}; (x+y);\}$

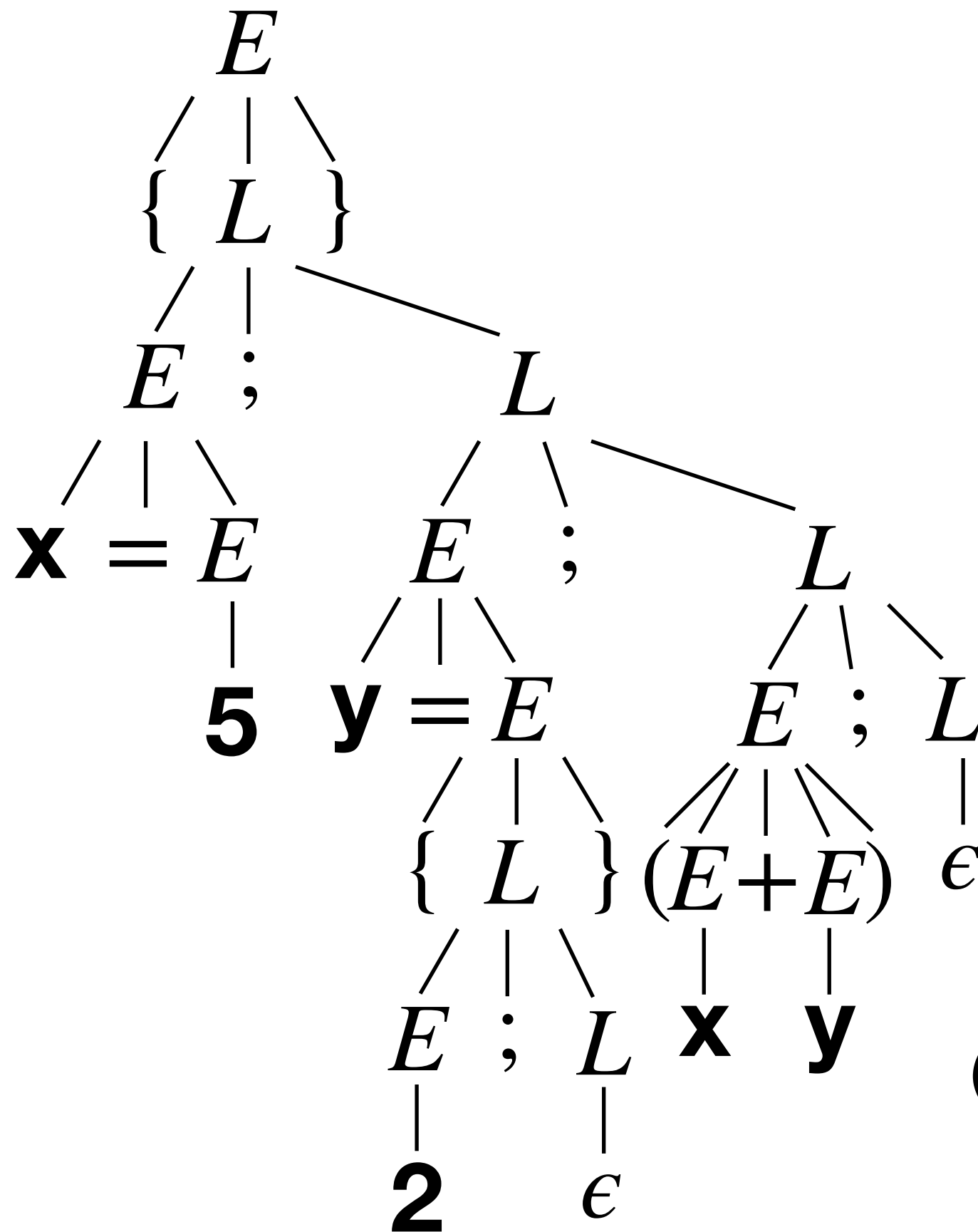


$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Recursive descent parsing for $\{x = 5; y = \{2;\}; (x+y);\}$



$$\begin{aligned}
 E &\rightarrow x = E \\
 &\quad | (E + E) \\
 &\quad | \{L\} \\
 &\quad | n \\
 &\quad | x
 \end{aligned}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

***First* sets for predictive parsing**

***First* sets for predictive parsing**

- First(s) is the set of possible first tokens in all strings that sentence s may expand to.
 - If s is a sentence of only terminals, $a_0a_1\dots a_n$, First(s) is a_0 .
 - If s begins with a terminal, $a_0(a_1|N_1)\dots$, First(s) is a_0 .
 - If s is a nonterminal N followed by s', $s = Ns'$, then there are two cases. If First(N) contains ϵ then First(Ns') is $(\text{First}(N) - \{\epsilon\}) \cup \text{First}(s')$. If not, then $\text{First}(Ns') = \text{First}(N)$.
 - If s is a nonterminal N, then $\text{First}(N) = \text{First}(s_0) \cup \text{First}(s_1) \cup \dots \cup \text{First}(s_k)$ where s_0 through s_k are the RHSs of the productions for nonterminal N.

Try an example. What is $\text{First}(\{L\})$?

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is $\text{First}(\{L\})$?

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

ANSWER: $\{\{\}\}$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is First(E)?

$$\begin{array}{l} E \rightarrow x = E \\ | (E + E) \\ | \{L\} \\ | n \\ | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is First(E)?

$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

ANSWER: $\{x, n, \{, (\}$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is First(L)?

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is First(L)?

$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

ANSWER: $\{x, n, \{, (, \epsilon\}$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is $\text{First}(L)$?

$$\begin{array}{l} E \rightarrow x = E \\ \quad | (E + E) \\ \quad | \{L\} \\ \quad | n \\ \quad | x \end{array}$$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Try an example. What is $\text{First}(L)$?

$$\begin{array}{l}
 E \rightarrow x = E \\
 \quad | (E + E) \\
 \quad | \{L\} \\
 \quad | n \\
 \quad | x
 \end{array}$$

ANSWER: $\{x, n, \{, (, \}\}$

$$L \rightarrow E; L \mid \epsilon$$

(where x is an ID, and n is a NAT)

Implementing recursive descent parsers

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.
 - We might call a function `expect_token(tok)` which consumes `tok` and progresses or reports failure.

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.
 - We might call a function `expect_token(tok)` which consumes `tok` and progresses or reports failure.
- Each nonterminal `N` becomes a call to `parse_N`:

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.
 - We might call a function `expect_token(tok)` which consumes `tok` and progresses or reports failure.
- Each nonterminal `N` becomes a call to `parse_N`:
 - `parse_N` uses `lookahead` to select a production.

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.
 - We might call a function `expect_token(tok)` which consumes `tok` and progresses or reports failure.
- Each nonterminal `N` becomes a call to `parse_N`:
 - `parse_N` uses `lookahead` to select a production.
 - With a RHS selected, each symbol is parsed in sequence.

Implementing recursive descent parsers

- Terminals reached in the grammar become a check against the current *lookahead* and then immediate success or failure.
 - We might call a function `expect_token(tok)` which consumes `tok` and progresses or reports failure.
- Each nonterminal `N` becomes a call to `parse_N`:
 - `parse_N` uses `lookahead` to select a production.
 - With a RHS selected, each symbol is parsed in sequence.
- Parsing begins with a call to `parse_S` (if `S` is start symbol).

Unsuitable CFGs for predictive parsing

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).
- To fix this, we can ***left-factor*** the grammar:

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).
- To fix this, we can **left-factor** the grammar:
 - Given a grammar: $N \rightarrow x\alpha_0 \mid x\alpha_1 \mid \dots \mid x\alpha_k \mid \beta$

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).
- To fix this, we can **left-factor** the grammar:
 - Given a grammar: $N \rightarrow x\alpha_0 \mid x\alpha_1 \mid \dots \mid x\alpha_k \mid \beta$
 - Refactor to: $N \rightarrow xN' \mid \beta \qquad N' \rightarrow \alpha_0 \mid \dots \mid \alpha_k$

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).
- To fix this, we can **left-factor** the grammar:

- Given a grammar: $N \rightarrow x\alpha_0 \mid x\alpha_1 \mid \dots \mid x\alpha_k \mid \beta$

- Refactor to: $N \rightarrow xN' \mid \beta$ $N' \rightarrow \alpha_0 \mid \dots \mid \alpha_k$

- This is basically the same as:

$$N \rightarrow x(\alpha_0 \mid \alpha_1 \mid \dots \mid \alpha_k) \mid \beta$$

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when the First sets for two RHSs in the grammar are not disjoint (have some tokens in common).
- To fix this, we can **left-factor** the grammar:

- Given a grammar: $N \rightarrow x\alpha_0 \mid x\alpha_1 \mid \dots \mid x\alpha_k \mid \beta$

- Refactor to: $N \rightarrow xN' \mid \beta$ $N' \rightarrow \alpha_0 \mid \dots \mid \alpha_k$

- This is basically the same as:

$$N \rightarrow x(\alpha_0 \mid \alpha_1 \mid \dots \mid \alpha_k) \mid \beta$$

- Left-factoring reduces the need for lookahead; or, put another way, articulates *each character* of lookahead needed to determine the next nonterminal to parse.

A simple recursive-descent parser

$$S \rightarrow n + S \mid n - S \mid n$$

Left factored: $S \rightarrow nS' \qquad S' \rightarrow + S \mid - S \mid \epsilon$

A simple recursive-descent parser

$$S \rightarrow n + S \mid n - S \mid n$$

Left factored: $S \rightarrow nS'$ $S' \rightarrow + S \mid - S \mid \epsilon$

```
def parse_S():
    if is_nat(peek(0)):
        expect_token(peek(0))
        if peek(0) == "+":
            expect_token("+")
            parse_S()
        elif peek(0) == "-":
            expect_token("-")
            parse_S()
        # epsilon is fine (the third production of S matches only n)
    else:
        error("Expecting a natural number.")
```

Check production nS'
Consume n from nS'
Select production +S
Recursively parse an S
Select production -S
Recursively parse an S
No RHS matches! Error.

Unsuitable CFGs for predictive parsing

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.
- If we wrote a recursive-descent parser for a left-recursive grammar, the parser could loop forever without progress!

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.
- If we wrote a recursive-descent parser for a left-recursive grammar, the parser could loop forever without progress!
- To fix this, we can replace left-recursion with right-recursion.

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.
- If we wrote a recursive-descent parser for a left-recursive grammar, the parser could loop forever without progress!
- To fix this, we can replace left-recursion with right-recursion.
 - Given a grammar: $N \rightarrow N\alpha_0 \mid N\alpha_1 \mid \dots \mid N\alpha_k \mid \beta$

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.
- If we wrote a recursive-descent parser for a left-recursive grammar, the parser could loop forever without progress!
- To fix this, we can replace left-recursion with right-recursion.
 - Given a grammar: $N \rightarrow N\alpha_0 \mid N\alpha_1 \mid \dots \mid N\alpha_k \mid \beta$
 - Refactor to: $N \rightarrow \beta N' \quad N' \rightarrow \alpha_0 N' \mid \dots \mid \alpha_k N' \mid \epsilon$

Unsuitable CFGs for predictive parsing

- Predictive parsing can fail when a production can recur immediately, on the left, without consuming any tokens.
- If we wrote a recursive-descent parser for a left-recursive grammar, the parser could loop forever without progress!
- To fix this, we can replace left-recursion with right-recursion.
 - Given a grammar: $N \rightarrow N\alpha_0 \mid N\alpha_1 \mid \dots \mid N\alpha_k \mid \beta$
 - Refactor to: $N \rightarrow \beta N' \quad N' \rightarrow \alpha_0 N' \mid \dots \mid \alpha_k N' \mid \epsilon$
- Right-recursion can then be implemented as a loop. (Consider how right-recursion corresponds to Kleene star!)

Regular grammars: tail-recursion corresponds to a loop

Regular grammars: tail-recursion corresponds to a loop

- A *regular grammar* is a CFG where every production:

Regular grammars: tail-recursion corresponds to a loop

- A **regular grammar** is a CFG where every production:
 - Has only terminals on the RHS: $N \rightarrow x_0 \dots x_k$
 - Or, has an empty RHS: $N \rightarrow \epsilon$

Regular grammars: tail-recursion corresponds to a loop

- A **regular grammar** is a CFG where every production:
 - Has only terminals on the RHS: $N \rightarrow x_0 \dots x_k$
 - Or, has an empty RHS: $N \rightarrow \epsilon$
 - Or, has only one nonterminal in tail position:

$$N \rightarrow x_0 \dots x_k N'$$

Regular grammars: tail-recursion corresponds to a loop

- A *regular grammar* is a CFG where every production:
 - Has only terminals on the RHS: $N \rightarrow x_0 \dots x_k$
 - Or, has an empty RHS: $N \rightarrow \epsilon$
 - Or, has only one nonterminal in tail position:

$$N \rightarrow x_0 \dots x_k N'$$

- *Regular* grammars always generate *regular* languages! All regular languages can be written as regular grammars.

Regular grammars: tail-recursion corresponds to a loop

- A *regular grammar* is a CFG where every production:
 - Has only terminals on the RHS: $N \rightarrow x_0 \dots x_k$
 - Or, has an empty RHS: $N \rightarrow \epsilon$
 - Or, has only one nonterminal in tail position:

$$N \rightarrow x_0 \dots x_k N'$$

- *Regular* grammars always generate *regular* languages! All regular languages can be written as regular grammars.
- Tail-recursive productions correspond to a Kleene-star:

$$N \rightarrow x_0 \dots x_k N \qquad N \rightarrow (x_0 \dots x_k)^*$$

Regular grammars: tail-recursion is equivalent to a loop

$$S \rightarrow n + S \mid n - S \mid n$$

```
def parse_S():
    if is_nat(peek(0)):
        expect_token(peek(0))
        if peek(0) == "+":
            expect_token("+")
            parse_S()
        elif peek(0) == "-":
            expect_token("-")
            parse_S()
    else:
        error()
```

```
def parse_S():
    while True:
        if is_nat(peek(0)):
            expect_token(peek(0))
            if peek(0) == "+":
                expect_token("+")
            elif peek(0) == "-":
                expect_token("-")
            else:
                return
        else:
            error()
```

This is true of tail-recursion generally, in CFGs, in Python, in C, in Java. Compilers which support ***tail-call optimization*** will perform a similar optimization internally.

Try an example. Is grammar E suitable for top-down parsing?

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$T \rightarrow T * F | F$$

$$F \rightarrow (E) | n$$

(where $n \in \mathbb{N}$)

Try an example. Is grammar E suitable for top-down parsing?

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

No. E and T are left-recursive!

$$T \rightarrow T * F | F$$

$$F \rightarrow (E) | n$$

(where $n \in \mathbb{N}$)

Try an example. How can make our E grammar suitable?

$$E \rightarrow T$$

$$| E + T$$

$$| E - T$$

$$T \rightarrow T * F | F$$

$$F \rightarrow (E) | n$$

(where $n \in \mathbb{N}$)

Try an example. How can make our E grammar suitable?

$$E \rightarrow TS$$

$$S \rightarrow + TS \mid - TS \mid \epsilon$$

$$T \rightarrow FG$$

$$G \rightarrow * FG \mid \epsilon$$

$$F \rightarrow (E) \mid n$$

(where $n \in \mathbb{N}$)

Try an example. Show the parse tree for **5-3+1**?

$$E \rightarrow TS$$

$$S \rightarrow + TS \mid - TS \mid \epsilon$$

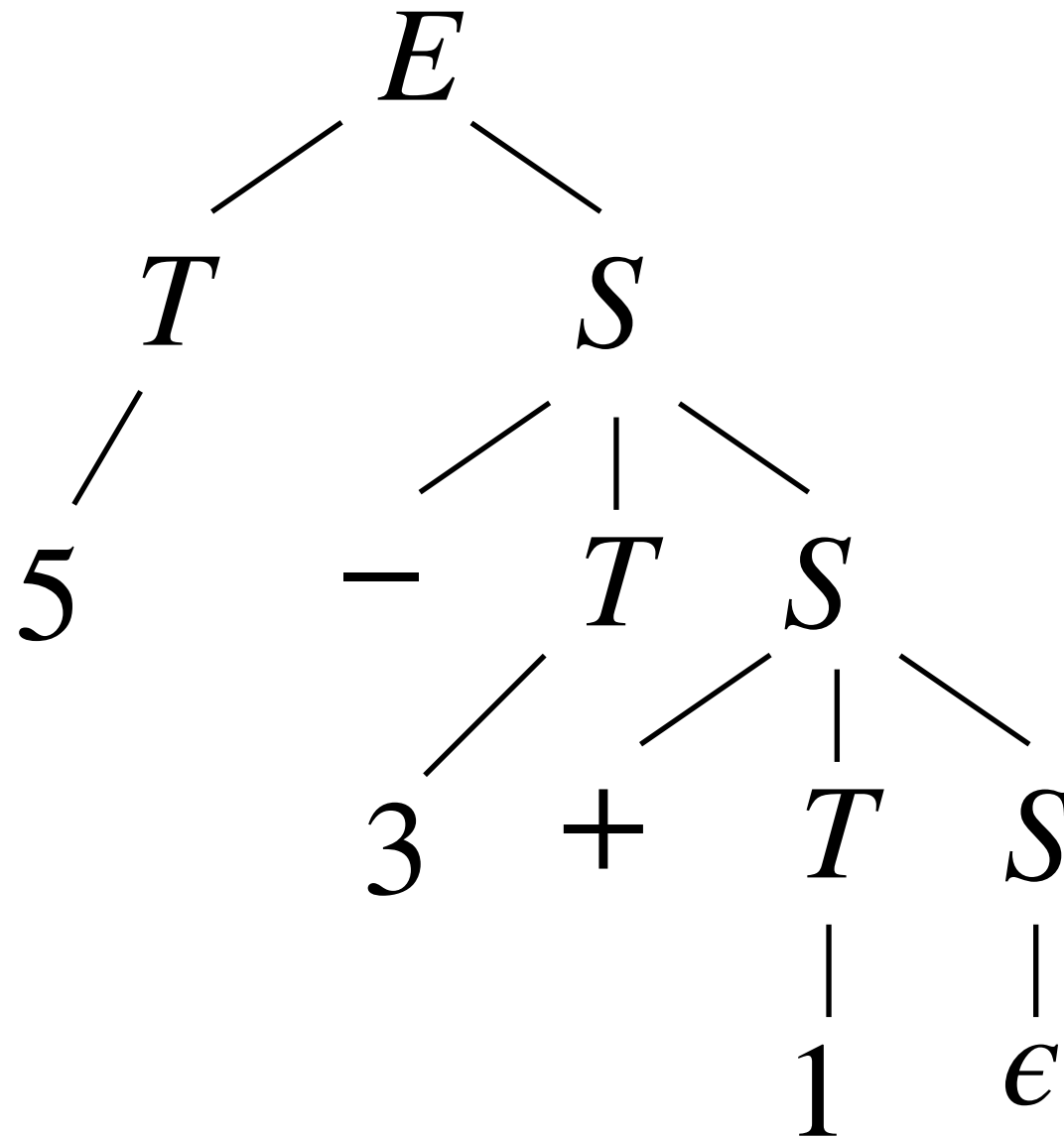
$$T \rightarrow FG$$

$$G \rightarrow * FG \mid \epsilon$$

$$F \rightarrow (E) \mid n$$

(where $n \in \mathbb{N}$)

Try an example. Show the parse tree for **5-3+1**?



Try an example. How can make our R grammar suitable?

$$\begin{array}{l} R \rightarrow R \mid R \\ \mid R R \\ \mid R^* \\ \mid (R) \\ \mid x \\ \mid () \end{array}$$

Try an example. How can make our R grammar suitable?

$$R \rightarrow J \\ | J | R$$

$$J \rightarrow K \\ | KJ$$

$$K \rightarrow P \\ | K^*$$

$$P \rightarrow (R) | x | ()$$

Try an example. How can make our R grammar suitable?

$$R \rightarrow J \\ | J | R$$

right-recursive and therefore
right-associative (not a problem for regex
disjunction, but may be a problem)

$$J \rightarrow K \\ | KJ$$

$$K \rightarrow P \\ | K^*$$

$$P \rightarrow (R) | x | ()$$

Try an example. How can make our R grammar suitable?

$$R \rightarrow J \\ | J | R$$

right-recursive and therefore
right-associative (not a problem for regex
disjunction, but may be a problem)

$$J \rightarrow K \\ | KJ$$

$$K \rightarrow P \\ | K^*$$

left-recursive, so K would infinite loop if we
attempt to directly translate this into a
recursive-descent, top-down, parser.

$$P \rightarrow (R) | x | ()$$

Try an example. How can make our R grammar suitable?

$$R \rightarrow JD$$

$$D \rightarrow |JD \mid \epsilon$$

$$J \rightarrow KU$$

$$U \rightarrow KU \mid \epsilon$$

$$K \rightarrow AS$$

$$S \rightarrow *S \mid \epsilon$$

$$A \rightarrow (R) \mid x \mid ()$$

Try an example. How can make our R grammar suitable?

Note that R is basically $\rightarrow$ regex $J(|J)^*$

$$R \rightarrow JD$$

$$D \rightarrow |JD \mid \epsilon$$

Note that J is basically $\rightarrow$ regex K^+

$$J \rightarrow KU$$

$$U \rightarrow KU \mid \epsilon$$

Note that K is basically $\rightarrow$ regex A^{**}

$$K \rightarrow AS$$

$$S \rightarrow *S \mid \epsilon$$

$$A \rightarrow (R) \mid x \mid ()$$

Try an example. What is First(A), First(K), First(D), and First(R)?

$$R \rightarrow JD$$

$$D \rightarrow JD \mid \epsilon$$

$$J \rightarrow KU$$

$$U \rightarrow KU \mid \epsilon$$

$$K \rightarrow AS$$

$$S \rightarrow *S \mid \epsilon$$

$$A \rightarrow (R) \mid x \mid ()$$

Try an example. What is First(A), First(K), First(D), and First(R)?

$$R \rightarrow JD$$

$$\text{First}(R) = \{ (, x \}$$

$$D \rightarrow |JD \mid \epsilon$$

$$\text{First}(D) = \{ |, \epsilon \}$$

$$J \rightarrow KU$$

$$\text{First}(J) = \{ (, x \}$$

$$U \rightarrow KU \mid \epsilon$$

$$\text{First}(U) = \{ (, x, \epsilon \}$$

$$K \rightarrow AS$$

$$\text{First}(K) = \{ (, x \}$$

$$S \rightarrow *S \mid \epsilon$$

$$\text{First}(S) = \{ *, \epsilon \}$$

$$A \rightarrow (R) \mid x \mid ()$$

$$\text{First}(A) = \{ (, x \}$$